Theory of JuGeo as an AG+DTT+AI Judgment Geometry Orderly Rewrite Draft `theory2.tex`

A reorganized theoretical companion to `theory.tex` and `plan.md`

March 20, 2026

Contents

Preface

This file is the beginning of an orderly rewrite of `theory.tex` under the cleaner scaffold recorded in `toc.tex`. The point is not to discard the existing monograph but to reorganize its content into a structure whose headings already tell the reader what semantic object is being treated, what mechanism is being proposed, and what proof or implementation burden follows from that treatment.

The governing thesis is unchanged. JuGeo is to be understood as a judgment geometry for program construction, verification, comparison, repair, orchestration, and mathematical ideation. The geometry is algebraic-geometric because locality, overlap, gluing, and obstruction are the right local-to-global notions. It is dependent-type-theoretic because judgments are always context-sensitive and evidence-bearing. It is AI-augmented because controlled proposal, analogical transfer, and frontier search are essential at project scale. What changes in this rewrite is presentation: the same content should become sharper, more implementable, and more orderly.

Part I

Thesis and Central Claim

Chapter 1

Introduction: What JuGeo is

JuGeo is not best understood as a theorem prover with an attached code generator, nor as a coding assistant with an attached verifier. It is a semantic machine whose primary state is a judgment-valued presheaf on a site of programmatic and mathematical coordinates. The objects of that site are not merely files. They are semantically meaningful regions: function contracts, implementation blocks, interface surfaces, tests, documentation clauses, relational comparison domains, proof obligations, orchestration frontiers, and theorem-search neighborhoods. Morphisms are lawful restrictions of context and support. A cover records a decomposition of a target region into semantically admissible local patches, while a hypercover records a recursively structured decomposition in which overlaps and higher overlaps themselves require local analysis before any global claim can be justified. This is the sense in which algebraic geometry is foundational rather than ornamental: the central operational question of JuGeo is always whether local sections descend.

The dependent-type-theoretic layer supplies the fibers over this site. At each coordinate U one has a context $\Gamma(U)$, a family of admissible claims over that context, and a notion of evidence-bearing inhabitance. The AI layer supplies controlled proposal and search over future semantic states: candidate local sections, candidate refinements of a cover, candidate overlap treaties, candidate auxiliary invariants, and candidate changes of coefficient theory or domain pack when the present language cannot trivialize an obstruction. The system is therefore AG+DTT+AI in a strict sense. Algebraic geometry gives the base site, covers, hypercovers, presheaves, sheaves, descent data, gluing laws, Čech cocycles, and obstruction classes. Dependent type theory gives context-sensitive judgment form and the discipline of evidence-bearing claims. AI gives nontrivial search over the space of local sections and semantic refinements. Remove any one of these three and the architecture no longer explains project-scale generation and verification.

Definition 1.1 (JuGeo as a judgment-geometry machine)**.** *A JuGeo instance is a tuple*

$$\mathfrak{D} = \big(\mathcal{S}_{\mathrm{proj}}, \Gamma, \mathscr{J}, \mathscr{E}, \mathscr{O}, \mathscr{D}, \mathsf{Prop}\big)$$

where $\mathcal{S}_{\mathrm{proj}}$ is a site of semantic coordinates, Γ is a context presheaf on that site, $\mathscr{J}$ is a presheaf of local dependent judgments, $\mathscr{E}$ is a presheaf of typed evidence bundles, $\mathscr{O}$ is a presheaf of residual obligations, $\mathscr{D}$ is a discrepancy presheaf on overlaps and higher overlaps, and Prop *is a controlled proposal operator that may emit local sections, refinements of covers or hypercovers, treaty candidates, or theory extensions, but may not by itself certify global closure.*

This definition should be read as an implementation directive. The semantic kernel must therefore materialize sites, covers, hypercovers, restriction maps, overlap objects, local sections, descent witnesses, discrepancy cocycles, obstruction records, and changes of cover as first-class

runtime objects. An implementation that stores only AST fragments, test results, solver answers, and agent transcripts has not yet built JuGeo in the sense of this dissertation, because it cannot state the local-to-global problem in the right ontology.

The global products of the system are H^0-like objects. A verified function, a certified repair, a coherent documentation alignment, a relational equivalence witness, and a theorem export are all global or scope-declared sections obtained after descent. Let $\mathfrak{U} = \{U_i \to U\}$ be an admissible cover of a target coordinate U. A family of local judgments $s_i \in \mathscr{J}(U_i)$ determines overlap discrepancies in $\mathscr{D}(U_i \times_U U_j)$ and hence a Čech 1-cocycle

$$\omega(s) \in \check{Z}^1(\mathfrak{U}, \mathscr{D}).$$

If $[\omega(s)] \in \check{H}^1(\mathfrak{U}, \mathscr{D})$ vanishes, then the failure of agreement is a coboundary and the system may seek new local representatives inside the present regime. If the class remains nontrivial, then no amount of local patching within the current cover and coefficient language will suffice: one must refine the cover, pass to a hypercover, strengthen the overlap treaty, enrich the invariant language, or change the relevant theory pack. In this precise sense, bugs and integration failures are obstruction classes, not metaphors.

Proposition 1.2 (Global-section criterion for publishable artifacts)**.** *Fix a target coordinate U together with an admissible cover or hypercover. A candidate artifact is fit to be reported as settled over U only if its local judgments form descent data, the associated discrepancy class in $\check{H}^1$ is trivial under the current agreement notion, and the residual obligations attached to the glued section are discharged by admissible evidence. If the obstruction class is nontrivial, the correct output is an obstruction object together with its support, provenance, and admissible change-of-cover or change-of-theory repair frontier.*

The proposition makes the public reporting standard sharp. JuGeo should never report closure merely because many local checks passed. It should report closure only when the local judgments glue and the declared scope matches the region on which descent has been achieved. This yields the thesis-level distinction between a compatible family of promising local sections and a genuinely global section. The former belongs to the search state; the latter may enter a certificate.

Algorithm sketch. For a target scope U, the runtime chooses an admissible cover or hypercover $\mathfrak{U}$ from the site topology. It elaborates the local dependent contexts $\Gamma(U_i)$, constructs local judgment candidates in $\mathscr{J}(U_i)$ using structural reasoning and controlled AI proposal, and attaches clausewise evidence in $\mathscr{E}(U_i)$. It then restricts these candidates to overlaps and higher overlaps, computes the induced Čech cochains, and checks whether the discrepancy cocycle is null, canonically resolvable, or persistent. In the first case it glues the local sections and emits a section of the coherent judgment sheaf over U. In the second case it records the additional descent data required for gluing. In the third case it emits an obstruction object and chooses among three typed moves: local repair within the present cover, a change of cover or hypercover, or a change of invariant language or domain pack. In this form JuGeo is a descent engine for judgment geometry, not a loose alliance of tools.

1.1 Judgment geometry as the semantic center

JuGeo should not be described as a verifier that later learned to generate code, nor as an AI generator that later learned to check itself. It should be described as a single semantic machine whose primary object is the judgment state of a project. That state records coordinates, clauses, evidence, residuals, obstructions, treaties, supports, and the admissible transitions between them. In

explicit AG language, the project is modeled over a site of semantic coordinates, local artifacts are sections of presheaves over that site, globally coherent artifacts are descended or sheafified sections, and persistent failures are obstruction classes attached to the chosen cover or hypercover. Once this state is made primary, verification, generation, repair, equivalence, and ideation become different operations on the same object rather than disconnected product features.

1.2 JuGeo relative to theorem provers, coding assistants, and agentic verifiers

Relative to traditional theorem provers, JuGeo aims to preserve one of their greatest strengths—explicit proof obligations and evidence provenance—while adding a local-to-global state model that ordinary proof scripts rarely maintain directly. Relative to coding assistants, it refuses to treat generated text as success merely because the text is plausible. Relative to ordinary agentic verifiers, it does not reduce orchestration to queueing tools over files. It instead gives the controller access to covers, overlap treaties, obstruction classes, and replay-local invalidation. These differences are not merely architectural taste. They imply concrete capabilities that traditional verification pipelines rarely expose in one place.

The strongest proof-backed advantages should be stated plainly. First, JuGeo can quantify repair complexity rather than merely report failure: on a fixed cover, the rank of the first obstruction space over a binary coefficient regime gives a lower bound, and in the exact POPL-style fragment an exact count, of the minimum number of independent fixes needed to remove all current incompatibilities. Second, equivalence is not left to heuristic regression evidence alone: local relational evidence glues to a global equivalence exactly when the corresponding descent obstruction vanishes on the chosen relational cover. Third, incremental analysis is algebraic rather than ad hoc: branch- or module-local recomputation can be combined by a Mayer–Vietoris style exact sequence, which means one can justify why a local change should force only local recomputation. Fourth, specification checking gains a product-cover discipline that factors large verification conditions into path-by-conjunct obligations and exposes when overlap transport can discharge one obligation from another. Together these are real improvements over traditional verification cultures that usually provide only local proof success, local counterexamples, or coarse incremental caches.

The computational consequence is that JuGeo must retain the objects needed to make these advantages executable: explicit covers and hypercovers, overlap matrices or treaty graphs, coboundary operators for the admitted exact fragments, gluing reports, and replay seals that say which local proof or solver results can survive change. Without those objects, the renamed system would only have new terminology. With them, JuGeo can say not only that something failed, but how many independent repairs appear necessary, whether a claimed equivalence is globally descended or only locally persuasive, and whether a local edit should or should not force global reopening.

1.3 The AG+DTT+AI thesis

The AG layer contributes the actual geometric backbone: a site of coordinates, admissible covers and hypercovers, presheaves of candidate sections, sheaves of coherent sections, Čech complexes for tracking local-to-global compatibility, and obstruction classes in degree 1 and above for recording when descent fails. The DTT layer contributes contexts, dependent claims, inhabitants, and residual obligations. The AI layer contributes controlled proposal, semantic search, and novelty over future semantic states. The claim of the thesis is that only the combination of these three is strong enough for long-codebase generation, verification, and mathematical discovery without collapsing into either

proof fetishism or prompt theater. In particular, the thesis should state unambiguously that global software artifacts are H^0-like objects, bugs and integration failures are often H^1 obstructions, and some persistent multi-region inconsistencies demand higher descent data rather than better local cleverness.

1.4 Main contributions

The dissertation claims five central contributions. First, it offers a common judgment geometry for software semantics, generation, repair, equivalence, orchestration, and mathematical ideation, and states that geometry explicitly in sheaf-theoretic and cohomological terms rather than by metaphor alone. Second, it develops a mixed-evidence discipline in which proofs, solver discharge, runtime witnesses, and controlled semantic judgments can coexist without being conflated. Third, it gives a project-scale account of large-codebase generation in terms of covers, hypercovers, descent data, obstruction classes, and support-aware closure rather than prompt sequencing alone. Fourth, it develops an account of theorem growth and mathematical discovery as purpose-conditioned search over future semantic state, including changes of cover, changes of coefficient theory, and changes of domain pack when obstruction classes persist. Fifth, it insists that all of these claims be implementation-guiding: every major theoretical object should induce authority centers, compiled views, invalidation rules, and theorem or testing obligations in code.

1.5 Problem classes addressed

The problem classes addressed by the thesis are intentionally broad because the central claim is that they are all special cases of one semantic machine. They include specification satisfaction, bug finding, equivalence and refinement, repair and transformation, documentation alignment, migration, regression closure, public-surface honesty, performance reasoning, concurrency and distributed failures, large-scale code generation, and purpose-directed mathematical ideation. The thesis does not claim that all of these are solved to the same maturity level. It claims that they can all be placed inside one judgment language strongly enough that progress in one class can improve the others.

Chapter 2

Research questions and thesis claims

The research questions of this dissertation are not invitations to metaphorical synthesis. They are questions about whether one can build, analyze, and falsify a concrete AG+DTT+AI machine of the kind just defined. Each major claim must therefore determine a semantic object, an executable algorithm, a theorem target, and an empirical failure mode. If a chapter cannot say what presheaf, sheaf, cover, hypercover, cocycle, obstruction class, or change-of-cover operation it contributes to the implementation, then that chapter has not yet earned its place in the monograph. The central research question is whether project-scale software and theorem work can be organized by a site-indexed judgment geometry whose local states are presheaf sections, whose successful products are descended global sections, whose failures are recorded as obstruction classes, and whose exploratory intelligence is supplied by controlled AI operating inside typed jurisdiction.

The intended operational heart of the system is a descent procedure

$$\mathsf{Desc}_{\mathfrak{U}} : \check{C}^0(\mathfrak{U}, \mathscr{J}) \longrightarrow \mathscr{J}^{\mathrm{coh}}(U) \sqcup \check{H}^1(\mathfrak{U}, \mathscr{D}),$$

together with companion procedures for restriction, gluing, challenge, change of cover, hypercover refinement, and theory extension. The representation question asks whether all principal subsystems can export into the same judgment presheaf without losing the distinctions that matter. The mixed-evidence question asks whether the evidence presheaf can preserve proof, solver discharge, runtime witness, and controlled semantic judgment as distinct support channels while still allowing lawful gluing. The orchestration question asks whether frontier management should be implemented as control over changing descent state rather than as file-level task scheduling. The ideation question asks whether new mathematical regimes should be proposed in response to persistent obstruction classes and evaluated by their ability to trivialize those classes under admissible transport. The point is that these are not four unrelated topics. They are four slices through the same local-to-global machine.

Proposition 2.1 (Core thesis package)**.** *The dissertation advances the following package of claims. First, there exists a finite site $\mathcal{S}_{\mathrm{proj}}$ of semantic coordinates together with admissible covers and hypercovers rich enough to encode the principal decomposition regimes of large-codebase work, and the main subsystems of the runtime can be forced to export into presheaves over that site. Second, for every finite admissible cover or hypercover there is a terminating descent algorithm that returns either a glued section or an explicit obstruction representative, with theorem targets of restriction functoriality, descent soundness, and no-silent-promotion of trust. Third, change-of-cover operators can be made semantically conservative and operationally useful: when support changes on a subregion, reopening can be localized to the star of that support under the current cover and can often be reduced further after refinement to a better cover or hypercover. Fourth, controlled AI can be restricted to*

typed proposal roles—local section proposal, overlap-treaty proposal, cover refinement, hypercover refinement, and theory-pack extension—without losing the search power needed for realistic project work. Fifth, persistent nontrivial classes in degree 1 *are informative rather than merely negative: they identify precisely those situations in which local patching is inadequate and where new invariants, new decompositions, or new domain mathematics are required.*

This proposition fixes the theorem burden of the dissertation. One theorem target is exactness in degree 0: compatible local judgments that satisfy the declared descent condition must glue to a coherent section, and every coherent section must restrict to compatible local data. Another is faithful failure in degree 1: when gluing fails, the runtime should report a discrepancy cocycle whose class witnesses that failure rather than collapsing everything into an undifferentiated error token. Another is conservativity under changes of cover: if $\pi : \mathfrak{V} \to \mathfrak{U}$ is a refinement, then sections already coherent on $\mathfrak{U}$ should remain coherent on $\mathfrak{V}$, while some previously opaque discrepancies may become local and hence repairable. Another is support-local invalidation: if an evidence change is supported on $V \subseteq U$, then reopening should be bounded by the overlap neighborhood determined by the active cover, and the bound should improve under better decompositions. A final target is proposal conservativity: AI may enlarge the search space of local sections and refinements, but it may not itself count as a descent witness unless its outputs are subsequently discharged by the admissible evidence disciplines of the affected clauses.

The implementation consequences are equally concrete. The repository should expose canonical data types for coordinates, covers, hypercovers, restrictions, sections, local evidence bundles, discrepancy cocycles, obstruction classes, change-of-cover maps, and proposal objects. The orchestrator should schedule work by expected reduction of unresolved obstruction mass, expected increase in coherent H^0-like sections, and expected improvement in the chosen cover, not by prompt count or file count. Certificate emission should depend on the descended section actually achieved, together with the scope on which it was achieved, and should never widen that scope by rhetoric. Ideation should begin when persistent obstruction classes survive local repair attempts; it should then search for imported lemmas, new invariants, alternative covers, or theory-pack changes that could turn a nontrivial Čech class into a coboundary.

Algorithm sketch. The research program can be realized by an outer control loop over a changing semantic atlas. At each epoch the runtime maintains a site, a current cover or hypercover for each active target, a presheaf of local judgments, a presheaf of evidence bundles, and a discrepancy presheaf on overlaps. It runs local checking and proposal to populate $\check{C}^0$, computes discrepancy cocycles in $\check{C}^1$, and applies descent. When a global section is produced, it updates the coherent judgment sheaf and emits only the corresponding scope-honest certificate. When a nontrivial obstruction persists, it invokes a typed choice among local repair, change of cover, hypercover refinement, or theory extension. The AI layer is evaluated by whether its proposals lower the measured obstruction burden and increase successful gluing under the same honesty constraints, not by fluency or novelty alone.

The claims are falsifiable in a strong engineering sense. If the common site-and-presheaf representation proves too cumbersome to support incremental recomputation, the representational thesis fails. If Čech-style discrepancy tracking does not localize real integration failures more sharply than coarser dependency heuristics, the AG thesis is weakened. If support-local reopening after a change of cover is not materially smaller than naive reopening, the local-to-global implementation claim fails. If typed AI proposal cannot improve descent success, reduce persistent obstruction classes, or discover useful new invariants more reliably than flatter prompt orchestration, the AI part of the thesis fails. If certificates regularly claim more scope than the descended sections warrant,

the public-honesty theorem target fails. These are not peripheral evaluation criteria. They are the conditions under which the dissertation should be believed or rejected.

2.1 Representation

The representational question is whether a coordinate-indexed judgment geometry is a better authority center than the more common alternatives such as isolated type annotations, static-analysis fact stores, theorem-prover contexts, or generic agent memory. "Better" here means more faithful public reporting, more reusable internal state across tasks, more stable incrementality under change, and clearer separation among proof, heuristic support, empirical witness, and residual uncertainty. The thesis prediction is that once all major subsystems export into one common language of coordinates, clauses, evidence, supports, and obstructions, the architecture gains a kind of semantic compositionality unavailable to siloed systems. In AG terms, the gain is that the state is no longer a bag of facts but a representable local-to-global object: one can ask for sections, restrictions, descent data, and obstruction classes instead of inventing ad hoc analogues in each subsystem.

2.2 Mixed evidence

The mixed-evidence question is whether exact structural reasoning and controlled semantic reasoning can cooperate without collapsing into each other. The thesis answer is that they can, but only if evidence families preserve their identity through the whole pipeline. A solver-backed clause must remain solver-backed when it reaches a public certificate. A controlled-oracle clause must remain explicitly semantic and challengeable. A runtime witness must remain scoped to the environment and trace family from which it was drawn. The implementation consequence is that evidence tags, provenance, and transport conditions are foundational data, not renderer decorations.

2.3 Long-horizon orchestration

The orchestration question is whether long-codebase work can be treated as semantic control over a changing judgment state rather than as mere workflow management over files and tools. The thesis says yes. The controller should optimize semantic leverage, treaty stability, and closure progress rather than local activity counts. The implementation consequence is that the orchestrator must own explicit frontier state, backpressure signals, challenge backlogs, replay risk, and support-local reopening behavior. Without that semantic state, "orchestration" degenerates into queueing prompts and commands.

2.4 Mathematical ideation

The ideation question is whether theorem mining and mathematical discovery can be purpose-conditioned strongly enough to be useful for software work rather than decorative. The thesis answer is that they can, provided the system searches over theory space in response to structured obstruction pressure, evaluates candidate regimes by expected semantic leverage and transportability, and archives both successful and failed explorations. This is the point where AG, DTT, and AI most visibly interact: the obstruction field defines where the pressure lies, dependent judgment

form defines what a new regime would need to express, and controlled AI proposes candidate mathematical moves inside that discipline.

2.5 Falsifiability

The monograph is only worth taking seriously if it is falsifiable. It would be falsified or at least weakened if the common judgment geometry proved too expensive to maintain, if mixed evidence routinely confused users rather than clarifying support, if support-aware invalidation could not be made materially more local than coarse text-based reopening, if the orchestrator could not outperform flatter prompt-and-tool baselines on long-horizon coherence, or if the mathematical-ideation layer consistently preferred elegant but low-leverage regimes. The code rehaul should therefore be built with the expectation that these claims may succeed, fail, or require revision under measurement.

Part II

Foundations of Judgment Geometry

Chapter 3

What a JuGeo type is

The first foundational task of the orderly rewrite is to state the semantic object that later chapters are actually about. A JuGeo type should not be treated as an annotation plus some external checking machinery. It should be treated as a coordinate-indexed admissibility object whose components already include the conditions for transport, gluing, and evidence. More sharply, the thesis should say that these admissibility objects live over a site of programmatic coordinates, that candidate realizations form presheaves, and that genuinely coherent realizations should satisfy a sheaf condition on the admitted covers. This is the point at which the AG and DTT layers stop being background rhetoric and become executable design constraints.

3.1 From ordinary annotations to coordinate-indexed admissibility objects

An ordinary annotation says too little for the system JuGeo is trying to become. A JuGeo type should instead be treated as a coordinate-indexed admissibility object with carrier structure, structural and semantic law families, restriction behavior, gluing behavior, and evidence policy. In explicit AG terms, one should distinguish a carrier presheaf $\mathscr{A}_T$ of candidate artifacts from a subpresheaf $\mathscr{P}_T$ of admissible local sections and, when the relevant laws are fully respected, a sheaf $\mathscr{S}_T$ of coherent sections. This definition is not rhetorical. It tells the implementation what kinds of records must exist if one wants the same notion of type to serve verification, generation, equivalence, repair, and ideation.

3.2 Coordinates: where context, support, and meaning become precise

The coordinate is the place where the system chooses what claim is being made, under which context, and over which support. Coordinates may refer to functions, paths, class surfaces, overlap regions, paired artifacts, or frontier states. The AG content should be stated directly: coordinates are the objects of a site $\mathcal{S}$, and admissible covers in the Grothendieck topology say which decompositions count as semantically legitimate. The implementation consequence is that coordinate constructors and normalizers are foundational code, not bookkeeping; they are the code-level realization of the base site on which the semantic sheaves live.

3.3 Carrier, laws, transport, gluing, and evidence policy

A JuGeo type is not exhausted by its carrier. The carrier A_c at coordinate c says what local artifacts are even eligible to be discussed there, but admissibility is determined by more than membership. The type must also declare its structural law family Φ_c^{str}, its semantic law family Φ_c^{sem}, its restriction discipline ρ_c, its gluing discipline γ_c, and its evidence policy ϵ_c. Without those additional components, the implementation cannot know what it means to move a judgment to a smaller coordinate, to assemble local judgments into a larger one, or to decide whether a runtime witness, a solver discharge, or an oracle judgment is admissible support.

The sheaf-theoretic sharpening is that restriction should define a contravariant action on admissible local sections. If $V \to U$ is a morphism in $\mathcal{S}$, then restriction should induce

$$\mathrm{res}_{U,V} : \mathscr{P}_T(U) \to \mathscr{P}_T(V),$$

and the attained fragment should satisfy descent on admitted covers:

$$\mathscr{S}_T(U) \;\cong\; \ker\Big(\prod_i \mathscr{S}_T(U_i) \rightrightarrows \prod_{i,j} \mathscr{S}_T(U_i \times_U U_j)\Big).$$

This is the mathematically explicit form of what later chapters call gluing. A verified artifact is a global section only when the descent condition is met. A merely promising artifact is a compatible family of local sections that may still fail to descend globally.

Definition 3.1 (Coordinate-indexed admissibility object)**.** *For a coordinate c, a JuGeo type is a tuple*

$$T(c) = \big(A_c, \Phi_c^{\mathrm{str}}, \Phi_c^{\mathrm{sem}}, \rho_c, \gamma_c, \epsilon_c\big)$$

where A_c is the local carrier, Φ_c^{str} is the structural law family, Φ_c^{sem} is the semantic law family, ρ_c specifies lawful restriction to subordinate coordinates, γ_c specifies admissible gluing data and gluing criteria, and ϵ_c specifies which forms of evidence may support which classes of claims at c.

This definition has an immediate implementation consequence. The core repository should not treat types as annotations and then bolt on separate mechanisms for transport, integration, and evidence classification. Those mechanisms belong to the type layer itself. A function guarantee, a relational comparison target, a documentation clause, and an orchestration frontier item all differ partly because their carriers differ, but also because their transport laws, gluing criteria, and admissible evidence differ.

Proposition 3.2 (Type-directed transport discipline)**.** *If $d \to c$ is a lawful restriction in the coordinate system, then any judgment admissible at c may be transported to a judgment candidate at d only through the restriction discipline ρ_c, and the resulting trust profile may strengthen only when the local policy explicitly permits such strengthening. In particular, restriction is not allowed to erase residual obligations or invent new support.*

Algorithm sketch. The implementation should realize this definition through canonical records. A coordinate builder constructs c. A type elaborator computes the structural and semantic law families applicable at c. A restriction procedure narrows the carrier, evidence bundle, and obligation set along declared coordinate morphisms. A gluing procedure takes a finite family of locally typed judgments and either emits a glued section together with remaining residuals or emits a typed obstruction family. The evidence policy is consulted at every aggregation point rather than only at certificate emission time.

Chapter 4

Locality, transport, gluing, and obstruction

The second foundational task is to explain why locality and gluing are the true center of the system. A long codebase, a proof-carrying interface, a donor migration, and a documentation alignment pass are all local-to-global problems. The implementation therefore needs first-class notions of covers, overlaps, gluing witnesses, and obstructions, not only dependency graphs and file trees. More explicitly, it needs enough sheaf theory to talk about descent data and enough cohomology to talk about the failure of descent.

4.1 Local-to-global structure: covers, overlaps, and gluing criteria

The foundational local-to-global claim is that software reasoning becomes truly project-scale only when the system can say what counts as a local section, what counts as an overlap, and what counts as successful gluing. A local section is not just a local proof or local code artifact. It is a judgment-bearing artifact with explicit support and transport discipline. An overlap is not just a shared symbol name. It is a region in which two local sections must agree on exported laws, interface assumptions, or behavioral summaries. Gluing is therefore a witness-producing operation, not a hopeful concatenation of locally plausible facts.

The implementation consequence is that the runtime should materialize cover objects, overlap objects, and gluing results explicitly. A gluing attempt should return either a larger section together with its support and derived obligations, or an obstruction object explaining which overlap laws failed and under which assumptions. This is one of the main places where the AG layer becomes executable rather than metaphorical.

For a chosen cover $\mathfrak{U} = \{U_i \to U\}$ and a discrepancy or obstruction presheaf $\mathscr{D}_T$, the right bookkeeping object is the Čech complex

$$\check{C}^0(\mathfrak{U}, \mathscr{P}_T) = \prod_i \mathscr{P}_T(U_i), \qquad \check{C}^1(\mathfrak{U}, \mathscr{D}_T) = \prod_{i,j} \mathscr{D}_T(U_i \times_U U_j),$$

together with the usual coboundary δ^0. A family of local sections $\{s_i\}$ glues exactly when its overlap discrepancy cocycle vanishes or is a trivial coboundary under the relevant notion of agreement. This is the right mathematical meaning of "local correctness does not imply global closure."

4.2 Covers and hypercovers

A plain cover is enough when the decomposition is shallow: a function split into path regions, a specification split into clauses, or a pairwise comparison split into input partitions. A hypercover is needed when the decomposition itself has internal depth: a project split into modules, each module split into interfaces and implementation regions, each interface split into clause families and domain-specific sublanguages. The gain is not formal sophistication for its own sake. The gain is that gluing obligations can be staged rather than attempted all at once.

For project-scale elaboration this matters directly. A large codebase should be decomposed semantically before it is decomposed organizationally. One wants a hypercover whose local pieces have manageable carriers, whose overlaps are informative rather than accidental, and whose higher-order overlaps are sparse enough that global assembly is not drowned in hidden interface debt. Hypercover choice is therefore an optimization problem over future gluing cost, not a clerical partition of files.

Algorithm sketch. A practical cover synthesizer should score candidate decompositions by expected overlap complexity, expected obligation localization quality, and expected invalidation savings under revision. It should emit not only a chosen cover but also an explanation trace: which interfaces determined the overlap graph, which hidden dependencies were detected, which regions remain semantically risky, and why the chosen decomposition is expected to make later gluing easier.

The cohomological payoff of hypercovers should be named rather than left implicit. Pairwise disagreement lives in degree 1, but project-scale failures often persist only on triple and higher overlaps: treaty nontransitivity, benchmark-interface-runtime triangles, or multi-module resource invariants that are locally satisfiable in every pair but not globally. Hypercovers are what make these higher compatibility conditions visible to the engine. In principle, higher Čech classes record where the present decomposition and present law language fail to support descent beyond pairwise checks.

4.3 Obstructions as the common language of failure

Obstruction should be the common result language for failures of extension, agreement, or transport. A failed solver discharge, a broken interface treaty, a documentation mismatch, or a failed integration should not be returned as unrelated status codes. They should be returned as typed nonexistence claims with coordinate, violated law, evidence family, and repair frontier.

Definition 4.1 (Obstruction object)**.** *An obstruction object is a record*

$$B = \big(c, \kappa, E, R, \Delta\big)$$

where c is the obstructed coordinate or overlap site, κ identifies the failed admissibility condition, E is the supporting evidence, R is a repair frontier describing semantically relevant local modifications, and Δ records the downstream obligations or certificates affected by the failure.

The power of the obstruction language is that it creates transport between modes. A counterexample from equivalence checking can become a debugging witness. A documentation contradiction can reopen an interface treaty. A failed cover can become evidence for a different project decomposition. This is why "bugs are obstructions" has to be implemented as a result shape, not repeated as a slogan.

The AG strengthening is that some obstructions should be treated as genuine cohomology classes. For a cover $\mathfrak{U}$ and an obstruction presheaf $\mathscr{D}_T$, a persistent overlap discrepancy defines a class

$$[\omega] \in \check{H}^1(\mathfrak{U}, \mathscr{D}_T).$$

If $[\omega] = 0$, then the disagreement is removable by changing local representatives inside the present regime; it is a coboundary and therefore a local repair problem. If $[\omega] \neq 0$, then the failure is not removable by local patching alone within the present coefficient language. It signals a true gluing obstruction: a bug, integration failure, or semantic mismatch whose repair may require a new treaty, a finer cover, a stronger invariant language, or even a new domain pack. This is the precise sense in which bugs are cohomology classes rather than merely colorful metaphors.

Chapter 5

Judgments, sections, and semantic products

The third foundational task is to define the semantic products of the system. The system should not be organized around booleans, one-off verdicts, or tool-specific envelopes. It should be organized around judgments and sections. Judgments are the local atoms. Sections are the coherent products that later parts of the system build, compare, repair, export, and certify.

5.1 Judgments are not boolean facts

A serious project cannot be steered by booleans alone. A solver answer, a test result, a generated patch verdict, and a human review each expose only a shadow of the real semantic state. What JuGeo needs is a judgment form that preserves attainment, partial attainment, contradiction, suspension, ambiguity, and irrelevance. Without that richer form, every subsystem is forced either to overclaim or to remain uselessly noncommittal.

Definition 5.1 (Unified judgment)**.** *A unified judgment is a record*

$$J = \big(c, \varphi, A, E, O, B, T, \Pi\big)$$

where c is a coordinate, φ is the asserted proposition or target relation, A is the local carrier, E is the evidence bundle, O is the residual obligation family, B is the obstruction family, T is the attained trust profile, and Π is the provenance trace by which the present state was reached.

The reason to make this record foundational is that it stops feature families from inventing unrelated top-level outputs. A bug finder returns a judgment with a populated obstruction family. A generator returns a judgment whose proposition is only partially settled but whose residual obligations have shrunk. A documentation checker returns a judgment whose evidence bundle is mixed across structural and semantic routes. A fleet planner returns a judgment over a project frontier. The envelopes differ in content, not in ontology.

5.2 Residual obligations are the living part of the system

Residual obligations are the mechanism by which the system remains honest without becoming mute. They record exactly what must still happen before a stronger public claim would be warranted. A residual obligation is therefore not an informal note. It must know where it lives, what kind of

discharge it expects, which other obligations depend on it, and whether it is structural, semantic, relational, orchestration-level, or domain-pack-specific.

Implementation consequence. The orchestrator should operate over the graph of residual obligations rather than over file lists or prompt queues. Progress is measured by discharging strategically valuable obligations, sharpening weak evidence into stronger evidence, localizing vague failures into explicit obstructions, and refining covers so that difficult obligations become more tractable. This gives the system a real notion of leverage.

5.3 Sections are the real products of the system

A section is the object one gets when a family of local judgments becomes coherent over some coordinate region. Verified functions, relational certificates, repaired module clusters, documentation alignment reports, and resumable project states are all sections of different kinds. Local computations should therefore be written with section construction, section extension, or section repair in mind rather than as isolated answers.

A section need not be global to be valuable. A coherent local section over an important region of the coordinate space is already a useful artifact if the residual obligations preventing globalization are explicit. This is what gives the theory a genuine account of partial success, which is indispensable for long-horizon work.

5.4 Comparison maps and explanation projections

Comparison maps are the morphisms that make cross-mode reasoning possible. A debugging witness should be able to project into a public explanation. A solver countermodel should be able to project into a repair candidate. A documentation mismatch should be able to project into an interface treaty challenge. The implementation should therefore not treat explanation as an after-the-fact rendering problem. It should treat explanation projection as a typed map from internal judgment state into a lower-authority but human-comprehensible view.

A useful comparison-projection record is

$$\pi = (\mathcal{J}_{\mathrm{src}}, \mathcal{J}_{\mathrm{dst}}, \phi_{\mathrm{cmp}}, \rho_{\mathrm{loss}}, \sigma_{\mathrm{scope}}),$$

where $\mathcal{J}_{\mathrm{src}}$ is the source judgment family, $\mathcal{J}_{\mathrm{dst}}$ the target family or rendered artifact, ϕ_{cmp} the relation the map is meant to preserve, ρ_{loss} the declared information loss of the projection, and σ_{scope} the support scope on which the projection remains honest. This matters because explanations, certificates, regression reports, and public status messages are all lossy views. The loss should be explicit, not hidden in rhetoric.

The theorem targets are projection faithfulness and scope honesty. Projection faithfulness says that every public-facing statement must be traceable to internal clauses, evidence, residuals, or obstructions actually present in the source judgment. Scope honesty says that the projection may compress detail, but it may not silently widen the region over which a claim appears settled. This is the doctrine that prevents JuGeo from sounding more certain in prose than it is in semantics.

Chapter 6

Trust, provenance, evidence, and certificates

This chapter should make trust a structured object rather than a mood. Once evidence plurality is admitted, the system needs an algebra for combining support without confusing proof with solver discharge, runtime witness with semantic judgment, or local evidence with global closure. The public certificate is then not a success badge. It is a faithful projection of that structured trust state under explicit loss and scope.

6.1 Evidence plurality: proof, solver discharge, runtime witness, and controlled semantic judgment

Evidence plurality should be treated as a design premise rather than as a regrettable compromise. Different clauses genuinely belong to different support channels. Arithmetic and bounded symbolic fragments may be solver-discharged. Some relational or structural claims may be proved in a richer proof assistant. Resource or environment claims may depend on runtime witnesses. Semantically rich claims about user intention or underspecified behavior may require controlled semantic judgment. The thesis does not try to erase these differences. It tries to preserve them while still allowing them to accumulate inside one judgment state.

This gives a concrete implementation rule: every judgment should carry an evidence vector or evidence family descriptor rather than a single scalar status. Promotion in trust must therefore be defined as a policy-indexed aggregation of typed support, not as confidence inflation. Public certificates should remain clausewise enough that a reader can distinguish what was proved, what was solver-discharged, what was runtime-supported, what was semantically judged, and what remains residual.

6.2 Trust as an ordered algebra of admissible support

Trust should be defined as an ordered algebra over typed support channels rather than as a scalar confidence score. A judgment may be strongly supported structurally and weakly supported semantically, or vice versa. A runtime witness may settle one clause while leaving a nearby clause entirely open. The implementation should therefore carry clausewise trust profiles, not a single top-level level that pretends all support has been homogenized.

A useful trust object is

$$\mathfrak{T} = (\mathcal{E}_{\mathrm{adm}}, \preceq, \oplus, \ominus, \uparrow_\pi, \downarrow_\chi),$$

where $\mathcal{E}_{\mathrm{adm}}$ is the family of admissible evidence bundles, $\preceq$ the partial order of support strength, $\oplus$ the policy-indexed aggregation law, $\ominus$ the demotion law under challenge or replay failure, $\uparrow_\pi$ the promotion maps justified by policy π, and $\downarrow_\chi$ the challenge-triggered demotion maps. The use of a partial order rather than a total order matters. Some evidence bundles are incomparable until an explicit policy says how to relate them.

The theorem targets are monotonicity under admissible aggregation, no-silent-promotion, and challenge conservativity. Monotonicity says that adding admissible evidence cannot weaken a clause unless the added evidence is contradictory. No-silent-promotion says that trust can strengthen only through named policy routes. Challenge conservativity says that when a clause is challenged, the system may demote or residualize it, but may not leave the old trust claim standing without explanation. This is the algebra the implementation needs if it is to be honest about mixed support.

6.3 Certificates as faithful projections of semantic state

The certificate should be a projection of judgment state, not a replacement for it. A serious certificate must preserve which clauses were established, under what support, over what scope, with what residuals and fragility. If it suppresses those distinctions, it becomes a source of institutional dishonesty rather than a public artifact of accountability.

A certificate object should therefore have explicit form,

$$\mathcal{K} = (\mathcal{C}_{\mathrm{scope}}, \Phi^+, \Phi^\pm, \mathcal{E}_{\mathrm{pub}}, \mathcal{R}_{\mathrm{pub}}, \mathcal{F}_{\mathrm{frag}}, \Pi_{\mathrm{prov}}),$$

where $\mathcal{C}_{\mathrm{scope}}$ is the certified scope, Φ^+ the positively established clause family, $\Phi^\pm$ the qualified or partially established clauses, $\mathcal{E}_{\mathrm{pub}}$ the public evidence summary, $\mathcal{R}_{\mathrm{pub}}$ the public residual obligations, $\mathcal{F}_{\mathrm{frag}}$ the fragility declarations, and Π_{prov} the provenance trace. This prevents certificates from flattening partial closure into a universal success story.

The implementation consequence is that certificate emission must be driven by canonical records rather than by ad hoc renderer logic. One wants theorem targets of certificate faithfulness, renderer conservativity, and replay-aware reopening. Certificate faithfulness says that every public claim in the certificate has a source in internal judgment state. Renderer conservativity says that the renderer may omit detail but may not invent stronger closure. Replay-aware reopening says that when support goes stale, the certificate reopens through named channels rather than silently remaining current.

6.4 Manifest integrity

The manifest is the persistent semantic memory that makes resumability and replay intelligible. It should record not only which artifacts exist, but which coordinates, obligations, evidence items, treaties, and certificates are currently believed to exist, together with their support assumptions and epoch boundaries. A build log is not enough. A cache is not enough. The manifest must preserve authority-bearing state across time.

A useful manifest object is

$$\mathcal{M} = (\mathcal{J}, \mathcal{O}, \mathcal{E}, \mathcal{X}, \mathcal{K}, \eta, \sigma),$$

where $\mathcal{J}$ is the persisted judgment family, $\mathcal{O}$ the live obligations, $\mathcal{E}$ the evidence archive, $\mathcal{X}$ the obstruction archive, $\mathcal{K}$ the public certificate family, η the epoch map, and σ the support-indexed

invalidation graph. Manifest integrity then means that these components remain mutually traceable, serializable, and challengeable under reload.

The theorem burden is concrete: serialization determinism, dependency-trace integrity, and stale-manifest conservativity. Serialization determinism says equal semantic states serialize the same way up to declared ordering. Dependency-trace integrity says every reopened item can be explained by support or epoch dependence. Stale-manifest conservativity says that if refresh has not occurred after an invalidating event, the system must demote authority rather than pretending persistence implies truth.

Chapter 7

Controlled oracles, solver federation, and runtime witnesses

This chapter should turn evidence plurality into executable routing doctrine. Once proofs, solvers, runtime traces, and controlled semantic judgments all exist, the system needs a disciplined account of how they coexist, when they may challenge one another, and how their outputs are federated without being confused. The point is not to blur them into one backend. The point is to give them a common judgment language while preserving their distinct jurisdictions.

7.1 Controlled oracle theory: query constructors, jurisdiction, and trust boundaries

The oracle should enter the system through typed query constructors rather than ad hoc prompt strings. A query constructor should specify at least the local context, target clause family, admissible output form, allowed provenance scope, and intended downstream use. This makes the oracle a controlled proposal mechanism rather than a hidden author. Once queries are forced through typed constructors, the runtime can log them, replay them, challenge them, and compare their realized semantic delta against their predicted value.

Jurisdiction is equally important. The oracle has jurisdiction over proposal and semantic refinement, not over silent promotion of trust. If an oracle proposes an invariant, a decomposition, or a bridge theorem, that proposal should enter the state as a typed candidate with explicit residual obligations. It may influence scheduling and search immediately, but it must not become proof-shaped merely because it is convenient. This boundary is one of the clearest places where the thesis constrains implementation discipline.

7.2 Evidence federation: reconciling incomparable support channels

Evidence federation should be a typed reconciliation process, not a convenience merge. Different channels return different kinds of things: proof terms, solver discharge artifacts, runtime witnesses, semantic proposals, and human ratifications. The implementation should therefore never aggregate them by raw confidence or source prestige. It should aggregate them only after translating them into a common clause-and-support form that preserves their original kind.

A federation record should be written as

$$\mathfrak{F} = (\mathcal{Q}, \mathcal{E}_{\text{in}}, \mathcal{N}_{\text{cmp}}, \mathcal{A}_{\text{agg}}, \mathcal{R}_{\text{out}}),$$

where $\mathcal{Q}$ is the target clause family, $\mathcal{E}_{\text{in}}$ the incoming evidence items, $\mathcal{N}_{\text{cmp}}$ the comparison normal form used to align them, $\mathcal{A}_{\text{agg}}$ the admissible aggregation policy, and $\mathcal{R}_{\text{out}}$ the resulting judgment revision. This means the implementation needs explicit evidence-normalization and evidence-comparison layers, not just a place where multiple results happen to be stored together.

The theorem targets are kind preservation, aggregation admissibility, and contradiction exposure. Kind preservation says that proof-backed support remains proof-backed after federation, solver support remains solver support, and so on. Aggregation admissibility says that the chosen policy respects the declared trust algebra. Contradiction exposure says that incompatible channels produce a typed conflict or demotion rather than an averaged-away compromise.

7.3 Semantic jurisdiction

Jurisdiction is the law that keeps powerful subsystems from overclaiming. A solver has jurisdiction over the fragment its family admits. A runtime trace has jurisdiction over the environment and execution family it actually witnessed. A controlled oracle has jurisdiction over proposal, decomposition, semantic refinement, and ranking inside declared query constructors. A human has jurisdiction where explicit ratification or policy says so. None of these jurisdictions should be silently widened just because one channel is convenient.

The implementation should therefore carry a jurisdiction map

$$\mathcal{J}_{\text{jur}} = \{k \mapsto (\mathcal{Q}_k, \mathcal{E}_k, \mathcal{L}_k, \mathcal{N}_k)\},$$

where each channel k is assigned its admissible query family $\mathcal{Q}_k$, evidence family $\mathcal{E}_k$, escalation limits $\mathcal{L}_k$, and non-theorems $\mathcal{N}_k$. This turns backend choice into a semantically auditable decision. The user should be able to ask not merely "what answered this?" but "what was that subsystem allowed to answer?"

The theorem burden is jurisdiction soundness and escalation honesty. Jurisdiction soundness says no channel is credited for clauses outside its declared authority. Escalation honesty says any attempt to use a result beyond that authority must appear as a new obligation, bridge theorem, or human challenge, not as an implicit trust jump.

7.4 Obligation splitting

Mixed obligations should not be routed whole when their subclauses belong to different evidence families. The correct operation is splitting: one separates structural fragments, semantic fragments, runtime-dependent fragments, and treaty-clarification fragments into coordinated but distinct obligations. This is the operation that prevents one strong subsystem from being misused as a universal solvent.

A split operation should be represented by

$$\text{Split}(o) \Downarrow (o_{\text{str}}, o_{\text{sem}}, o_{\text{run}}, o_{\text{trt}}, \psi),$$

where the resulting obligations may be empty and ψ records the recomposition law by which their later results must be assembled. The implementation consequence is direct: the obligation store

should preserve split lineage, recomposition dependencies, and channel-local results under one parent identifier.

The theorem targets are split soundness, recomposition faithfulness, and residual honesty. Split soundness says the child obligations jointly cover the parent claim under the declared recomposition law. Recomposition faithfulness says the parent result is no stronger than what the child results justify together. Residual honesty says unresolved children stay visible rather than being lost in recomposition. This is the exact place where JuGeo becomes a mixed-evidence system rather than a sequence of backend fallbacks.

Chapter 8

Projects, modules, hypercovers, and fleets

This chapter should say explicitly that project scale is not merely "more files." It is a change in semantic topology. Once modules, registries, tests, documentation, benchmarks, and public surfaces interact, the right object is a covered semantic space with nontrivial overlaps, higher-order compatibility data, and resumable frontier state. Fleets become meaningful only against this object.

8.1 From single-artifact reasoning to project geometry

Project geometry begins the moment one admits that modules, interfaces, registries, import components, benchmarks, and public APIs all create partially overlapping semantic regions. Once that is admitted, the architecture should stop treating the project as a flat bag of files plus a dependency graph. It should instead treat the project as a covered semantic object in which local closure is common and global closure is earned only through gluing and treaty stabilization.

The implementation consequence is that planning, frontier control, archives, and regression should all be indexed by project geometry rather than by file lists alone. A support region may cut across several files. An overlap may involve several modules and a registry. A benchmark witness may support only one treaty family. These are not edge cases. They are the normal shape of project-scale semantics.

The mathematical point should be stated in sheaf language. A project is not just a graph; it is an object X in the coordinate site together with a chosen hypercover $\mathcal{H}_\bullet \to X$. Local module summaries are 0-cochains, interface treaties are part of the descent data on 1-simplices, and repeated multi-region integration failures are naturally expressed as nontrivial higher cocycles. The fleet is then not a set of independent writers but a distributed search over local sections and cochain corrections whose success is judged by whether they kill obstruction classes rather than merely increase text volume.

8.2 Fleet semantics and economic choice under obligations

Fleet semantics should be understood as budgeted competition over obligation-reducing moves. A fleet member should not merely emit code. It should bid on a semantic delta: which obligations it expects to reduce, which overlaps it expects to stress, what new residuals it risks introducing, and how its move interacts with treaty stability. This turns parallelism into a mathematically inspectable search procedure.

A useful fleet bid is

$$b = (\Delta\mathcal{O}, \Delta\mathcal{T}, \Delta\mathcal{E}, c, u, \rho),$$

where $\Delta\mathcal{O}$ is the predicted obligation delta, $\Delta\mathcal{T}$ the treaty delta, $\Delta\mathcal{E}$ the evidence delta, c the projected cost, u the uncertainty profile, and ρ the support region. The controller can then choose among bids by expected semantic leverage rather than by first-complete or highest-confidence heuristics.

The theorem targets are bid honesty, calibration by obligation family, and no-unearned-certificate. Bid honesty says accepted bids must be comparable against realized semantic deltas. Calibration says bidder quality should be learned per support regime and task class. No-unearned-certificate says no fleet result can strengthen public closure before going through ordinary validation and certificate projection.

8.3 Closure and resumability

Closure at project scale should mean more than "the current run stopped changing files." A project is semantically closed only when its accepted sections glue under the active treaties, its residual obligations are explicitly bounded, its certificates name their fragility, and its manifest is rich enough for replay. Resumability then means that later work can restart from this persisted semantic state rather than from prose memory or shell history.

A resumable state should be represented as

$$\mathcal{R}_{\text{proj}} = (\mathcal{M}, \mathcal{F}, \mathcal{B}, \Pi),$$

where $\mathcal{M}$ is the manifest, $\mathcal{F}$ the frontier of admissible next moves, $\mathcal{B}$ the remaining budget state, and Π the replay trace. This lets the orchestrator resume with semantic awareness of what was settled, what was tentative, what was challenged, and what must be reopened after environment or code change.

The theorem targets are replay adequacy, resumable-frontier fidelity, and support-local reopening. Replay adequacy says the persisted trace and manifest are sufficient to reconstruct admissible future moves. Resumable-frontier fidelity says a resumed frontier is semantically equivalent to the frontier that would have been computed from scratch on the same state. Support-local reopening says later edits should reopen exactly the persisted region justified by support and epoch dependence, not the entire project by default.

Chapter 9

Mathematical interlude: a more explicit formal core

This interlude should collect the formal objects implicit in the earlier chapters and state them in a compact, implementation-facing way. The goal is not to suspend the thesis in abstract notation. The goal is to show that the central vocabulary of coordinates, judgments, evidence, obstructions, and trust can be given a small enough core that code can actually be organized around it.

9.1 A site for programmatic judgment

A useful formal core begins with a site $\mathcal{S}$ whose objects are semantic coordinates and whose morphisms are lawful restriction maps. A coordinate should record at least region, context, support, epoch, and mode. Covers on this site encode admissible decompositions of programmatic work: path partitions, module families, relational comparison regions, documentation/code pairs, runtime slices, and orchestration frontiers. This is the algebraic-geometric base over which the later judgment objects live.

The implementation consequence is that the code should have an explicit coordinate constructor and a small family of canonical restriction morphisms. Without that, the site language collapses back into prose. One wants theorem targets of coordinate canonicalization and restriction preservation: equal semantic regions should normalize to equal coordinates, and restriction should preserve only what the governing type declares preservable.

On top of this site the thesis should posit several presheaves and sheaves: a carrier presheaf $\mathscr{A}$ of candidate artifacts, an admissibility presheaf $\mathscr{P}$ of locally acceptable sections, a certificate sheaf $\mathscr{K}$ of publicly exportable clauses under a fixed policy, and one or more obstruction presheaves $\mathscr{D}$ recording overlap disagreement. The practical slogan should be replaced by the mathematical one: JuGeo's runtime manipulates sections of these presheaves, computes their descent data over covers, and reacts to their obstruction classes.

9.2 Trust as an ordered algebra of admissible support

At the formal-core level, trust should be treated as a policy-indexed algebra over evidence bundles, not as a decoration on judgment results. This lets the same trust object support proof-driven subsystems, solver families, runtime witnesses, and controlled semantic judgments while keeping promotion and demotion explicit. The chapter should therefore make clear that trust is part of the semantic state, not an afterthought attached by renderers.

9.3 Obstructions as structured nonexistence

An obstruction should be understood as structured nonexistence: evidence that no admissible section, extension, or gluing exists of the requested kind under the current support, trust policy, and overlap laws. This makes obstruction the common language of bug finding, failed refinement, broken treaties, and failed orchestration moves. It is also what gives the system a principled notion of when to escalate from local repair to theorem growth or domain formation.

The implementation consequence is that obstruction objects should persist in manifests and archives, not disappear after a failed run. The theorem target is obstruction transport: if a later subsystem consumes an earlier failure, it should inherit the coordinate, violated law, and support scope rather than receiving only an informal error message.

The cohomological version of this statement should be explicit. If an obstruction is represented by a cocycle $\omega \in \check{Z}^1(\mathfrak{U}, \mathscr{D})$, then local repair corresponds to replacing the current 0-cochain by another representative whose coboundary cancels ω. If no such representative exists inside the current regime, the nonzero class $[\omega]$ witnesses that the present language of local sections is insufficient for global descent. In software terms, this is the moment where one stops trying tiny patches and instead changes the invariant language, the cover, or the coefficient theory. That is the active mathematical role of cohomology in the thesis.

Part III

Usage Modes and Unified Problem Classes

Chapter 10

Specification and satisfaction

A specification in JuGeo should be treated neither as commentary nor as a bag of tests. It is a target geometry for admissible sections. The specification says what local sections are required, on which supports they are required, what counts as evidence for each clause, how clausewise truths are transported across overlaps, and what kind of certificate may be exported when local data are glued. Once specification is made geometric in this sense, verification, mixed-mode authoring, and synthesis become different ways of constructing or extending a section toward the same target rather than unrelated features.

This is the first place where the dissertation should say explicitly that specification verification and orchestration control belong to one mathematics. A specification checker working on a fixed artifact asks whether there exists a section s over a chosen coordinate family such that s realizes the target geometry with admissible evidence and vanishing overlap obstruction. The orchestrator asks the same question in dynamic form: given only a partial section, an archive, and a frontier of admissible moves, which sequence of local extensions is most likely to produce such an s? The objects are the same in both stories—clauses, covers, local sections, overlap laws, evidence policies, residual ledgers, witnesses, and obstruction classes. What changes is only whether the system is evaluating one candidate section or steering a search over many partial ones.

Put differently, specification verification is the static face of JuGeo, while orchestration is the control-theoretic face of the same descent problem. The verifier computes truth of local clauses and gluing conditions for a proposed section. The orchestrator manages the state whose components are those same clause tables, residuals, witnesses, and overlap constraints, and chooses actions that improve the chance of eventual descent. This is why the later orchestration chapter must not introduce a second semantic kernel. It is operating on the same target geometry that the present chapter uses to define satisfaction.

10.1 Specifications as target geometry: clauses, truth conditions, and target sections

A usable specification object must elaborate to more than a natural-language sentence. For a coordinate c, I will treat a specification as a finite family

$$\mathsf{Spec}(c) = \big(\{U_i\}_{i\in I}, \{\tau_i\}_{i\in I}, \{\chi_i\}_{i\in I}, \{\epsilon_i\}_{i\in I}, \Gamma_\cap\big)$$

where each U_i is a support region in a chosen cover of c, each τ_i is a target section describing the intended local behavior on U_i, each χ_i is a truth-condition constructor for judging whether a candidate local artifact realizes τ_i, each ϵ_i is the evidence policy admissible for that clause, and $\Gamma_\cap$

is the overlap law family that says what agreement must hold on pairwise and higher overlaps. A textual guarantee such as "returns a sorted list with no duplicates" therefore compiles into several local targets rather than one slogan: an output-shape clause, an order clause, a duplicate-freedom clause, and, where necessary, a stability or exception-behavior clause.

The semantic center of specification should therefore be a Clause Registry, not a docstring parser. The Clause Registry owns clause identifiers, support regions, elaborated truth conditions, and public/export tags. A Truth-Condition Compiler elaborates natural-language fragments and domain-pack clauses into χ_i objects whose structural part can be discharged by solvers and whose semantic part can be routed to controlled oracle judgment. A Target Section Store owns the current target family $\{\tau_i\}$ and supports transport along refactors, migrations, and interface renamings. A Domain Pack Registry contributes reusable clause schemas, vocabulary, and transport laws for libraries and application domains. Without these authority centers, "the specification" remains a rhetorical overlay rather than an executable semantic object.

Definition 10.1 (Specification object)**.** *For a project coordinate c, a specification object is a target geometry whose local sections describe admissible realizations over a cover of c, together with clausewise truth constructors, overlap compatibility laws, and an evidence policy for each clause. Satisfaction of the specification means not merely that some global artifact exists, but that there exists a section whose restrictions satisfy the local truth conditions and whose overlap data glue without contradiction.*

Proposition 10.2 (Coverwise satisfaction target)**.** *Let $\{U_i\}_{i\in I}$ be a finite cover for a coordinate c. Suppose that for each i there is a local section s_i and evidence bundle E_i such that $\chi_i(s_i, E_i)$ is attained with admissible trust, and suppose the overlap laws in $\Gamma_\cap$ hold on all nonempty overlaps. Then the gluing procedure should emit either a global section $s \in \Gamma(c)$ satisfying the specification or a typed obstruction whose support lies in a minimal failing overlap family. In particular, specification satisfaction is a gluing problem with explicit failure witnesses, not a post hoc summary bit.*

Implementation consequence. The repository architecture implied by this section includes at least five explicit modules: a clause elaborator, a truth-condition compiler, a target-section authority center, a domain-pack registry, and a satisfaction certificate projector. Certificates must cite the originating clauses, not only the final verdict, because later chapters on debugging, migration, and public honesty require clause-level reopening and projection.

10.2 Clausewise truth

The right semantic primitive is clausewise truth, because most real specifications are only partially attained at any given time. For a clause i and candidate local section s, the evaluator should return

$$\mathsf{Truth}_i(s, E) \in \{\mathsf{satisfied}, \mathsf{violated}, \mathsf{residual}, \mathsf{inapplicable}\},$$

together with a trust level, provenance trace, and any witness or obstruction data used to reach that state. The point of this four-way form is that a system which collapses residual and violated into one status cannot distinguish "we know it fails" from "we have not yet closed the obligation," and a system which collapses inapplicable into satisfied silently overclaims.

Clausewise truth must also preserve evidence-family identity. A solver certificate for a length-preservation clause, a runtime witness for an exception-path clause, and a controlled semantic judgment for a readability clause may all support the same target section, but they should never be merged into an opaque score. The implementation therefore needs a Clause Evaluator that

records clause status together with evidence provenance, a Residual Ledger that tracks unsolved local obligations, a Counterexample Store that indexes violations by clause identifier and support region, and a Trust Join Policy that forbids a public projector from strengthening the local trust of a clause when evidence is aggregated.

A useful theorem target here is monotonicity of clause status under evidence extension. If a clause is already violated, adding more admissible evidence may sharpen the explanation but may not silently relabel it satisfied. If a clause is residual, additional evidence may discharge it or contradict it, but transport to a larger coordinate must not erase the fact that it was unresolved on a smaller one. This theorem is not a mathematical luxury. It is what makes incremental verification and truthful public manifests possible.

Algorithm sketch. For each clause, normalize the clause text with domain-pack vocabulary, split structural from semantic subconditions, choose admissible evidence channels from ϵ_i, dispatch each subcondition to the appropriate checker, and then join the returned evidence without collapsing provenance. Emit a clause record containing status, trust, witness set, obstruction set, and downstream dependencies. The global specification verdict is then computed as a faithful projection of the clause table rather than as an independently invented summary.

10.3 Mixed-mode programming: partial semantic authorship, holes, and local obligations

Mixed-mode programming should be formalized as partial semantic authorship. The programmer does not author only code text. The programmer authors part of a target section directly and leaves the rest open. In JuGeo surface syntax, `@guarantee` contributes exported postcondition clauses, `assume()` introduces caller-owned obligations, `check()` introduces callee-owned local obligations, `given()` imports treaty families from domain packs or prior theory, `hole()` introduces an existential local section to be synthesized, and `@spec` declares that an entire artifact should be constructed as a witness to a target section. The semantic output of elaborating such a file is therefore a partial section together with an obligation frontier.

Construction 10.3 (Mixed-mode elaboration)**.** *Given a file and dependent context Γ, elaborate each surface form to a semantic contribution:*

$$\begin{aligned}
\texttt{@guarantee} &\mapsto \textit{exported clause family},\\
\texttt{assume()} &\mapsto \textit{upstream obligation owned by callers or enclosing treaties},\\
\texttt{check()} &\mapsto \textit{local invariant obligation at the current support},\\
\texttt{given()} &\mapsto \textit{imported theorem or domain-pack treaty},\\
\texttt{hole()} &\mapsto \textit{existential local section variable plus boundary conditions},\\
\texttt{@spec} &\mapsto \textit{artifact-level target geometry whose witness is absent.}
\end{aligned}$$

The elaboration result is a partial section s_{auth}, a family of residual obligations O, and a frontier of existential sites H to be extended.

This construction implies concrete authority centers. A Fragment Registry owns every mixed-mode fragment with source location and context. A Hole Context Builder computes the exact local environment, expected type, imported treaties, and overlap boundary that constrain each hole. A Local Obligation Ledger records whether an `assume()` is still open, discharged by transport,

contradicted by witness, or intentionally exported as a public assumption. A Treaty Store records which `given()` clauses came from humans, from domain packs, or from prior certificates, because later migration and refinement logic must distinguish them.

The theorem target is locality of obligation ownership. An obligation generated inside a support region should reopen only those dependent clauses and certificates whose support intersects the changed region or whose transport laws explicitly depend on the reopened clause. This is the semantic basis for incremental recompilation and non-global invalidation.

10.4 Generation as extension: partial sections, synthesis steps, and residuals

Generation should be stated as an extension problem. One begins not from nothing but from a partial section

$$s_0 \in \Gamma(V, T)$$

defined on a support subfamily V consisting of authored clauses, imported treaties, already-verified artifacts, and unresolved holes. A synthesis step attempts to extend s_0 to a larger support $V \cup U_j$ by proposing a local section on U_j, checking boundary agreement on $U_j \cap V$, and then either gluing the proposal into the current section or emitting residual obligations and obstructions. This is exactly the same local-to-global calculus already used for verification; generation differs only in which local sections are currently missing.

The right implementation object is therefore not "generated code" in isolation but a Synthesis Step Record containing the target site, the candidate local section, the evidence used to justify it, the residual obligations it leaves behind, and the overlap treaties it either preserved or stressed. An Extension Planner chooses which frontier site to address next. A Residual Prioritizer orders open obligations by leverage, boundary risk, and downstream certificate impact. An Orchestration Controller decides whether to call a structural compiler, a domain-pack rewrite, an oracle-guided synthesizer, or a repair engine. A Manifest Builder records what was authored, what was synthesized, what remains residual, and which certificates depend on each step.

Algorithm sketch. Choose a frontier hole or unresolved target site with maximal semantic leverage. Freeze the current boundary treaties on overlaps. Query the appropriate synthesis authority—template expansion, domain pack, transformation library, or controlled oracle—for a candidate local section. Elaborate the candidate into clauses, discharge structural obligations, request semantic judgments where jurisdiction permits, and compute the residual set that remains after local checking. If overlap laws hold, glue the candidate and update the frontier; if not, emit an obstruction object with a localized repair frontier. Iterate until the residual ledger reaches the configured closure criterion or until the orchestrator decides that further progress requires human authorship.

A central theorem target is residual monotonicity under accepted extension. If a synthesis step is accepted, then outside the edited support its truth table and trust profile may not worsen, and inside the edited support every remaining uncertainty must appear explicitly in the new residual ledger. This theorem is the bridge between generation, debugging, and public honesty: accepted synthesis may enlarge the section, but it may not hide uncertainty.

Chapter 11

Debugging, counterexamples, and repair

Debugging should not be described as an auxiliary developer experience surrounding the "real" verifier. It is one of the core usage modes of judgment geometry. A bug is an obstruction to extending or maintaining a coherent section. A counterexample is a witness that a clause or relation fails at a particular support. A repair is a controlled local modification that aims to remove the obstruction while preserving as much of the surrounding glued structure as possible. The same semantic state that supports generation and certification should therefore support localization, witness extraction, repair planning, and non-regression closure.

11.1 Debugging as obstruction localization

The semantic task of debugging is not merely to find a bad line. It is to localize an obstruction to the smallest support and overlap family that explains the failure. Given an obstruction object

$$B = \big(c, \kappa, E, R, \Delta\big),$$

the debugger should compute a localization

$$\mathrm{Loc}(B) = \big(U_{\min}, \partial U_{\min}, \mathcal{L}, W, \Pi\big)$$

where $U_{\min}$ is the smallest support region that still carries the failure, $\partial U_{\min}$ is the boundary whose treaties constrain admissible repairs, $\mathcal{L}$ is the violated law family, W is any extracted witness family, and Π is the provenance slice showing how the current state arose. This is what turns a cohomological or treaty-level failure into an actionable debugging target.

The implementation consequence is that the debugging subsystem needs explicit authority centers: an Obstruction Index keyed by clause and support, a Localization Functor that maps structural and semantic failures into human-usable diagnostics without discarding semantic identity, a Support Influence Graph that explains which neighboring clauses and treaties constrain the failing region, and a Challenge Backlog that tracks not-yet-explained residuals. The existing distinction between proof failure, runtime failure, semantic mismatch, and migration drift should survive into the debugger because they induce different repair frontiers.

Proposition 11.1 (Minimal-support localization target)**.** *For every obstruction family produced by the checker, the localization procedure should return either a support-minimal explanation region or*

an explicit reason why support minimization is undecidable under the current evidence. In the latter case, the exported diagnostic must still identify the overlap family whose unresolved ambiguity blocks sharper localization. A debugger that reports only file-level blame has failed to realize the geometry it claims to use.

11.2 Counterexamples as semantic witnesses

A counterexample should be treated as a first-class semantic witness, not as a transient log line. For a violated clause or relation, a counterexample witness can be represented as

$$W = \big(c, \iota, \xi, \kappa, E, \Pi\big)$$

where c is the support coordinate, ι is the triggering input or environment, ξ is the observed trace, model, schedule, or output fragment, κ identifies the violated clause or relational obligation, E is the evidence bundle, and Π is the provenance path linking this witness to the current section. This single shape is rich enough to cover a Z3 model falsifying a path condition, a runtime trace falsifying an invariant, a paired execution witness falsifying equivalence, or a schedule witness falsifying a concurrency guarantee.

The Counterexample Archive should therefore be an authority center in its own right. It indexes witnesses by clause family, relation family, support, trust, environment assumptions, and replayability conditions. A Witness Replayer tests whether a witness still lives after a proposed repair. A Witness Transporter maps a local witness into every clause, treaty, and public manifest whose restriction depends on the same support. This transport is implementation-critical: a single counterexample to a public API promise should reopen not only the function clause but also documentation claims, CLI help text, regression certificates, and release manifests that relied on that promise.

A useful theorem target is counterexample persistence under unchanged support. If a candidate repair leaves the support and boundary conditions of a witness unchanged, then the witness remains valid unless the system can exhibit a transport law showing why the witness no longer applies. This theorem underwrites cheap non-regression checks and prevents systems from declaring a bug fixed simply because surrounding code was rephrased.

11.3 Repair as controlled surgery on a partial section

Repair should be defined as controlled surgery on a partial section. One has a current section s with an obstructed region U, and the goal is to replace the restriction $s|_U$ by a new local section s'_U such that the repaired section

$$s^{\mathrm{rep}} = s[U \leftarrow s'_U]$$

agrees with the original section outside U, satisfies the relevant boundary treaties on ∂U, and removes or weakens the targeted obstruction without introducing stronger failures elsewhere. The boundary condition is the key point. A repair that ignores overlap treaties is merely a fresh generation attempt with hidden regression risk.

This definition implies a Repair Planner that computes the repair support closure, a Boundary Treaty Extractor that freezes the interface and behavior obligations the repair must preserve, a Patch Synthesizer that proposes candidate surgeries, and a Regression Closure Engine that checks whether preserved clauses and witnesses still hold outside the edited support. The planner should optimize for semantic leverage, not textual minimality alone. Sometimes the smallest textual patch has a large support shadow because it perturbs a widely used treaty; sometimes a larger textual rewrite is semantically more local because it restores a previously broken overlap invariant.

Algorithm sketch. Start from a localized obstruction and compute the transitive support closure needed to make the failing law editable. Freeze every neighboring clause and treaty that lies on the boundary of this region. Synthesize one or more candidate local replacements using structural rewrites, domain-pack adaptations, proof-directed transformations, or controlled oracle proposals. For each candidate, replay archived witnesses, discharge reopened obligations, and compare the resulting clause table against the previous one outside the repair region. Accept only candidates whose residuals are explicit and whose boundary treaties remain coherent.

11.4 Repairs and generations should be governed by the same semantic calculus

The cleanest architecture treats repair and generation as the same extension problem with different boundary data. Generation starts from a sparse or empty section and extends into uncovered supports. Repair starts from a mostly glued section, deletes or marks as obstructed a local region, and then extends back into that region under frozen boundary treaties. The orchestrator should therefore not maintain disjoint planners for "generate" and "fix." It should maintain one Section Transition Engine whose moves are local extension, local contraction, witness replay, clause reopening, and certificate reprojection.

This unification has several concrete payoffs. First, the same residual ledger can rank both missing artifacts and broken artifacts. Second, the same manifest can record authored, synthesized, and repaired regions without losing provenance. Third, the same domain packs and relation registries can serve synthesis, transformation, migration, and bug fixing. Fourth, the same theorem targets apply: locality of reopening, monotonicity of trust outside edited supports, and faithfulness of public projection.

Proposition 11.2 (Repair-generation reduction)**.** *Every repair problem can be reduced to a generation problem on a punctured section with frozen boundary treaties, and every generation problem can be viewed as repair of the empty section. Therefore any implementation that duplicates clause elaboration, witness handling, or certificate logic across the two modes is carrying unnecessary semantic debt.*

Chapter 12

Equivalence and refinement

Equivalence and refinement are not side topics reserved for proof engineers. They are among the most practical usage modes of the judgment geometry because migration, optimization, API stabilization, regression closure, and donor inheritance all require relational rather than unary reasoning. The central discipline is that equivalence is always relative to a chosen relation, and refinement is the operational form of that relativity when one artifact is intended to replace, extend, or restrict another under controlled obligations.

12.1 Equivalence is always relative to a relation

There is no relation-free notion of equivalence worth implementing. Two artifacts may be extensionally equal on returned values, observationally equal on traces, API-equivalent on exported surface, explanation-equivalent on user-visible stories, or performance-equivalent only up to a resource envelope. The system should therefore represent an equivalence judgment as

$$\mathsf{Eqv}_R(x, y; c)$$

where R is a relation family indexed by coordinate and support. The relation object must declare its carrier pairing, its local truth conditions, its admissible witnesses, its overlap laws, and the public projection allowed when results are exported.

The required authority center is a Relation Registry. It owns named relation families such as extensional equality, observational equivalence, backward compatibility, trace inclusion, simulation, resource-bounded similarity, and documentation equivalence. A Relational Elaborator compiles user requests like "behaviorally equivalent except for logging" into a precise relation family and support restriction. A Paired-Cover Builder chooses how to decompose the comparison problem into local regions. A Relational Certificate Projector exports only what the chosen relation justifies. This is what prevents casual misuse of the word "equivalent" in code review, migration reports, and public release notes.

Proposition 12.1 (Coverwise relational gluing target)**.** *Suppose a relation family R is stable under restriction and admits gluing along a chosen paired cover. If each local comparison problem on the cover is solved with admissible witnesses and the overlap compatibility laws hold, then the implementation should emit either a global equivalence witness or a typed relational obstruction localized to the failing overlap family. Equivalence checking is therefore a section comparison problem, not a monolithic binary verdict.*

12.2 Refinement is the most practical face of equivalence

Refinement is the case in which one artifact is allowed to be more specific, more efficient, more constrained, or more informative than another while remaining acceptable relative to a declared relation. In practice, this is the form needed for replacing an implementation, specializing a generic interface, migrating from a donor project, tightening a postcondition, or swapping one generated candidate for another. A refinement judgment should therefore state which obligations are preserved, which are strengthened, which are intentionally discharged by stronger evidence, and which public stories must remain unchanged.

The implementation consequence is a Refinement Checker that does not merely ask whether two artifacts are alike, but whether the candidate replacement transports every required treaty and certificate into the new context without illicit trust inflation. A refinement that preserves behavior but silently weakens the evidence behind a public guarantee is not acceptable unless policy says it may be exported as partial. A Substitution Manifest should record exactly which clauses were preserved, strengthened, weakened, or reopened. This manifest becomes central for migration, optimization, and generated-code governance.

A useful theorem target is transitivity of admissible refinement under compatible relation families. If artifact B refines A under relation R and artifact C refines B under relation R, then C should refine A provided the boundary treaties and evidence policies compose. This theorem is what allows long-horizon orchestrators to chain local substitutions without redoing every global comparison from scratch.

12.3 Relational obligations and witnesses

Relational reasoning needs its own obligation and witness forms. Unary obligations talk about one section satisfying one clause family. Relational obligations talk about paired inputs, paired traces, simulation maps, commutative diagrams, compatible error surfaces, or transport of certificates across versions. The authority center here should be a Relation Manifest containing: the chosen relation family, the paired cover, the local obligations induced by that relation, the admissible witness constructors, and the projection rules for public reports.

Witnesses are correspondingly diverse. An extensional comparison may use paired input-output tables or solver-generated separating models. An observational relation may use trace alignment or bisimulation checkpoints. A compatibility relation may use request/response examples and versioned surface manifests. A refinement proof may use simulation functions, treaty transport maps, and replay of archived counterexamples. What matters is not uniform data shape but uniform judgment shape: every witness must say which relation it supports, on which support, with what trust, and with what boundary assumptions.

Algorithm sketch. Normalize the requested relation into a relation-manifest entry. Build a paired cover that respects module boundaries, public interfaces, and known dependency cuts. For each local paired site, generate the relevant relational obligations and dispatch them to structural, runtime, or semantic witness constructors. Check overlap compatibility, propagate failures into the relational obstruction index, and emit either a glued relational certificate or a minimal counterexample family. The final report should distinguish "not equivalent," "equivalent under a weaker relation," and "refinement holds but with downgraded support," because those are semantically different outcomes with different downstream consequences.

Chapter 13

Documentation, migration, and public honesty

Documentation, migration, and public reporting belong in the same chapter because they are all questions about what a section says beyond the local code that implements it. Documentation externalizes clause families for human readers. Migration transports sections and obligations across versions, donors, or platforms. Public honesty governs what may be claimed when support is mixed, partial, or still residual. If these are separated from the main semantic machinery, the system will overclaim in public and under-record the real sources of drift.

13.1 Documentation should become a live semantic participant

Documentation should be treated as a live participant in the judgment geometry, not as inert prose attached after the fact. A docstring, README claim, CLI help sentence, tutorial example, changelog entry, or generated explanation is a family of clauses over public coordinates. Each such clause has support, vocabulary, truth conditions, admissible evidence, and export policy. The right authority center is therefore a Documentation Clause Registry that elaborates documentation into clause families aligned with the same coordinates used by code and certificates.

This immediately implies two implementation modules. First, a Documentation Elaborator parses public text into clauses, links terms to domain-pack vocabulary, and records whether a clause is descriptive, normative, comparative, or pedagogical. Second, an Explanation Projector maps internal clause tables and certificates back into public language without inventing stronger claims. This makes documentation queryable, comparable, and reopenable. When a counterexample invalidates a user-visible statement, the system should know exactly which documentation clauses are affected and which release surfaces must be updated.

13.2 Documentation alignment

Documentation alignment is the problem of comparing the executable section and the explanatory section over the same or related supports. It is not enough to ask whether text “mentions” the code. One must detect omitted guarantees, stale names, false promises, changed preconditions, outdated examples, and trust mismatches between the public story and the current evidence. Alignment should therefore produce a typed result: matched clauses, unmatched executable clauses, unmatched documentation clauses, contradicted claims, downgraded claims, and residual claims whose status cannot yet be exported honestly.

The required authority centers are a Documentation Alignment Engine, a Surface Vocabulary Normalizer, and a Public Clause Diff Store. The alignment engine compares clause families across code, tests, certificates, and prose. The normalizer resolves synonymous domain terms, versioned names, and shorthand. The diff store records what changed, who owns the discrepancy, and which public surfaces are still out of date. This is what turns documentation maintenance from a vague admonition into a typed verification mode.

Proposition 13.1 (No-silent-overclaim target)**.** *If a public documentation clause depends on executable clauses whose current support is residual or contradicted, then the aligned public projection may not export the clause as settled fact. It must either downgrade the claim, attach its trust level, or mark it as pending. A documentation system that forgets residuality is not merely incomplete; it is semantically dishonest.*

13.3 Migration and donor inheritance

Migration and donor inheritance are transport problems. A donor project, previous version, or external library contributes a family of sections, treaties, examples, and certificates defined over one coordinate system. Migration asks whether these can be transported into a new coordinate system through an adaptation map while preserving the obligations that matter. The right semantic object is a migration treaty

$$M = \big(D, \alpha, \Lambda, E, \Omega\big)$$

where D is the donor bundle, α is the adaptation map into the recipient coordinate system, Λ is the family of clauses intended to survive transport, E is the supporting evidence and provenance from the donor side, and Ω is the residual set that the recipient must still discharge locally.

This requires a Donor Manifest authority center, a Migration Treaty Checker, and a Domain-Pack Translator. The donor manifest records what came from where, under what version assumptions, and with what trust. The treaty checker decides which donor claims transport unchanged, which need revalidation, and which are merely heuristic hints. The translator rewrites vocabulary, API names, and environment assumptions through domain-pack knowledge. This architecture is necessary for truthful donor reuse: imported code or ideas are never "just inherited," but inherited with a typed trail of transported and non-transported obligations.

13.4 Public API, CLI, and explanation semantics

Public surfaces are semantic objects in their own right. An API is not merely a set of function names; it is a coordinate family of call shapes, return treaties, failure modes, side-effect stories, stability promises, and versioned availability. A CLI is not merely a parser; it is a surface of commands, flags, exit codes, environment assumptions, diagnostic messages, and example traces. Explanations are not merely textual niceties; they are public projections of internal judgment state. A serious implementation therefore needs a Surface Manifest that owns the public coordinates, exported clauses, admissible evidence thresholds, and version/diff history.

A Release Certificate Builder should project internal clause tables, trust levels, and residuals into API references, CLI help, changelogs, and explanation outputs. A Public Surface Checker should verify compatibility claims across releases and catch API drift, CLI drift, and explanation drift as ordinary relational obstructions. A Behavior Sampler may generate replayable examples, but only examples justified by current clause tables and manifests should be promoted to public

surfaces. This is the implementation point at which "public honesty" stops being editorial policy and becomes a typed export rule.

13.5 The public story should remain honest about partiality

Public honesty is the rule that exported narratives may forget detail but may not strengthen support. The public story is a projection of semantic state, not an independent creative act. If a guarantee is solver-proven, say so. If it is runtime-backed on a restricted environment, say so. If it is based on controlled semantic judgment, say so. If it remains residual or partially migrated, say so. This demands an Honesty Projector that is support-nonexpansive and trust-noninflating, together with a Manifest Integrity Checker that ensures every exported sentence can be traced back to internal clause families and evidence.

Proposition 13.2 (Public honesty theorem target)**.** *Let P be the public projection from internal judgment state to exported manifests, documentation, API references, CLI help, and generated explanations. Then P should be monotone with respect to forgetting internal detail but non-monotone with respect to strengthening trust: it may coarsen, summarize, or suppress internal structure, but it may not export a claim with stronger support, broader applicability, or smaller residual set than the source state justifies.*

The implementation consequences are concrete. Exported manifests need explicit trust labels. Residual clauses affecting public behavior need stable identifiers so they can be shown in release reports and explanations. Partial migrations need donor provenance attached to public compatibility claims. Generated explanations need citation hooks into clause IDs and certificates. Without these mechanisms, public communication becomes the least rigorous part of a system whose entire point is to make rigor compositional.

Chapter 14

Unified problem atlas

The thesis claim only becomes serious if the apparently different problem classes can be expressed by one normal form rich enough to guide implementation. The right chapter therefore is not a catalog of applications but an atlas of parameter settings for the same semantic machine. Every usage mode below chooses a coordinate family, a target relation, a cover strategy, an evidence policy, an obstruction extractor, and a closure criterion. What changes from mode to mode is not the underlying calculus but which section is being sought, compared, repaired, transported, or publicly projected.

Construction 14.1 (Problem-class normal form)**.** *A JuGeo problem class is specified by a sextuple*

$$\mathcal{P} = \big(C, R, \mathcal{U}, \epsilon, \beta, \Omega\big)$$

where C is the coordinate family, R is the target unary or relational judgment family, $\mathcal{U}$ is the cover or hypercover constructor, ϵ is the evidence-admissibility policy, β is the obstruction and witness extractor, and Ω is the closure criterion deciding when orchestration may stop. Improvements to any one component should transfer across all modes that instantiate the same kernel.

14.1 Specification satisfaction

For specification satisfaction, C ranges over implementation supports and public interfaces, R is realization of a target section compiled from clause families, $\mathcal{U}$ is chosen to align with semantic clauses and interface boundaries, ϵ admits proofs, solver discharge, runtime witnesses, and controlled semantic judgments according to clause policy, β emits violated or residual clauses, and Ω is clausewise closure relative to the requested trust threshold. This is the baseline mode, but it is already enough to force the architecture: clause registries, target-section stores, residual ledgers, and certificate projectors are not optional conveniences but required authority centers.

14.2 Bug finding and diagnosis

For bug finding, the coordinate family is the current section together with its overlap structure; the target relation is not a new spec but continued coherence of the existing one; the obstruction extractor is primary. What matters is the localization of failing supports, witness replay, and repair frontiers. This means the same clause tables used for verification should feed the debugger directly. A separate "bug system" that re-derives facts from text or logs is evidence that the unification has failed.

14.3 Equivalence and relational refinement

For equivalence and refinement, C is a paired coordinate family and R is a user-declared relation rather than unary realization. The same cover machinery applies, but over paired supports and relation-specific overlaps. The same residual ledger applies, but now for relational obligations. The same certificates apply, but projected through a relation manifest. This is why the repository should have one relation registry and one witness archive rather than bespoke comparison code for migrations, optimizations, and compatibility checks.

14.4 Repair and program transformation

Repair and transformation are section transitions constrained by boundary treaties. The coordinate family is the current section plus an editable support region, the target relation is preservation of required clauses outside that region together with restoration or strengthening inside it, and closure is measured by removal of the targeted obstruction without exporting new hidden residuals. Because this is extension on a punctured section, the same orchestrator, domain packs, residual prioritizer, and certificate projector should serve both repair and original generation.

14.5 Large-codebase generation

Large-codebase generation differs from unary verification only in sparsity and orchestration pressure. The coordinate family spans modules, interfaces, shared data models, tests, documentation surfaces, and deployment constraints. The cover constructor must synthesize hypercovers that make extension feasible. The evidence policy admits authored clauses, donor manifests, domain-pack treaties, solver discharge, runtime traces, and controlled proposals. The closure criterion is not "all text generated" but existence of a sufficiently glued section with explicit residuals and trustworthy manifests. This makes orchestration a semantic control problem rather than prompt queue management.

14.6 Documentation alignment

For documentation alignment, the coordinate family pairs executable and explanatory supports, and R is truth-preserving alignment between code clauses and public clauses. The witness extractor emits omitted guarantees, stale examples, false claims, and downgraded trust. The closure criterion is public honesty under current evidence. Seen this way, documentation is not a sidecar product but another relational mode of the same kernel.

14.7 Migration and donor inheritance

Migration and donor inheritance instantiate the atlas with cross-version or cross-project transport maps. Coordinates are versioned or donor-recipient paired sites. The target relation is admissible transport of clauses, examples, and certificates through an adaptation map. The evidence policy must distinguish transported donor support from newly established recipient support. The obstruction extractor identifies nontransportable treaties, renamed or missing APIs, and donor assumptions that no longer hold. The same domain-pack registry that helps synthesis also helps vocabulary and API transport.

14.8 Regression closure

Regression closure is the mode in which the current section must remain coherent under local change. Coordinates are changed supports plus their dependency shadows. The target relation is preservation of previously attained clause families and witnesses outside reopened regions. The obstruction extractor is driven by archived counterexamples, invalidated certificates, and reopened treaties. Closure is achieved when every affected artifact has either been re-certified or truthfully downgraded. This mode makes locality the decisive implementation property: if reopening is coarse, the whole thesis weakens.

14.9 Research ideation and theorem discovery

Research ideation is not a break from the atlas but its most ambitious instance. Coordinates are theory-space regions, candidate lemma sites, analogy bridges, and purpose-conditioned frontier states. The target relation is semantic leverage relative to a current obstruction field or design goal. Evidence may be partial, exploratory, and explicitly low-trust, but still typed. The obstruction extractor measures not only contradiction but also expressive insufficiency: missing language, missing invariants, missing abstractions. The same archive, provenance graph, and closure discipline used for software work become the backbone of mathematically serious ideation.

14.10 Performance obligations

Performance reasoning fits the atlas once resource claims are treated as clauses over execution and allocation supports. Coordinates include hot paths, input regimes, cache states, and deployment environments. The target relation may be a unary budget claim or a relational "no worse than" refinement. Witnesses come from profilers, traces, solver reasoning over size relations, or benchmark manifests. The obstruction language then expresses performance regressions, missing asymptotic arguments, and environment-scoped claims in the same form used for functional bugs.

14.11 Concurrency failures

Concurrency failures require richer coordinates, not a different semantic machine. Coordinates range over threads, tasks, shared resources, synchronization boundaries, and schedule fragments. Overlaps encode happens-before assumptions, lock discipline, and message-order constraints. Witnesses are schedules, interleavings, trace pairs, and model-checker counterexamples. Repairs remain local surgery under boundary treaties, except that the boundary now includes temporal and ownership constraints. If the implementation has a witness archive and a relational checker, concurrency becomes a demanding but natural problem class rather than a foreign add-on.

14.12 API drift

API drift is a surface-manifest obstruction. Coordinates are versioned public surfaces: names, signatures, effects, error behavior, and explanatory examples. The target relation is backward compatibility, forward compatibility, or a declared migration treaty. Witnesses are breaking calls, changed outputs, altered error surfaces, and unsupported explanation claims. The same machinery used for documentation alignment and relational refinement should therefore power API

drift detection. A separate drift checker that cannot cite clause IDs, trust levels, and migration obligations would be architecturally inconsistent.

14.13 Generated-code governance

Generated-code governance is the atlas mode in which provenance and policy are primary coordinates. The target relation is not only behavioral admissibility but admissibility of origin, trust, review status, and export rights. Witnesses include unverified generated fragments, unsupported public claims, untransported donor assumptions, and provenance gaps. Closure may require proof, replay, human audit, or explicit public downgrading. This is why provenance graphs, manifests, and certificate integrity checks belong in the semantic core rather than in release engineering alone.

Proposition 14.2 (Atlas unification target)**.** *If each problem class above can be factored through the same judgment kernel $\mathcal{P}$ with only its parameters changed, then advances in cover synthesis, witness transport, residual prioritization, trust projection, and domain-pack elaboration should transfer across modes. If a problem class resists this factorization except by ad hoc side channels, then either the atlas is incomplete or the central thesis is overstated. The chapter is therefore not descriptive only; it states a falsifiable implementation program.*

Part IV

Python Semantics

Chapter 15

Names, scopes, closures, and module state

Python name semantics is where coordinate formation becomes operational rather than abstract. Names are introduced under nested scopes, rebound through module and class execution, captured by cells, and later reopened by import, live mutation, or hot reload. The implementation consequence is that the runtime should not treat Python binding as a thin front-end concern. It should preserve scope constructors, environment summaries, capture routes, and epoch-sensitive module state as first-class semantic objects.

15.1 Scope semantics: coordinate formation over locals, globals, and cells

The scope system should be treated as a coordinate constructor for later obligations. A name is never meaningful in isolation. It is meaningful only relative to a scope stack, a cell-capture boundary, a module epoch, and the mode of later use. A useful scope coordinate is

$$\mathcal{C}_{\text{scope}} = (\Gamma_{\text{loc}}, \Gamma_{\text{glob}}, \Gamma_{\text{cell}}, m, \eta, \kappa),$$

where Γ_{loc} is the local environment summary, Γ_{glob} the module-level global summary, Γ_{cell} the captured-cell family, m the owning module coordinate, η the active epoch, and κ the usage kind such as read, write, closure capture, import projection, or reflective access. This makes scope part of the semantic address of later judgments.

The implementation consequence is that name resolution should emit coordinate-bearing binding records rather than only AST-local symbol-table entries. A later solver obligation, explanation, or repair candidate should be able to say not merely which identifier was involved, but which scope coordinate governed it. One wants theorem targets of resolution determinism on the admitted fragment and scope-preserving normalization from syntax into binding records.

15.2 Global and local bindings: obligation ownership and module state

Global and local bindings should be represented as ownership relations over obligations, not merely as variable maps. When a function body refers to a global name, the resulting obligations do not belong wholly to the function's local region. They partially depend on the owning module state.

Likewise, a local rebinding may discharge one clause while reopening another that depends on shadowing or rebinding discipline. This means binding ownership should be explicit.

A suitable module-state record is

$$\mathcal{M}_{\mathrm{mod}} = (\Gamma_{\mathrm{exp}}, \Gamma_{\mathrm{priv}}, \Theta_{\mathrm{mod}}, \Sigma_{\mathrm{imp}}, \eta),$$

where Γ_{exp} is the exported binding family, Γ_{priv} the private module family, Θ_{mod} the module-level obligation and invariant family, Σ_{imp} the import-state summary, and η the module epoch. A local binding judgment should then point back into this object whenever a name escapes local scope or depends on module-global law.

The theorem burden is ownership honesty and epoch-sensitive reopening. Ownership honesty says that a local judgment may not pretend independence from module state when its support references global bindings. Epoch-sensitive reopening says that rebinding a module-level name reopens exactly those judgments whose support depends on the old module epoch. This is how Python's apparently simple global/local discipline becomes implementation-guiding rather than informal.

15.3 Closure capture: cell transport, latent context, and proof consequences

Closure capture should be treated as transport of latent context into a later semantic event. A closure is not merely a function bundled with some values. It is a transport boundary across which obligations, assumptions, and fragility may pass from definition time to call time. The implementation should therefore preserve not only captured values but the semantic role of each captured cell.

A useful closure-capture record is

$$\mathcal{L}_{\mathrm{clos}} = (\Gamma_{\mathrm{cell}}, \Theta_{\mathrm{cap}}, \rho_{\mathrm{trans}}, \eta_{\mathrm{def}}, \eta_{\mathrm{use}}),$$

where Γ_{cell} is the captured-cell family, Θ_{cap} the obligations transported by capture, ρ_{trans} the transport law governing how those obligations are reused at call sites, and $\eta_{\mathrm{def}}, \eta_{\mathrm{use}}$ the definition and use epochs. This lets the system distinguish stable captured constants, mutable cells, and captures whose meaning depends on later module evolution.

The theorem targets are capture-faithfulness, transport coherence, and stale-cell honesty. Capture-faithfulness says that the closure summary faithfully records which cells and assumptions were transported. Transport coherence says that later method or function contracts may refer to captured context only through the declared transport law. Stale-cell honesty says that if a captured cell's meaning has changed across epochs without an admitted transport witness, the system must reopen affected judgments rather than silently reusing the older closure story.

Chapter 16

Function values, method binding, class construction, and descriptor lookup

This chapter should state plainly that Python object semantics is not a fringe appendix to the thesis. If JuGeo aims to reason about real code, then function values, binding routes, class creation stages, and descriptor-mediated lookup must be treated as central semantic objects. The implementation consequence is that the runtime should preserve route tags, epochs, receiver assumptions, and staged class-construction records rather than flattening these phenomena into generic call edges and field reads.

16.1 Function values and method values: receiver binding, dispatch route, and contract transport

Function values and method values need distinct semantic records because Python binding is not mere currying. I will use

$$\mathcal{F} = (\mathrm{code}, \Gamma_{\mathrm{def}}, \Gamma_{\mathrm{cell}}, \mathrm{defaults}, \mathrm{kwdefaults}, \mathrm{ann}, \mu, \eta)$$

for a function value and

$$\mathcal{M} = (\mathcal{F}, o, \rho, \Theta_o, \tau, \eta_{\mathrm{bind}})$$

for a method value. Here Γ_{def} is the defining global environment summary, Γ_{cell} the captured-cell family, μ the function-level effect summary, η the defining epoch, o the receiver abstraction, ρ the descriptor route by which binding occurred, Θ_o the receiver-side invariant context, and τ the transport map carrying the function contract into a receiver-indexed method contract.

The operational judgment is not "look up a callable and add one argument." It is a route-tagged binding judgment

$$\mathrm{Bind}(o, a) \Downarrow (r, m, \phi_{\mathrm{ok}}, \phi_{\mathrm{exc}}, \epsilon),$$

where r distinguishes at least direct function binding, recognized descriptor binding, `staticmethod`, `classmethod`, dynamic fallback, and failure. The algorithm is: first perform route-tagged attribute lookup; then, only on routes that produce bindable descriptors, construct $\mathcal{M}$ by transporting the receiver invariant into the callable contract; otherwise return the unbound value or an exceptional branch. This keeps method identity, receiver assumptions, and descriptor provenance explicit.

The theorem burden is concrete. One wants binding-route soundness, stating that recognized routes agree with Python's admitted binding behavior; transport coherence, stating that τ neither

strengthens nor weakens the function contract except through the declared receiver assumptions; and method-equality honesty, stating that any comparison of method values mentions ρ, Θ_o, and epoch. The implementation consequence is equally concrete: the runtime authority center needs separate datatypes for function values, descriptor results, and bound methods, and every call-edge explanation must retain the binding route that produced it.

16.2 Class objects: construction pipeline, instance families, and invariant transport

A class object should be normalized into a staged record

$$\mathcal{C} = (\text{bases}, \text{body\_env}, \text{dict}, \text{mro}, \text{meta}, \Theta_{\text{cls}}, \Theta_{\text{inst}}).$$

This record exposes the semantic faces that matter for implementation: class creation, class-surface behavior, instance-family behavior, and invariant transport through inheritance. The class chapter should therefore generate distinct coordinates for class creation, class-level laws, instance-level laws, and method-level contracts parameterized by receiver assumptions.

16.3 Descriptor lookup: route-tagged attribute resolution, effect summaries, and explanation obligations

Attribute access should be represented by a route-tagged lookup judgment rather than by a generic field-read abstraction. For object abstraction o and attribute name a, the system should compute

$$\text{Lookup}(o, a) \Downarrow (r, v, \phi_{\text{ok}}, \phi_{\text{exc}}, \epsilon, \mu),$$

where r is the route tag, v the resulting value abstraction, ϕ_{ok} and ϕ_{exc} the success and exceptional conditions, ϵ the exception family, and μ an effect summary for mutation, allocation, caching, or validation performed during lookup.

The first implementation should distinguish at least the following routes: data descriptor, instance dictionary hit, non-data descriptor, plain class attribute, dynamic fallback through `__getattr__` or `__getattribute__`, and explicit failed lookup. The corresponding algorithm is: compute the MRO lookup chain; check recognized data descriptors first; if none apply, inspect the instance dictionary summary; then inspect non-data descriptors or plain class attributes; then handle dynamic fallback under explicit uncertainty; otherwise emit a failed-lookup route. This is not merely explanatory prose. It is the algorithm that Python-object domain packs and explanation machinery should follow.

The theorem burden is also explicit. One wants recognized-route soundness, effect-summary faithfulness, and explanation fidelity. Recognized-route soundness says that emitted route tags agree with Python lookup behavior on the admitted fragment. Effect-summary faithfulness says that a descriptor summarized as observationally inert, cache-only, or validation-only must replay that way under the same abstraction. Explanation fidelity says that every public explanation about attribute access can be traced back to a route-tagged lookup record and its support assumptions. Only with these concrete claims does descriptor semantics become implementable rather than atmospheric.

Chapter 17

Heap objects, identity, aliasing, and mutation

Python semantics is not exhausted by names, call edges, and control flow. A large fraction of real behavior is mediated by heap objects whose state may be shared, mutated, compared by identity, or observed through descriptor and container routes. The implementation consequence is immediate: the runtime authority center should maintain explicit heap-object records, alias-region summaries, mutation epochs, and observation routes. If these are collapsed into anonymous "values," then repair impact, cache invalidation, contract transport, and explanation fidelity all become less honest than the thesis requires.

The chapter should therefore distinguish three things that ordinary static accounts often blur together: the carrier of a value, the identity token by which a heap object persists through time, and the observational surface by which clients can witness change. The AG language of support overlap becomes concrete here: aliasing is literally overlap in the support of judgments about mutable state, and mutation is a local change whose reopening effect depends on that overlap graph. In sheaf language, mutable-object reasoning is descent over overlapping observation regions whose compatibility can be broken by updates.

17.1 Primitive and heap-mediated values: carriers, state, and observational consequences

A practical semantic account should separate immediate carriers from heap-mediated carriers. I will write

$$V \ ::= \ \mathsf{Prim}(k, c) \mid \mathsf{HeapRef}(\iota, \eta)$$

where $\mathsf{Prim}(k, c)$ denotes a primitive-style value with kind tag k and canonical representative c, while $\mathsf{HeapRef}(\iota, \eta)$ denotes a reference to heap object ι observed at epoch η. The heap object itself should be normalized as

$$\mathcal{H}_\iota = (\iota, \kappa, \sigma, \Theta, \eta_{\mathrm{alloc}}, \eta_{\mathrm{now}}, \alpha, \mu),$$

where κ is the object kind, σ is the object-state summary (dictionary, slots, buffer, closure cells, or other carrier-specific content), Θ is the invariant family attached to the object, α is the current alias-region summary, and μ is the effect history relevant for replay and explanation.

The central observation judgment should not be a generic read. It should be a route-tagged observational judgment

$$\mathrm{Observe}(v, \pi) \Downarrow (r, w, \phi_{\mathrm{ok}}, \phi_{\mathrm{exc}}, \epsilon, \eta'),$$

where π is an observation path, r is the route tag, w is the observed result abstraction, and η' is the epoch at which the observation is valid. For the first implementation, r should distinguish at least primitive projection, heap-field read, slot read, buffer read, descriptor-mediated read, container indexing, iterator-mediated observation, and residual dynamic observation. This matters because the same surface expression may have different invalidation and theorem consequences depending on the route by which it was obtained.

The staged implementation algorithm is straightforward:

1. normalize the input into either Prim or HeapRef;
2. if primitive, return an observationally stable result with no heap dependency;
3. if heap-mediated, acquire the object summary $\mathcal{H}_\iota$ at the current epoch;
4. resolve the observation path π through the carrier-specific route table;
5. emit the observed result together with the epoch dependency and any exceptional branches;
6. record which support coordinates depend on this observation so that later mutation can reopen them locally rather than globally.

The theorem burden is implementation-facing. One wants carrier honesty, saying that primitive and heap-mediated cases are not conflated; snapshot faithfulness, saying that any observation justified at epoch η' is replayable against the same object summary; and route fidelity, saying that public explanations about reads and projections can be traced to an admitted route tag. The implementation consequence is that object summaries, not merely values, must be first-class cached artifacts.

17.2 Aliasing as shared geometry: support overlap, mutation, and frame conditions

Aliasing should be described as shared support rather than as an afterthought of pointer analysis. Two judgment coordinates overlap in the heap exactly when their evidence or obligations depend on the same identity token or on an alias region containing that token. That suggests the record

$$\mathcal{A} = (\Lambda, \sim, \mathrm{may}, \mathrm{must}, \eta),$$

where Λ is the family of tracked object identities, $\sim$ is the maintained alias partition or approximation, and may and must summarize the may-alias and must-alias consequences exported to other chapters.

Mutation should then be expressed by a route-tagged update judgment

$$\mathrm{Mutate}(\iota, \pi, \delta) \Downarrow (r, \eta^+, \phi_{\mathrm{fr}}, \phi_{\mathrm{exc}}, \epsilon, \Delta_{\mathrm{inv}}),$$

where π is the target path inside object ι, δ is the update description, r is the update route, ϕ_{fr} is the resulting frame condition, and Δ_{inv} is the invalidation frontier induced by the update. The route tag should distinguish at least direct dictionary write, slot write, descriptor-backed write, mutating method call, in-place container update, view mutation, weak update under alias uncertainty, and residual live mutation.

The implementation algorithm should be staged rather than monolithic:

1. resolve the target identity and target path;

2. compute the must-alias and may-alias neighborhoods relevant to the update;
3. classify the update as strong, weak, view-mediated, or residual;
4. bump the epoch of every definitely affected object summary;
5. derive a frame condition recording what remains unchanged under the admitted abstraction;
6. reopen only those obligations whose support overlaps the affected alias region.

This is where the AG language earns its keep. A mutation is local when the overlap graph is explicit; without that graph, the implementation is forced into coarse reopening. Cohomologically, alias-induced interference is what turns seemingly separate local sections into coupled data on overlaps.

The theorem targets are concrete. Frame honesty says that any coordinate declared unaffected by the update remains observationally stable under the same alias assumptions. Support-local invalidation says that a mutation reopens every dependent coordinate in the tracked overlap region and need not reopen coordinates outside it. Alias monotonicity says that when the analysis weakens from must-alias to may-alias, the system may lose precision but must not fabricate noninterference. The implementation consequence is that alias summaries and epoch bumps belong in the same authority center as heap-object records rather than in a detached optimizer.

17.3 Identity and equality: observational criteria and proof consequences

Python forces a clean separation between identity and equality. Identity is witnessed by `is`; equality is a routed behavioral relation that may call user code, return `NotImplemented`, mutate state, or raise. The semantics should therefore expose two distinct judgments:

$$\mathrm{Identical}(v_1, v_2) \Downarrow (\mathsf{yes/no}, \phi, \eta)$$

and

$$\mathrm{Equal}(v_1, v_2) \Downarrow (r, b, \phi_{\mathrm{ok}}, \phi_{\mathrm{exc}}, \epsilon, \mu).$$

The first is about shared identity token or admitted immediate canonicality. The second is about the route by which comparison was attempted. For the first implementation, r should distinguish primitive canonical equality, left rich-comparison, right rich-comparison, symmetric container comparison, identity fallback, `NotImplemented` escalation, and exceptional comparison.

This distinction matters for proof obligations. A theorem about identity should mention heap tokens and epochs; a theorem about equality should mention the comparison route, the behavioral assumptions on `__eq__`, and any side effects incurred during comparison. In particular, substitutivity is not automatic. One only gets an observational substitutivity claim relative to a coordinate family that is stable under the same comparison route and effect assumptions. This is especially important for memoization, cache keys, set membership, and repair explanations that rely on "sameness."

The theorem burden is therefore threefold. Identity soundness says that admitted identity claims coincide with shared heap token or admitted immediate canonicality on the tracked fragment. Equality-route honesty says that every discharged equality claim identifies the route by which it was established. Observational substitutivity says that if a coordinate family is declared stable under a comparison relation R, then replacing v_1 by v_2 is licensed only when the proof object names R and its support assumptions. The implementation consequence is simple but nonnegotiable: certificates and explanations must never print "equal" without also carrying which equality relation was used.

Chapter 18

Exceptions, context managers, iterators, generators, and async

Python control semantics is not a single successful trace decorated with occasional errors. Exceptions, generator suspension, iterator exhaustion, context-manager cleanup, and `await` points are distinct semantic routes through a computation. A judgment geometry that ignores these routes cannot explain why a proof reopened, why a cleanup obligation survived a failure path, or why a coroutine was cancelled only at a suspension boundary. The implementation therefore needs route-tagged result records, temporal obligations, and suspension-state summaries as first-class objects.

The unifying idea is that many Python computations produce not just values but staged outcomes. Return, raise, yield, await, stop, and cancel are different constructors of the same operational surface. In AG terms, they define different local charts on the control-state sheaf; exceptional transport failure is often a failure of matching along those charts rather than a mere alternate return value. This chapter should normalize them so that theorem statements and runtime traces speak one language.

18.1 Exceptions as alternate semantic paths: branch structure and witness extraction

A useful result domain should make exceptional paths explicit:

$$\mathcal{R} ::= \mathsf{Ret}(v, \eta) \mid \mathsf{Raise}(x, \eta, \pi) \mid \mathsf{Yield}(y, \mathcal{S}) \mid \mathsf{Suspend}(\mathcal{S}) \mid \mathsf{Stop}(p, \eta),$$

where x is an exception witness, π records the raising site, and $\mathcal{S}$ is a suspension state. For ordinary statement or expression execution the judgment should be

$$\mathrm{Exec}(n, \Sigma) \Downarrow (r, q, \phi_{\mathrm{ok}}, \phi_{\mathrm{exc}}, \epsilon, \mu),$$

with route tag r distinguishing normal return, direct raise, translated raise, handler-caught raise, `finally`-mediated raise, implicit stop, and residual dynamic failure.

An exception witness should itself be structured:

$$\mathcal{X} = (\mathrm{cls}, \mathrm{payload}, \mathrm{cause}, \mathrm{context}, \pi, \eta).$$

This is not bureaucracy. It is what allows later chapters to say whether an import failed before or after partial initialization, whether a cancellation was converted into a domain exception, or

whether a context manager suppressed a particular family of failures. Public explanations should cite $\mathcal{X}$-records rather than flattened traceback strings.

The theorem targets are clear. Exceptional-branch preservation says that normalization and routing do not erase admitted exceptional branches. Catch-route soundness says that when a handler is reported to catch an exception family, that claim matches Python subclass matching on the admitted fragment. Witness extraction fidelity says that the public exception explanation is reconstructible from the internal $\mathcal{X}$ record and the recorded route. The implementation consequence is that exception families must survive every phase boundary rather than being squeezed into booleans.

18.2 Context managers: temporal obligations, cleanup laws, and effect boundaries

A context manager is not merely syntax sugar around two calls. It is a temporal contract with an entry phase, a body phase, and an exit phase whose arguments depend on the body outcome. The semantic record should be

$$\mathcal{W} = (\text{enter}, \text{exit}, \Theta_{\text{pre}}, \Theta_{\text{body}}, \Theta_{\text{post}}, \mu_{\text{enter}}, \mu_{\text{exit}}),$$

where the Θ components record the obligations and guarantees at each phase boundary. The operational judgment should be

$$\text{With}(w, B) \Downarrow (r, q, \phi_{\text{body}}, \phi_{\text{cleanup}}, \epsilon, \mu),$$

where r distinguishes at least enter-failure, normal-body/propagate, normal-body/suppress, exceptional-body/propagate, exceptional-body/suppress, and exit-failure.

The staging algorithm should be made explicit:

1. evaluate the manager expression and resolve the `__enter__`/`__exit__` routes;
2. execute `__enter__` and bind the body variable if needed;
3. execute the body under the body obligations;
4. package the body outcome as either normal or exceptional triple;
5. call `__exit__` with that triple;
6. branch on the suppression result and emit the post-body obligations that remain open.

This gives the implementation a precise place to attach cleanup claims, suppression explanations, and effect boundaries.

The theorem burden is concrete. Cleanup-law soundness says that admitted context managers run their exit path on every route that Python would require within the fragment. Suppression honesty says that any claim of exception suppression is supported by the recorded exit result. Effect-boundary faithfulness says that effects attributed to the body are not silently reassigned to the manager, or conversely. The implementation consequence is that `with` should compile to a staged semantic node, not to a macro-expanded block that loses phase information.

18.3 Async and task semantics: suspended state, awaits, and concurrent obligations

Iterators, generators, coroutines, and async generators all rely on suspended state. A single record should capture the common core:

$$\mathcal{S} = (\mathrm{kind}, \mathrm{frame}, \Gamma_{\mathrm{cell}}, pc, \eta, \Theta, \chi),$$

where $\mathrm{kind} \in \{\mathrm{iter}, \mathrm{gen}, \mathrm{coro}, \mathrm{agen}\}$, pc is the resumable control point, and χ summarizes the accepted resume operations. The central operational judgment is

$$\mathrm{Resume}(\mathcal{S}, u) \Downarrow (r, \mathcal{S}', o, \phi_{\mathrm{ok}}, \phi_{\mathrm{exc}}, \epsilon, \mu),$$

where u is the resume input and r distinguishes `next`, `send`, `throw`, `await`, yield, normal return, `StopIteration`, `StopAsyncIteration`, and cancellation injection.

The implementation should stage resume as follows:

1. recover the suspended frame and its epoch-indexed locals;
2. validate that the requested resume operation is admitted by χ;
3. inject value, exception, or cancellation into the frame;
4. step until the next suspension, terminal return, or exceptional escape;
5. emit the updated suspension state together with any yielded value or terminal outcome.

This same structure handles iterator exhaustion, generator `return` payload transport, coroutine awaiting, and async-generator stop conditions. It also gives a clean place to record that cancellation is only delivered at recognized suspension points. In geometric terms, resumption glues local control charts along explicit suspension boundaries, and cancellation is a nontrivial transport event across those charts.

The theorem targets are suspension-state faithfulness, await-transport soundness, and cancellation-route honesty. Suspension-state faithfulness says that a resumed frame behaves like the recorded $\mathcal{S}$ under the same environment summary. Await-transport soundness says that obligations flowing across an `await` are neither duplicated nor forgotten. Cancellation-route honesty says that if a coroutine is reported cancelled, the certificate identifies the suspension boundary at which the cancellation became visible. The implementation consequence is that generators and coroutines need structured state summaries rather than opaque runtime handles.

Chapter 19

Imports, package fixed points, re-exports, and dynamic loading

Import semantics is execution plus namespace transport under a global module cache. It is therefore not adequately modeled as a pure name-resolution step. Python imports may allocate a module shell, expose partial state through `sys.modules`, re-export names through package surfaces, and converge only to a staged fixed point in the presence of cycles. The implementation consequence is that imports require their own authority center, with explicit module states, transport maps, and epoch-indexed cache entries.

The AG viewpoint is especially natural here. A package is not just a directory; it is a family of local module sections that may agree, partially agree, or obstruct one another on overlap through re-exports and cyclic expectations. The fixed-point language should be taken literally: import closure is a least staged settlement of module states compatible with execution order and cache visibility on the admitted fragment.

19.1 Import is execution plus namespace transport

A module should be represented by

$$\mathcal{M} = (\text{qname}, \text{spec}, \text{dict}, \text{exports}, \text{deps}, \eta, \sigma),$$

where $\sigma \in \{\text{absent}, \text{resolving}, \text{partial}, \text{ready}, \text{failed}\}$ records the initialization state. The import judgment should be

$$\text{Import}(q, \nu) \Downarrow (r, \mathcal{M}, \tau, \phi_{\text{ok}}, \phi_{\text{exc}}, \epsilon, \Delta),$$

where q is the qualified name, ν is the import form, and τ is the namespace-transport map describing which bindings are exported to the caller. The route tag r should distinguish cache hit, fresh load, cycle-visible partial load, package attribute transport, explicit re-export, star-import transport, and failure.

The staging algorithm should be implementation-level doctrine:

1. resolve the module spec and loader route;
2. if the module is absent, allocate a shell and place it in `sys.modules` before body execution;
3. mark the module as `resolving` and execute the top-level body;

4. materialize the exported namespace, respecting package surfaces, `__all__`, and admitted re-export rules;

5. finalize the module as `ready` or `failed`;

6. transport bindings into the caller according to the import form.

This explains both partial initialization and import cycles. A cyclic import is not magical; it is a route in which a consumer reads a module whose shell is visible but whose exported surface is only partially settled. For package-level re-exports and star imports, the transport map τ must record whether a name came from the defining module, a package aggregator, or a re-export chain.

The theorem targets are equally concrete. Cache-coherence soundness says that repeated imports of a ready module return the same module identity unless an explicit epoch change is recorded. Partial-initialization honesty says that any use of a not-yet-ready module is tagged as such rather than silently treated as a fully initialized section. Namespace-transport fidelity says that `import`, `from ... import ...`, and star-import claims are justified by the recorded transport map. The implementation consequence is that module states and transport maps must be retained for explanation and invalidation.

19.2 Import cycles and package fixed points

Import cycles should be treated as fixed-point problems over partially exposed module shells. If S is a strongly connected component of the import graph, then the relevant semantic object is not a linear load order but a staged operator

$$\mathfrak{F}_S : \prod_{m \in S} \mathcal{M}_m \to \prod_{m \in S} \mathcal{M}_m$$

whose iterates reflect shell creation, partial export exposure, and later stabilization. The AG reading is that the SCC is a local overlap cluster whose descent data cannot be judged from a single module in isolation.

The theorem burden is cycle fixed-point stability: on the admitted fragment, cyclic package import reaches a staged fixed point whose exported summaries are stable unless a participating module reopens. The implementation consequence is that SCC-level import state should be cached and invalidated as a group rather than as a bag of unrelated files.

19.3 Re-exports, star imports, and package surfaces

Package surfaces should be treated as exported overlap objects rather than convenience syntax. Re-exports and star imports alter which clauses are publicly visible on a package coordinate and therefore which downstream modules depend on them. The implementation should maintain explicit package-surface summaries with provenance: defined-here, re-exported, star-exported, or dynamically residual.

This is again AG in an operational form. A package surface is a local chart on the public namespace sheaf, and re-export chains are transport maps between charts. When those maps become inconsistent or stale, the failure should appear as a public-surface obstruction rather than a mysterious missing import.

19.4 Dynamic import and reflection

Dynamic import widens the semantic aperture. The relevant judgment should be

$$\mathrm{DynImport}(e, \Sigma) \Downarrow (r, q, \mathcal{M}, \phi_{\mathrm{ok}}, \phi_{\mathrm{exc}}, \epsilon, \mu),$$

where e is the import expression and r distinguishes literal-name import, computed-name import under admitted classifier, loader-hook import, reflective `__import__`, `importlib`-mediated import, direct `sys.modules` observation, and residual dynamic import. Reflection over modules should be treated similarly: `getattr(pkg,name)`, `hasattr`, and direct cache inspection are not mere field reads but routes through a live module surface.

The implementation should admit a graded algorithm. If the module name is a literal string, route through ordinary import semantics. If it is computed but constrained to a finite family, refine the result to that family and attach the classifier assumptions. If custom loaders or import hooks are involved, import should become an effectful plugin boundary with explicit residual obligations. If the code inspects `sys.modules` directly, the result should be tied to the current module-cache epoch rather than treated as timeless fact.

This is the place to be explicit about non-theorems. The thesis should not claim complete semantic recovery for arbitrary import hooks, custom loaders backed by native code, or reflective mutation of package objects outside the tracked fragment. What it should claim is route honesty: every dynamic-import explanation names the route taken, the assumptions under which refinement was possible, and the residual uncertainty left behind. The implementation consequence is that dynamic import must never be hidden under the same certificate vocabulary as ordinary static import.

19.5 Proof targets for import semantics

The theorem burden for import semantics should be stated as a finite list of implementation-guiding targets rather than as vague confidence in module handling. The core targets are:

1. *module-cache coherence*: if a module remains at the same epoch and no explicit live mutation touches its record, repeated import yields the same module identity and compatible export summary;

2. *partial-state honesty*: any judgment using a module in state `resolving` or `partial` carries that state into its support assumptions;

3. *transport fidelity*: import-form explanations correspond to the recorded transport map, including re-export and star-import routes;

4. *cycle fixed-point stability*: on the admitted fragment, cyclic package import reaches a staged fixed point whose exported summaries are stable unless a participating module reopens;

5. *dynamic-route honesty*: reflective or loader-mediated import claims always record their route and any residual uncertainty.

The theorem layer should also declare what it refuses to prove. It does not claim extensional equivalence of all import orders, because top-level execution can have order-sensitive effects. It does not claim completeness for arbitrary import hooks. It does not identify package-level name visibility with source-text presence alone. These non-theorems are part of the implementation contract: they determine when the runtime should residualize instead of overclaiming.

The implementation consequence is sharp. There should be a dedicated import authority center whose cache keys include module identity and epoch, whose outputs include transport maps and partial-initialization status, and whose invalidation machinery is shared with hot reload and live mutation rather than treated as a parsing concern.

Chapter 20

Class creation, metaclasses, descriptors, and behavioral surfaces

A Python class is not merely a dictionary handed to `type`. It is the result of a staged program involving base normalization, metaclass selection, body execution in a prepared namespace, class-object construction, descriptor finalization, and inheritance-mediated surface generation. If the thesis is serious about real Python, then class semantics must be carried as staged records rather than flattened into nominal type declarations.

The implementation consequence is that JuGeo should treat class creation as a pipeline with explicit intermediate states. Metaclasses and descriptors are not exotic edge cases; they are contract transformers that shape the public behavioral surface of both class objects and instances. In AG terms, class construction glues local namespace sections into a surface sheaf whose later routes must remain traceable back to the gluing history.

20.1 Class creation as staged semantics

A class-construction record should make the stages explicit:

$$\mathcal{K} = (\text{name}, \text{bases}, \text{kw}, \rho_{\text{prep}}, E_{\text{body}}, \rho_{\text{new}}, \rho_{\text{init}}, \text{mro}, \Theta_{\text{cls}}, \Theta_{\text{inst}}, \eta).$$

Here ρ_{prep} is the route by which the namespace was prepared, E_{body} is the body-execution summary, and $\rho_{\text{new}}, \rho_{\text{init}}$ are the metaclass construction routes. The judgment should be

$$\text{BuildClass}(h, b, \kappa) \Downarrow (r, \mathcal{C}, \phi_{\text{ok}}, \phi_{\text{exc}}, \epsilon, \mu),$$

where h is the class header, b the body, κ the surrounding context, and r distinguishes base-normalization, metaclass conflict, namespace preparation, body execution, class-object creation, `__set_name__` finalization, and post-creation hooks such as `__init_subclass__`.

The staged algorithm should be explicit:

1. normalize bases, including admitted `__mro_entries__` behavior;
2. select the metaclass and detect conflicts;
3. call `__prepare__` if present and obtain the body namespace;
4. execute the class body in that namespace;

5. invoke metaclass `__new__` and `__init__`;

6. run descriptor finalization such as `__set_name__`;

7. compute MRO, class invariants, and instance-family invariants.

This staging is not optional ornamentation. It is what lets the implementation explain partial initialization, injected attributes, and why a metaclass changed the eventual behavioral surface.

The theorem targets are stage soundness, MRO honesty, and invariant transport. Stage soundness says that any admitted class object corresponds to the recorded pipeline. MRO honesty says that method and attribute routes later derived from the class agree with the computed linearization. Invariant transport says that class-level and instance-level contracts introduced during construction remain properly indexed by the class that actually emerged from the pipeline. The implementation consequence is that class creation needs its own staged artifact, not just the final `dict`.

20.2 Descriptor resolution routes

Descriptor lookup should be refined beyond the earlier generic attribute chapter to account for instance surfaces, class-object surfaces, and `super()`-mediated surfaces. The operative judgment should be

$$\mathrm{ResolveDescr}(s, a, \chi) \Downarrow (r, v, \phi_{\mathrm{ok}}, \phi_{\mathrm{exc}}, \epsilon, \mu),$$

where s is the surface owner, a the attribute name, and $\chi \in \{\mathrm{inst}, \mathrm{class}, \mathrm{super}\}$ identifies the lookup regime. The route tag r should distinguish data descriptor, instance dictionary hit, non-data descriptor, plain class attribute, metaclass descriptor, `super()` route, dynamic fallback, and explicit failure.

This matters because the precedence order differs across surfaces. Instance access gives data descriptors priority over instance state; class-object access itself may trigger descriptor logic on the metaclass; `super()` skips part of the MRO but preserves descriptor application. A single untagged "attribute read" loses precisely the information that later proof and repair stages need. The implementation should therefore record not only the resulting value but the route and the class/MRO coordinate at which the route was resolved.

The theorem burden is route completeness on the admitted fragment, precedence honesty, and descriptor-finalization coherence. Route completeness says that the recognized tags cover the intended Python fragment. Precedence honesty says that if the runtime emits a data-descriptor route, then the other competing routes were correctly subordinated. Finalization coherence says that descriptor behavior after class creation respects the `__set_name__` and metaclass stages that produced it. The implementation consequence is that descriptor resolution must be an explicit authority center shared by object lookup, class lookup, and surface-generation passes.

20.3 Metaclasses as contract transformers

A metaclass is a transformation on class construction and class surface, not merely a nominal superclass of classes. The thesis should therefore model a metaclass by a contract-transformer record

$$\mathcal{T}_{\mathrm{meta}} = (\Theta_{\mathrm{in}}, \Theta_{\mathrm{out}}, \tau_{\mathrm{attrs}}, \tau_{\mathrm{methods}}, \mu, \eta),$$

where Θ_{in} and Θ_{out} record the pre- and post-transformation contract spaces, while τ_{attrs} and τ_{methods} describe how attributes and methods are rewritten, wrapped, or generated. The operational

judgment is

$$\mathrm{MetaApply}(m, \mathcal{K}) \Downarrow (r, \mathcal{C}, \phi_{\mathrm{ok}}, \phi_{\mathrm{exc}}, \epsilon, \mu),$$

with routes for transparent pass-through, namespace rewrite, method wrapping, descriptor injection, registration side effect, and failure.

This formulation matters because many real Python libraries use metaclasses to synthesize fields, register subclasses, install descriptors, or enforce structural constraints. Those effects are not external annotations; they alter the semantic surface that later code actually sees. The implementation should therefore capture recognized metaclass behaviors as typed transformations and residualize the rest.

The theorem targets are transformer disclosure, contract monotonicity or declared weakening, and class/instance-surface coherence. Transformer disclosure says that any recognized metaclass effect is surfaced in the semantic record rather than hidden behind the final class object. Contract monotonicity says that a metaclass may strengthen or weaken obligations only through an explicit declared route. Surface coherence says that claims about instance behavior generated by metaclass action remain compatible with the actual class surface. The implementation consequence is that metaclass summaries belong beside class summaries, not in an optional plugin log.

20.4 Generated behavioral surfaces

A Python class exports more than named attributes. It exports a behavioral surface determined by special methods, descriptors, inheritance, and metaclass-generated rewrites. A useful surface record is

$$\mathcal{S}_{\mathrm{beh}}(\mathcal{C}) = (\Sigma_{\mathrm{call}}, \Sigma_{\mathrm{num}}, \Sigma_{\mathrm{container}}, \Sigma_{\mathrm{iter}}, \Sigma_{\mathrm{async}}, \Sigma_{\mathrm{ctx}}, \Sigma_{\mathrm{match}}, \eta),$$

where each Σ summarizes a family of behavior induced by the class and its ancestors. This is the semantic object that later chapters on protocols, repair, and generated contracts should consume.

The generation algorithm should proceed in stages: gather inherited surface facts from the MRO, overlay class-local methods and descriptors, apply metaclass and decorator transformations that are known to affect behavior, and emit a route-annotated surface summary. Special methods matter here because Python dispatches many operations through slots and class-level lookup rather than ordinary instance attribute access. The implementation should record this explicitly rather than pretending that all behavior is reducible to normal method calls.

The theorem burden is surface honesty and route-to-surface coherence. Surface honesty says that when the implementation claims a class supports iteration, context management, arithmetic, or pattern matching, the claim is backed by the corresponding generated surface coordinate. Route-to-surface coherence says that specific call or lookup explanations can point back to the surface summary that justified them. The implementation consequence is that behavioral surfaces should be cached and invalidated as first-class artifacts, especially under monkey patching and hot reload.

Chapter 21

Annotations, decorators, registries, and generated contracts

Modern Python libraries communicate semantics through annotations, decorators, and registry effects as much as through ordinary function bodies. These mechanisms are operational, not documentary. An annotation may require deferred evaluation or import additional symbols; a decorator may wrap or replace the callable it decorates; a registry update may alter downstream dispatch. The implementation consequence is that contract extraction must treat these phenomena as executable semantic routes rather than as comments around already-known code.

The common theme is latent behavior. The system should capture not only what source text says directly, but also what auxiliary mechanisms cause to become true of the resulting object surface. AG enters here through surface transport: decorators and registries change how local clauses descend to public surfaces.

21.1 Annotations as latent behavior

Annotations should be represented by

$$\mathcal{A} = (\text{raw}, \text{mode}, \Gamma_{\text{ann}}, \eta, \delta),$$

where raw is the stored annotation payload, mode records whether the payload is eager, stringized, deferred, or already normalized, Γ_{ann} is the environment needed for evaluation, and δ records any dependencies imported by annotation interpretation. The judgment

$$\text{AnnEval}(a, c) \Downarrow (r, \psi, \phi_{\text{ok}}, \phi_{\text{exc}}, \epsilon, \mu)$$

should distinguish raw-object interpretation, string-literal forward reference, deferred thunk evaluation, `typing`-mediated normalization, and failure.

This matters because annotations are not semantically inert. They may be postponed, they may require imports to resolve, and evaluating them may itself open dynamic paths. The implementation should therefore avoid the common mistake of treating `__annotations__` as if it were already a final theorem object. Instead, annotations should remain latent semantic resources until an admitted interpreter consumes them.

The theorem targets are annotation-route honesty and environment fidelity. Route honesty says that any claim derived from annotations records whether the annotations were read raw, evaluated, or partially resolved. Environment fidelity says that evaluated annotations are tied to

the environment summary that made them meaningful. The implementation consequence is that annotation handling must be staged and epoch-aware, especially when imports, decorators, or hot reload can change the meaning of names used in annotations.

21.2 Generated contracts

Generated contracts are the point where annotations and decorators become directly useful to JuGeo. A generated-contract record should be

$$\mathcal{G} = (\Theta_{\mathrm{pre}}, \Theta_{\mathrm{post}}, \Theta_{\mathrm{exc}}, \Theta_{\mathrm{mut}}, \Pi, \eta),$$

where Π is the provenance map telling which clause came from source text, annotation interpretation, decorator translation, registry policy, or inherited surface rule. The operational judgment is

$$\mathrm{SynthesizeContract}(f, \mathcal{A}, \mathcal{D}) \Downarrow (r, \mathcal{G}, \phi_{\mathrm{ok}}, \phi_{\mathrm{exc}}, \epsilon, \mu),$$

with routes for direct source guarantee, annotation-derived clause, recognized decorator wrapper, class-level contract injection, and residual unknown transformation.

The synthesis algorithm should be staged:

1. collect source-declared guarantees, assumptions, and checks;
2. interpret annotations under the current annotation policy;
3. apply recognized decorator translators in source order, recording whether each wraps, replaces, or augments the target;
4. merge generated clauses into $\mathcal{G}$ with explicit provenance and conflict records;
5. residualize any clause whose source cannot be transported honestly.

This is the point where the thesis should insist that JuGeo-generated contracts remain explainable: a postcondition extracted from a decorator is valid only if the system can name the decorator route and the transformation law it used.

The theorem burden is contract provenance, non-forgery, and merge coherence. Provenance says that every generated clause identifies its source route. Non-forgery says that the implementation does not invent a stronger contract than the admitted translators justify. Merge coherence says that when several generators contribute clauses, the result is either a compatible merged contract or an explicit obstruction rather than silent clause loss. The implementation consequence is that generated contracts must be stored as structured records, not pasted text.

21.3 Registry surfaces

Registries are live semantic objects because later dispatch may depend on them. A generic registry record is

$$\mathcal{R} = (\kappa, \mathrm{keyspace}, \mathrm{entries}, \Theta, \eta, \mu),$$

where κ is the registry kind, keyspace describes the admissible dispatch keys, and Θ records the contract family exported by the registry. The relevant judgments are

$$\mathrm{Register}(R, k, v) \Downarrow (r, \eta^{+}, \phi_{\mathrm{ok}}, \phi_{\mathrm{exc}}, \epsilon, \Delta)$$

and

$$\mathrm{ResolveReg}(R, k) \Downarrow (r, v, \phi_{\mathrm{ok}}, \phi_{\mathrm{exc}}, \epsilon, \mu).$$

The route tags should distinguish direct registration, decorator-mediated registration, subclass hook registration, fallback resolution, MRO-based resolution, and residual dynamic resolution.

This covers single-dispatch families, plugin registries, command tables, event handlers, and many framework-specific surfaces. The point is not to hard-code every library, but to require that any recognized registry export a typed dispatch surface and an epoch. Once that is done, downstream reasoning about call edges, plugin availability, and repair impact becomes honest.

The theorem targets are registration-route honesty, fallback determinacy on the admitted fragment, and registry-epoch coherence. The implementation consequence is that registries belong in the same invalidation and explanation framework as imports and class surfaces. A newly registered handler is not a comment on the program; it is a semantic state change.

21.4 Theorem burden

This chapter should end with a direct theorem slate rather than rhetoric. The key positive targets are:

1. *annotation interpretation honesty*: derived clauses reflect the recorded annotation-evaluation route and environment;

2. *decorator transport soundness*: recognized decorators preserve or transform contracts exactly as their registered translators state;

3. *registry-surface fidelity*: registry-based dispatch explanations correspond to the tracked registry state at the cited epoch;

4. *generated-contract provenance*: every generated clause is traceable to a source route and can be challenged routewise.

The non-theorems matter just as much. The chapter should not claim complete understanding of arbitrary decorators with opaque side effects, global reflection during annotation evaluation, or untracked registry mutation performed through unknown code paths. These are residual routes, not failures of honesty. Indeed that is the implementation lesson: the system becomes trustworthy not by pretending to know every dynamic trick, but by distinguishing recognized transformations from residual ones with precision.

Chapter 22

Protocols, proxies, delegation, and unstable object surfaces

Many Python objects do not present a stable, closed surface determined solely by their class dictionary. They may delegate to wrapped objects, satisfy protocols structurally, expose attributes only through `__getattr__` or `__getattribute__`, or proxy behavior across process or network boundaries. A thesis that wants implementation-guiding semantics must therefore classify surface stability explicitly.

The central design claim is that JuGeo should reason about object surfaces, not just object names. Some surfaces are stable enough to support strong transport theorems; others are unstable and require conservative reopening or residualization. This distinction directly affects repair, equivalence, and public-surface honesty. In AG terms, object behavior is not one chart but a small sheaf of surface descriptions whose overlaps may be stable, delegated, or residual.

22.1 Stable versus unstable surface area

An object surface should be summarized by

$$\mathcal{U}(o) = (\Sigma_{\text{stable}}, \Sigma_{\text{derived}}, \Sigma_{\text{dynamic}}, \upsilon, \eta),$$

where Σ_{stable} collects attributes and behaviors justified by class and instance structure alone, Σ_{derived} collects generated or inherited surface facts, Σ_{dynamic} collects routes mediated by dynamic hooks, and υ records the stability classification of each exported surface element. The access judgment

$$\text{Surface}(o, a) \Downarrow (r, \upsilon, \phi_{\text{ok}}, \phi_{\text{exc}}, \epsilon, \mu)$$

should be understood as operating against $\mathcal{U}(o)$ rather than against an untyped bag of names.

The implementation should classify at least four stability levels: stable declared surface, stable generated surface, unstable delegated surface, and residual dynamic surface. This is not philosophical nuance. It determines when a proof can survive a local edit, when a repair is allowed to assume an attribute exists, and when documentation claims about an API can be discharged without runtime witnesses. Geometrically, these levels say how confidently a local surface section descends along neighboring regions without reopening.

The theorem targets are stability-class honesty and surface monotonicity under admitted updates. Stability-class honesty says that a surface element marked stable really is invariant under the tracked mutation families. Surface monotonicity says that edits outside the support of a stable element do not reopen claims about it. The implementation consequence is that every exported attribute or behavior summary should carry a stability tag.

22.2 Delegation chains

Delegation should be made explicit as a chain rather than collapsed into a final answer. The relevant record is

$$\mathcal{D} = \big[(o_0, a_0, r_0), \ldots, (o_n, a_n, r_n)\big],$$

a finite route chain from the original receiver to the object that finally satisfied the request. The judgment

$$\mathrm{Delegate}(o, a) \Downarrow (\mathcal{D}, v, \phi_{\mathrm{ok}}, \phi_{\mathrm{exc}}, \epsilon, \mu)$$

should distinguish wrapper forwarding, proxy transport, descriptor on delegate, dynamic fallback, explicit adaptation, and failure. A chain rather than a single route is necessary because wrappers and proxies often layer behavior: validation in one object, caching in another, actual effect in a third.

The implementation algorithm should detect delegation loops, bound the admitted chain length for strong claims, and record which links are stable versus dynamic. This makes repair and explanation significantly more honest. If a method call reached a database proxy only after crossing three wrappers and a lazy-binding adapter, the public explanation should say so. In AG terms, the delegation chain is the actual overlap path by which a local section is transported across surface charts.

The theorem burden is delegation-chain faithfulness, bounded-termination on the admitted fragment, and blame localization. Faithfulness says that a reported delegation chain is replayable against the same surface summaries. Bounded-termination says that the algorithm either resolves within the admitted bound or residualizes rather than looping semantically. Blame localization says that if the chain fails, the obstruction names the failing link rather than the original call site only. The implementation consequence is that call-edge explanations must admit chains, not just endpoints.

22.3 Protocol obligations

Protocols should be treated as dependent interface obligations rather than as lists of attribute names. A protocol record should be

$$\mathcal{P} = (M, \Phi_{\mathrm{str}}, \Phi_{\mathrm{sem}}, \rho_{\mathrm{var}}, \eta),$$

where M is the member family, Φ_{str} the structural obligations, Φ_{sem} the behavioral obligations, and ρ_{var} the variance or substitution discipline relevant to transport. The operational judgment is

$$\mathrm{Implements}(o, \mathcal{P}) \Downarrow (r, b, \phi_{\mathrm{ok}}, \phi_{\mathrm{exc}}, \epsilon, \mu),$$

with routes for nominal witness, structural witness, delegated witness, runtime-checkable witness, and residual semantic witness.

This matters because Python protocols are often only partly checkable structurally. Attribute presence is not the same as behavioral satisfaction. A file-like object may expose `read` and `write` yet violate sequencing or error conventions. The thesis should therefore insist that structural protocol satisfaction and semantic protocol satisfaction remain separate coordinates, even when the public API presents them as one notion of compatibility. Geometrically, protocol satisfaction is descent of a surface section into a protocol sheaf, and delegated witnesses may fail to descend globally.

The theorem targets are structural-surface soundness and semantic residual honesty. Structural-surface soundness says that any structural witness to protocol satisfaction is backed by the recorded

surface summary. Semantic residual honesty says that clauses beyond the structural frontier remain open or semantically supported rather than silently promoted. The implementation consequence is that protocol checking should always return a mixed-evidence record, never an undifferentiated bit.

22.4 Why this matters for repair

Repair systems fail most often when they treat unstable surfaces as if they were stable interfaces. A local patch that appears to preserve a method signature may still break a proxy chain, invalidate a registry-based protocol witness, or change the dynamic surface exported through `__getattr__`. The repair judgment should therefore be indexed by surface stability:

$$\mathrm{RepairImpact}(\delta, \Sigma) \Downarrow (\Delta_{\mathrm{safe}}, \Delta_{\mathrm{reopen}}, \Delta_{\mathrm{resid}}, \eta^{+}),$$

where the three frontiers distinguish coordinates that remain safe, coordinates that must reopen, and coordinates that become residual due to newly unstable surface routes.

The AG interpretation is exact. Repair is local only when the overlap graph is honest about unstable surfaces. Delegation and proxying increase overlap in ways that text-local diffs cannot see. That is why this chapter belongs in the core semantics rather than in a later engineering appendix.

The theorem burden is repair-locality conditional on stability classification. If a coordinate depends only on stable surface elements untouched by the patch, it should not reopen. If it depends on unstable delegated or dynamic surface elements, reopening is the honest result. The implementation consequence is that repair planning must consume surface-stability records and delegation chains, not merely AST diffs.

Chapter 23

`exec`, `eval`, monkey patching, hot reload, and live mutation

This chapter is about semantic apertures: operations that can alter the effective meaning of code after the static surface seems settled. Python permits code generation, rebinding of exported names, in-place class mutation, and development-mode reload. These are not reasons to abandon semantics; they are reasons to make epochs, residual routes, and invalidation frontiers explicit.

The central implementation claim is that live mutation should be tracked by epoch-indexed summaries of modules, classes, functions, and registries. Without epochs, the system cannot state when a cached proof, surface summary, or import transport map was obtained, and therefore cannot reopen obligations honestly under live change. In AG terms, these events change the local sections themselves; without epoch tags the system loses track of which cochains even belong to the same cover.

23.1 Semantic apertures in the Python world

Python semantic apertures are operations that widen the local chart through which a coordinate must be understood. `exec`, `eval`, monkey patching, and reload are not uniform dynamic hazards; they are route-indexed mutations of the current section family. The implementation should therefore classify apertures, record their scope, and state which existing local sections may fail descent afterward.

23.2 Epoch-indexed module and object summaries

Every authority-bearing runtime object affected by live mutation should carry an epoch:

$$\eta \in \mathsf{Epoch},$$

and summaries should be indexed accordingly. A module or object fact is never simply "true"; it is true at an epoch and under a support discipline. This is the minimum structure required for honest invalidation, replay, and public reprojection after mutation.

23.3 `exec` and `eval` as bounded or residual events

The relevant event record is

$$\mathcal{E}_{\text{code}} = (\text{kind}, \text{src}, \Gamma_{\text{in}}, \Gamma_{\text{out}}, \eta_{\text{in}}, \eta_{\text{out}}, \Delta_{\text{defs}}, \Delta_{\text{deps}}),$$

where kind $\in \{\texttt{exec}, \texttt{eval}\}$. The operational judgment should be

$$\text{ExecEval}(k, s, G, L) \Downarrow (r, v, \phi_{\text{ok}}, \phi_{\text{exc}}, \epsilon, \mu),$$

where G and L are the global and local mappings and r distinguishes literal-bounded execution, bounded evaluation in explicit environments, definition-producing execution, import-producing execution, reflective environment write, and residual dynamic execution.

The implementation should adopt a staged admission policy. If the source is statically known and the environment mappings are explicit tracked dictionaries, parse and analyze the code, extract reads and writes, and route the resulting effects through the ordinary semantics. If the source or environment is not sufficiently known, residualize the event and attach an invalidation frontier over the touched namespace. The chapter should be explicit that implicit current-frame local mutation through `exec` is not strong theorem territory across Python implementations; it is a residual or bounded event depending on the environment route.

The theorem targets are bounded-event soundness and residual honesty. Bounded-event soundness says that admitted literal or explicit-environment cases are handled by the same semantic machinery as ordinary code. Residual honesty says that unknown dynamic code injection is not hidden under ordinary proofs. The implementation consequence is that `exec` and `eval` should produce event artifacts and epoch bumps, not merely scary warning flags.

23.4 Monkey patching and late rebinding

Monkey patching is semantic mutation of public surfaces after original definition time. The patch judgment should be

$$\text{Patch}(t, a, v) \Downarrow (r, \eta^{+}, \phi_{\text{ok}}, \phi_{\text{exc}}, \epsilon, \Delta_{\text{inv}}),$$

where t is the target object, module, or class, and r distinguishes module rebinding, class attribute replacement, instance patch, descriptor replacement, wrapper insertion, and failure. The important consequence is not just that the target changed, but that dependent summaries may now be stale: bound methods, behavioral surfaces, registry entries, and import transport maps can all require reopening.

The implementation should therefore treat every patch as an epoch event. It should identify the target authority center, bump the corresponding epoch, and invalidate all cached facts whose support intersects the patched surface. A class-method replacement should invalidate generated behavioral surfaces and any method-route certificates that referenced the old attribute. A module-name rebind should invalidate import transport explanations and any registry entries pointing to the old value.

The theorem targets are epoch coherence and patch-impact completeness on the admitted dependency graph. Epoch coherence says that no proof artifact may survive as current after the epoch it depended on has changed. Patch-impact completeness says that every cached artifact whose support intersects the patched surface is marked stale or reopened. The implementation consequence is that monkey patching cannot be modeled as mere assignment; it is a semantic event with system-wide but support-local consequences.

23.5 Hot reload and development-mode semantics

Hot reload is not equivalent to fresh import. In Python, `importlib.reload` typically re-executes module code in an existing module object, while external references to prior objects may persist. The relevant judgment should be

$$\mathrm{Reload}(m, \pi) \Downarrow (r, \mathcal{M}', \phi_{\mathrm{ok}}, \phi_{\mathrm{exc}}, \epsilon, \Delta_{\mathrm{stale}}),$$

where π is the reload policy and r distinguishes in-place re-execution, shadow-module replacement, partial reload failure, stale-reference exposure, and residual custom reload route.

The implementation algorithm should stage reload as follows:

1. identify the module object and current import epoch;
2. re-execute the module body under the chosen policy;
3. compare the new exported surface with the old one;
4. compute the stale-reference frontier consisting of external bindings that still point at pre-reload objects;
5. bump the module epoch and reopen dependent coordinates.

This is where the thesis should say plainly that development-mode semantics differs from release-mode semantics. Reload may preserve identity of the module object while changing the identities of functions, classes, and registries inside it. In AG terms, reload replaces local sections on the same support and must trigger a new descent check rather than pretending the old cocycle still applies.

The theorem burden is reload-route honesty and stale-reference detection on the admitted fragment. The chapter should not claim global semantic equivalence between reloaded and freshly started programs. What it should claim is that when reload is recognized, the resulting module and stale frontier are reported honestly. The implementation consequence is that JuGeo should expose development-mode semantics as a separate route family rather than quietly pretending it is ordinary import.

Chapter 24

Task-local context, cancellation, exception groups, and process boundaries

Python concurrency is not one phenomenon. `contextvars`, task cancellation, `TaskGroup` aggregation, threads, and processes each define different transport laws for state and failure. A judgment geometry that wants to reason about async code and system boundaries must therefore make task-local context, cancellation routes, and process replication explicit.

The implementation consequence is that concurrency coordinates must include more than a function and its arguments. They must also include task context, scheduler-visible suspension points, cancellation state, and boundary-crossing mode. Without that extra structure, explanations about failure, replay, and repair become misleading. This is also where AG becomes especially concrete: concurrent and replicated execution produces several local sections of what users often think is "one" state, and divergence among them is naturally an obstruction to gluing.

24.1 Concurrency in Python is not one phenomenon

Threads, tasks, processes, queues, and grouped exceptions are not different user interfaces to one semantics. They induce different notions of locality, support overlap, and transport. The thesis should therefore refuse a single flattened concurrency model and instead represent each route family separately while keeping them inside one judgment language.

24.2 Task-local context as hidden but structured input

Task-local context should be modeled as a structured input channel rather than as invisible ambient magic. A task record should be

$$\mathcal{T} = (\tau, \Gamma_{\mathrm{ctx}}, \chi_{\mathrm{cancel}}, \sigma_{\mathrm{sched}}, \eta, \Theta),$$

where τ is the task identity, Γ_{ctx} the current `contextvars` snapshot, χ_{cancel} the cancellation state, and σ_{sched} the scheduler-visible status. The relevant judgment is

$$\mathrm{ContextRead}(x, \mathcal{T}) \Downarrow (r, v, \phi_{\mathrm{ok}}, \phi_{\mathrm{exc}}, \epsilon, \eta'),$$

with routes for direct task-local read, inherited context read, task-spawn copied context, thread boundary, and process boundary.

This matters because task-local context often influences behavior without appearing in function signatures. Logging context, request identifiers, locale, tracing scopes, and transaction handles may all flow through `contextvars`. The thesis should therefore insist that public explanations can mention hidden context inputs when they are semantically relevant. Otherwise one risks proving a function correct relative to one context and reporting it as context-free truth.

The theorem targets are context-transport soundness and hidden-input honesty. Context-transport soundness says that when a task is spawned or resumed, the recorded context transfer agrees with the relevant runtime rule on the admitted fragment. Hidden-input honesty says that any proof or explanation depending on task-local state records that dependency. The implementation consequence is that task coordinates must include context summaries.

24.3 Cancellation and exception-group semantics

Cancellation is a routed control event, not a boolean status. In `asyncio`, cancellation is typically injected at an await or suspension boundary, may be caught or transformed, and often coexists with cleanup logic. The task-step judgment should therefore be

$$\mathrm{StepTask}(\mathcal{T}) \Downarrow (r, \mathcal{T}', q, \phi_{\mathrm{ok}}, \phi_{\mathrm{exc}}, \epsilon, \mu),$$

where r distinguishes ordinary progress, suspension, cancellation injection, cancellation suppression, terminal cancellation, ordinary exception, and exception-group emission. Exception groups should be structured as trees

$$\mathcal{G}_{\mathrm{exc}} = (\mathrm{kind}, \mathrm{members}, \pi, \eta),$$

rather than as flattened lists, because `except*` semantics operates on the tree structure.

The implementation should be explicit about staging:

1. step the task until a suspension boundary or terminal event;
2. if cancellation is pending, inject it only at an admitted boundary;
3. run any `finally` or cleanup handlers that the runtime would execute;
4. if multiple child tasks fail under a grouping construct, form an exception-group tree and preserve branch provenance;
5. route `except*` handlers by splitting the tree rather than by flattening it.

This is the only honest basis for reasoning about cancellation-safe cleanup and grouped failures. Geometrically, cancellation changes which local chart of the task-state sheaf is active, and exception groups are trees of local failures that must not be collapsed before projection.

The theorem targets are cancellation-delivery honesty, cleanup preservation, and exception-group projection soundness. Delivery honesty says that a task is reported cancelled only at a boundary where cancellation may actually become visible. Cleanup preservation says that recognized cleanup obligations survive cancellation routing. Projection soundness says that group-handling explanations respect the tree structure of the aggregated failure. The implementation consequence is that cancellation and grouped exceptions require their own data models, not ad hoc flags on generic error records.

24.4 Process boundaries and replicated state

Process boundaries change identity, sharing, and initialization laws. A process record should be

$$\mathcal{P} = (\mathsf{pid}, \sigma_{\text{local}}, \sigma_{\text{shared}}, \kappa_{\text{start}}, \eta, \Theta),$$

where $\kappa_{\text{start}} \in \{\text{fork}, \text{spawn}, \text{forkserver}\}$ records the process-start mode. The central judgment is

$$\text{Spawn}(f, \vec{a}, \pi) \Downarrow (r, \mathcal{P}, \phi_{\text{ok}}, \phi_{\text{exc}}, \epsilon, \mu),$$

with routes for fork inheritance, spawn re-import, forkserver dispatch, IPC call, shared-memory transport, and failure.

The chapter should say plainly that most object identities do not transport across process boundaries. Under `spawn`, modules may be re-imported and global state reinitialized. Under `fork`, memory begins as replicated but diverges unless managed through explicit shared state. Serialization constraints, import side effects, and environment differences all become proof-relevant. The implementation should therefore treat process crossing as a semantic route change, not as just another call edge.

The theorem targets are boundary soundness, serialization honesty, and shared-state route fidelity. Boundary soundness says that process-start claims match the admitted runtime mode. Serialization honesty says that transported values are identified as serialized copies rather than as original identities. Shared-state route fidelity says that any claim of cross-process sharing names the mechanism—queue, pipe, manager proxy, shared memory, or other admitted route. The implementation consequence is that process-aware reasoning needs separate identity and state summaries per process coordinate.

24.5 Replicated-state obstructions

Replicated state is where the AG obstruction language becomes especially natural. If several tasks or processes carry local replicas of what is informally "the same" application state, then divergence is an obstruction to gluing those local sections into one coherent global judgment. A replicated-state obstruction should therefore be recorded as

$$B_{\text{rep}} = (S, \kappa, E, R, \Delta),$$

where S is the family of replica supports, κ identifies the violated consistency law, E is the evidence bundle, R the repair frontier, and Δ the dependent certificates and manifests affected by the divergence.

This is the exact place to say that distributed and concurrent bugs are not outside the theory: they are failures of descent across replicated local sections. Repair may mean replay, resynchronization, stronger transport law, or admission that the present consistency regime is too weak.

Part V

Structural Reasoning and Exact Encodings

Chapter 25

Z3 and the structural frontier

This chapter should fix the solver boundary once and for all. Z3 is not the semantic authority for JuGeo as a whole. It is the authority for recognized structural fragments after normalization, support selection, and routing. The implementation object corresponding to this doctrine should therefore be a solver-family record

$$\mathcal{F}_{\mathrm{z3}} = (\mathrm{name}, \mathrm{adm}, \mathrm{nf}, \mathrm{emit}, \mathrm{lift}, \mathrm{thm}, \mathrm{nonthm}),$$

where adm is the admissibility predicate on obligations, nf the required normal form, emit the encoding function into solver clauses, lift the model-reconstruction procedure, thm the theorem targets claimed for the family, and nonthm the explicit non-theorems that the family refuses to claim. The point of this record is practical. Engineers should be able to ask which family an obligation belongs to, what normalization that family requires, what witness shape it returns, and what the family refuses to prove.

The six encoding chapters should be read as six concrete solver families rather than as six essays about SMT. Family I handles scalar refinements, guards, arithmetic, and path conditions. Family II handles collections, finite maps, heap summaries, alias partitions, and interface abstractions. Family III handles strings, naming laws, and documentation shadows. Family IV handles structured sequences, heap slices, and support-aware mutation. Family V handles tensor extents, affine legality, and disciplined quantified fragments. Family VI handles partial functions, exception-valued semantics, algebraic surfaces, and model reconstruction. The implementation should expose these families directly, because routing, cache keys, regression suites, and explanations all depend on knowing which abstraction boundary is in force.

The routing algorithm for a structurally eligible obligation o should be:

1. normalize o into a typed structural core together with explicit support and imported assumptions;
2. test admissibility of o against the registered families in a fixed priority order or by a declared classifier;
3. if exactly one family admits o, emit that family's clause bundle and metadata;
4. if several families admit o, choose by the declared routing policy and record the rejected alternatives;
5. if no family admits o, residualize rather than forcing an inexact encoding;
6. on solver success, emit a discharge artifact tagged by family and support;

7. on solver failure, run model lifting and emit a structural obstruction rather than a generic failure bit.

The theorem targets for this chapter are family-routing soundness, emission soundness, and witness-lifting faithfulness. Family-routing soundness says that any obligation routed to a family lies inside that family's declared admissible fragment. Emission soundness says that the emitted clause bundle preserves the intended structural meaning of the normalized obligation. Witness-lifting faithfulness says that any lifted countermodel really falsifies the clause it claims to falsify when replayed through the same normalization and family boundary. The non-theorems should be equally explicit. This chapter does not claim global completeness for arbitrary Python semantics, does not claim that every semantic guarantee has a useful structural shadow, and does not permit solver success to stand in for semantic understanding outside a declared family.

25.1 Z3 should own the structural frontier, not the whole semantic world

Z3 should be given authority only over a declared structural frontier. The frontier is the class of obligations that admit a family seal, a canonical normal form, an exact emitter, and a replay-valid witness lifter. A useful admission predicate is

$$\mathrm{Adm}_{z3}(o, f) \iff \mathrm{NF}_f(o)\downarrow \wedge \mathrm{Emit}_f(o) \text{ is exact} \wedge \mathrm{Lift}_f \text{ is defined},$$

where f is one of the registered structural families. This definition should be compiled into code, not left as policy prose. If an obligation fails any conjunct, JuGeo should residualize it or split it rather than silently pretending it lives in a total SMT world.

This distinction matters because a vanishing solver query is not the same as a globally settled judgment. A solver proof discharges only the structural shadow that the family admits. Global promotion requires three additional conditions: the obligation's support must be recorded, the family theorem target must be cited, and any semantic or treaty residuals on neighboring overlaps must already be settled or explicitly preserved. The POPL-style soundness story should therefore be imported here in restricted form: structural vanishing can certify safety only relative to the cover, the family, and the expressivity of the chosen shadow. JuGeo should say this explicitly so that users can tell the difference between exact local proof and full semantic closure.

25.2 The code should make solver-lifted obligations explicit

Every solver-routed obligation should become a durable runtime record rather than an anonymous backend call. A useful record is

$$\mathfrak{s} = (o, f, N_f(o), \Gamma_{\mathrm{guard}}, \Gamma_{\mathrm{import}}, \mathcal{R}, \beta_f, \rho_f, \kappa_f),$$

where o is the source obligation, f the selected family, $N_f(o)$ the canonical form, Γ_{guard} the guard bundle, Γ_{import} the imported assumptions, $\mathcal{R}$ the support region, β_f the family seal used for replay, ρ_f the expected result schema, and κ_f the expected failure schema. The emitted clause bundle should be a compiled view of this object, never its authority center.

The implementation should then make the lifecycle explicit: construct the record, emit clauses, run the solver, lift either proof or countermodel, attach the resulting evidence to the common judgment state, and archive the replay conditions. This is the only way to support later claims

about why a discharge remained valid after change, why it had to reopen, or why a family switch changed the answer. Ordinary SMT-based tools often lose this information by treating each query as disposable. JuGeo should not.

25.3 Countermodels should become first-class semantic witnesses

A structural countermodel should be treated as a witness object in the common judgment geometry, not as debugger output. The witness should record at least the family, the normalized clause identifiers violated, the support region, the overlap or boundary on which the failure becomes visible, and a replay-valid lifted artifact in source-local terms. A useful witness record is

$$\mathfrak{r}_{\mathrm{struct}} = (f, \chi, \mathcal{R}, \omega, w_{\mathrm{lift}}, \pi_{\mathrm{replay}}).$$

Such a record lets JuGeo compare failures across runs, cluster them by obstruction family, and decide whether a contradiction is local, imported, or the visible face of a broader incompatibility.

This is also the right place to import one of the stronger POPL-style consequences. On admitted binary exact fragments, the system may compute an overlap-level obstruction matrix from the family witnesses and use its rank as an estimate, and in specially controlled cases a proof-backed count, of the minimum number of independent repairs still required. Traditional verifiers usually stop at "counterexample found." JuGeo should be able to continue to "counterexample found here, on this overlap, under this cover, and the present obstruction basis suggests at least this many independent repairs remain."

Chapter 26

Exact Z3 encodings I: base refinements, guards, arithmetic, and path conditions

This chapter should make the scalar structural frontier concrete while keeping it subordinate to the broader AG+DTT+AI story. The role of exact Z3 encodings is not to replace geometry, but to provide exact local sections on structural charts where the coefficient theory is decidable. Solver discharge lives on local support regions; its outputs become part of global descent only after they are attached to their support, guards, and clause provenance. The algebraic-geometric language remains active even here: these are exact local computations that later participate in gluing, witness transport, and obstruction formation.

26.1 The encoding layer should begin from a typed structural core

Every exact encoding family should consume the same typed structural core. The front end, normalization layer, and obligation splitter should produce a record

$$\Sigma = (\mathcal{V}, \mathcal{S}, \mathcal{P}, \mathcal{A}, \mathcal{U}, \mathcal{R}),$$

where $\mathcal{V}$ is the family of symbolic names, $\mathcal{S}$ assigns solver sorts to those names, $\mathcal{P}$ is the active path-condition set, $\mathcal{A}$ is the imported assumption set, $\mathcal{U}$ is the set of admitted uninterpreted summary symbols, and $\mathcal{R}$ is the support region from which the obligation was harvested. No family in Chapters I–VI should read raw source syntax as its authority center. Raw syntax may guide normalization, but the emitter should see only this core plus the family-specific payload.

The algorithmic consequence is immediate. For every candidate structural obligation, the system should:

1. refresh local names into a stable SSA-like symbolic namespace;
2. infer or validate the sort assignment in $\mathcal{S}$;
3. separate path conditions from imported assumptions so that later explanation can cite them differently;
4. register any interface summaries, heap summaries, or route tags in $\mathcal{U}$ rather than inlining them opportunistically;

5. reject the obligation if the resulting Σ is ill-sorted, support-incoherent, or family-inadmissible.

This is the right place to be strict, because every later theorem about encoding soundness depends on the existence of a clean pre-encoding object.

The theorem targets are typing preservation under normalization, support preservation under symbolic renaming, and clause-origin traceability from emitted formulas back to $\mathcal{P}$, $\mathcal{A}$, and $\mathcal{U}$. The corresponding non-theorems should also be written down. The system does not claim that every Python expression has a faithful structural lowering into Σ. It does not claim that unresolved sort ambiguity may be guessed away. And it does not claim that a structurally typed core already decides whether an obligation is semantically important. It only claims that without such a core, exact encodings are not exact enough to trust.

26.2 Scalar refinements

The scalar layer should make one severe promise: whenever a clause is routed here, its structural meaning is represented without approximation inside a declared scalar family, and whenever that is impossible the clause is residualized rather than weakened. The authority center for this layer should therefore be a normalized family record

$$\mathcal{F}_{\text{scalar}} = \big(b,\, x,\, \Gamma_{\text{guard}},\, \Phi_{=},\, \Phi_{\leq},\, \Phi_{\text{lin}},\, \Phi_{\text{enum}},\, \Phi_{\text{bool}},\, \mathcal{R},\, \chi\big),$$

where b is a base sort in $\{\mathsf{Bool}, \mathsf{Int}, \mathsf{Real}, \mathsf{Enum}(E)\}$, x is the distinguished symbolic binder, Γ_{guard} is the active path-sensitive guard set, $\Phi_{=}$ is the family of equalities and disequalities, $\Phi_{\leq}$ the family of order atoms, Φ_{lin} the family of linear arithmetic constraints, Φ_{enum} finite-choice membership constraints, Φ_{bool} exact boolean implications or equivalences, $\mathcal{R}$ the support region from which the family was harvested, and χ the clause-origin map back to source obligations and interface imports.

The normalization object for scalar clauses should be concrete enough to compare, cache, and replay. A workable normal form is

$$N_{\text{scalar}}(\lambda) = \big(b,\, x,\, I_x,\, C_x,\, E_x,\, B_x,\, \Gamma_{\text{guard}},\, \mathcal{R},\, \chi\big),$$

where I_x is a finite interval presentation for order constraints, C_x is a finite set of linear combinations of the form

$$a_1 y_1 + \cdots + a_n y_n + a_0 \;\bowtie\; 0, \qquad a_i \in \mathbb{Z}, \quad \bowtie \in \{=, \neq, \leq, <, \geq, >\},$$

E_x is a normalized equality or disequality family, and B_x is the exact boolean skeleton. If two obligations differ only by binder names, conjunct order, or syntactic sugar such as chained comparisons, they should normalize to the same N_{scalar}.

Construction 26.1 (Scalar emission algorithm)**.** *Given typed core*

$$\Sigma = (\mathcal{V}, \mathcal{S}, \mathcal{P}, \mathcal{A}, \mathcal{U}, \mathcal{R}),$$

and a scalar clause family $N_{\text{scalar}}(\lambda)$, the emitter should proceed as follows.

1. *Alpha-refresh binders in $\mathcal{V}$ into a stable symbolic namespace indexed by coordinate, local binder role, and dominance position.*

2. *Validate that every atom in C_x is linear over the admitted scalar sorts; if a multiplication contains two symbolic non-constant factors, or if the obligation depends on floating semantics rather than exact* Real *semantics, emit a residual structural obligation instead of a clause.*

3. *Compile I_x into bound constraints, E_x into equalities or disequalities, C_x into Presburger atoms, E_x^{enum} into finite disjunctions or datatype equalities, and B_x into exact propositional structure.*

4. *Attach each emitted clause to its clause-origin in χ and to its imported assumptions in $\mathcal{A}$ so that later unsat-core and countermodel explanations can distinguish local code facts from imported interface facts.*

5. *Produce the clause bundle*

$$\mathcal{C}_{\mathrm{scalar}} = (\Gamma_{\mathrm{guard}}, \Gamma_{\mathrm{body}}, \Gamma_{\mathrm{import}}, \chi, \mathcal{R}),$$

where Γ_{body} contains only exact scalar-family clauses.

The key implementation discipline is family rejection, not family optimism. Mixed integer-string claims, nonlinear shape arithmetic, floating-point bit-precise obligations, and semantically interpreted predicates such as "is approximately sorted" do not belong here merely because they contain numbers. This chapter should authorize only exact scalar shadows. Everything else must remain an open obligation, an imported summary, or a different family.

26.3 Branching, joins, and path-sensitive obligations

Branching should be treated as first-class structure in the scalar layer rather than as an afterthought added by explanation code. The normalized branch object should be

$$\mathcal{B} = \big(\pi,\, \Sigma_\pi,\, \Gamma_\pi,\, \Delta_\pi,\, \mathcal{R}_\pi,\, \eta_\pi\big),$$

where π is the branch path formula, Σ_π the branch-local typed core, Γ_π the branch-local clause family, Δ_π the branch-local obligation deltas, $\mathcal{R}_\pi$ the support slice active on that branch, and η_π the replay stamp recording which earlier branch summaries this branch depends on.

The join operator should not collapse branches into a single fact set unless the collapse itself is exact. A concrete join normal form is

$$\mathrm{Join}(\mathcal{B}_1, \mathcal{B}_2) = \big(\pi_1, \pi_2,\, \Gamma_{\mathrm{common}},\, \Gamma_{\mathrm{ite}},\, \Gamma_{\mathrm{split}},\, \mathcal{R}_{\bowtie}\big),$$

where Γ_{common} contains atoms provable on both branches, Γ_{ite} contains exact if-then-else value definitions of the form

$$x^{\bowtie} = \mathrm{ite}(\pi_1, x_1, x_2),$$

and Γ_{split} contains obligations that must remain branch-indexed because no exact merged scalar form exists. This distinction matters because many failures of path sensitivity are really failures to distinguish branch-merge equality from branch-indexed coexistence.

Construction 26.2 (Join discipline for scalar obligations)**.** *Let two branch summaries share a post-dominator and a common target variable set V. For each $v \in V$:*

1. *If both branches assign the same normalized scalar expression to v, place that equality in Γ_{common}.*

2. *If both branches assign well-sorted scalar expressions of the same base sort to v, emit an exact join variable*

$$v^{\bowtie} = \mathrm{ite}(\pi_1, e_1, e_2).$$

3. *If branch outputs differ in sort, support provenance, or admitted family, do not invent a merged value; instead emit branch-indexed obligations*

$$\pi_1 \Rightarrow \lambda_1, \qquad \pi_2 \Rightarrow \lambda_2$$

and record the failure to merge as a join residual.

4. *If one branch ends in exception, return, contradiction, or imported uninterpreted summary, represent that control distinction explicitly in the enclosing obligation rather than coercing the branch into an ordinary value.*

Path-sensitive obligations should be emitted with their actual guards, not with globally hoisted approximations. If a postcondition only holds on π, then the emitted clause should be

$$\Gamma_{\text{guard}} \models \pi \Rightarrow \varphi$$

rather than the stronger but unjustified φ. This is also where support matters: the support of the obligation is the support of π together with the support of φ, not merely the text region of the postdominating block.

Proposition 26.3 (Guard exactness at joins)**.** *If branch-local clause bundles are exact scalar encodings relative to their branch guards, and if the join constructor emits only* Γ_{common}, Γ_{ite}*, and branch-indexed residuals as above, then every discharged joined scalar obligation is entailed by the original branch semantics under the same replay stamp. In particular, no discharge may rely on an unrecorded strengthening from branch-indexed truth to global truth.*

26.4 Preconditions, postconditions, and interface clauses

The scalar layer should treat callable boundaries as clause packs with exact structural interfaces. For a function, method, or exported callable surface f, the boundary record should be

$$\mathcal{I}_f = \big(\vec{a},\, r,\, \Gamma_{\text{pre}},\, \Gamma_{\text{post}},\, \Gamma_{\text{frame}},\, \Gamma_{\text{iface}},\, \mathcal{R}_f,\, \chi_f\big),$$

where $\vec{a}$ are symbolic inputs, r the symbolic result, Γ_{pre} the exact structural precondition family, Γ_{post} the exact structural postcondition family, Γ_{frame} the frame or non-mutation clauses admitted at this layer, Γ_{iface} imported interface laws from enclosing packs or protocols, $\mathcal{R}_f$ the support region for the boundary, and χ_f the source map into guarantees, assumptions, imported laws, and replayed summaries.

A precondition should be emitted as an obligation on callers, not silently assumed by the callee unless the coordinate explicitly marks a local proof context. A postcondition should be emitted under primed result variables and under exactly the precondition and path hypotheses actually available. Interface clauses should be split into three families:

1. *local structural clauses*, harvested from the current callable body and its declared surface syntax;

2. *imported boundary clauses*, replayed from already stabilized upstream interfaces;

3. *summary clauses*, introduced as constrained uninterpreted relations when the callee body is not in scope or not routed to the scalar family.

Construction 26.4 (Boundary emission)**.** *For a call*

$$y := f(x_1, \ldots, x_n)$$

under guard family Γ_{guard}*, the boundary emitter should:*

1. *instantiate* $\mathcal{I}_f$ *with fresh symbolic arguments* $\vec{a}^*$ *and result* r^**;*
2. *emit caller-side obligations*
$$\Gamma_{\text{guard}} \Rightarrow \Gamma_{\text{pre}}[\vec{a}^*/\vec{x}];$$
3. *if* Γ_{post} *is available as an exact scalar family, substitute* r^* *into the caller state and emit the postcondition under the same guard;*
4. *otherwise introduce a constrained summary symbol*
$$S_f(\vec{x}, y)$$
together with any exact scalar interface clauses declared in Γ_{iface} *or* Γ_{frame}*;*
5. *attach every emitted boundary clause to* χ_f *so that later countermodels can distinguish failure of a caller obligation from failure of an imported interface law.*

This chapter should be explicit about what counts as an interface clause at the scalar layer. Statements such as "returns a sorted list" or "preserves semantic intent" do not belong here unless a declared bridge theorem projects them into exact scalar shadows. By contrast, constraints such as non-negativity of a length result, monotonicity of a numeric rank, or exact relation between a flag and an integer status code do belong here if normalization can expose them without loss.

26.5 Exact failure artifacts

A structural chapter is not complete until its failure language is as exact as its success language. The scalar layer should therefore return a first-class failure artifact

$$\mathcal{X}_{\text{scalar}} = \big(k,\, M,\, U,\, \Gamma_{\text{active}},\, \Pi,\, \mathcal{R},\, \chi,\, \rho\big),$$

where $k \in \{\mathsf{sat\_countermodel}, \mathsf{unsat\_core}, \mathsf{unknown}, \mathsf{inadmissible}\}$ is the artifact kind, M is a satisfying model when one exists, U an unsat core when one exists, Γ_{active} the exact active clause bundle, Π the replayed path-condition bundle, $\mathcal{R}$ the support region, χ the clause-origin map, and ρ the witness-lifting recipe.

The witness-lifting recipe should be algorithmic, not rhetorical. For a satisfying model that falsifies a wanted implication, the lifter should:

1. read off symbolic scalar assignments from M for all names appearing in the active clause bundle;
2. project those assignments back through alpha-normalization and branch renaming into source-local symbolic names;
3. attach the minimal guard slice from Π needed to make the witness relevant;

4. emit a clausewise explanation object

$$\mathrm{Explain}_{\mathrm{scalar}}(\mathcal{X}_{\mathrm{scalar}}) = \{(\lambda_i, \mathrm{status}_i, \chi_i)\}_i,$$

where status_i is one of used, violated, guarded_out, or imported.

For unsatisfiable bundles, the artifact should expose not merely that the bundle is inconsistent, but which normalized clauses in χ participate in the contradiction and whether the contradiction is local, imported, or path-generated. This matters because an unsat core over a call boundary may be either a caller bug, an impossible branch, or an inconsistent imported interface summary. The artifact shape must preserve that distinction.

Remark 26.5 (Failure artifacts are semantic witnesses)**.** *The point of $\mathcal{X}_{\mathrm{scalar}}$ is not debugging convenience alone. It is the exact point where solver output becomes a reusable semantic witness in the larger judgment geometry. A scalar countermodel may reopen a guarantee, challenge an interface treaty, or seed a repair frontier. An inadmissibility artifact may reroute an obligation to a different family without pretending that the scalar family learned anything. The artifact therefore belongs in the authority layer, not only in logs.*

26.6 Theorems for the scalar encoding layer

The theorem targets for the scalar layer should be written as implementation obligations over the concrete records above.

Proposition 26.6 (Scalar normalization determinism)**.** *If two scalar obligations differ only by alpha-renaming, syntactic sugar for guards or comparisons, and permutation of conjuncts or finite-choice elements, then they normalize to the same N_{scalar} and therefore produce the same cache key, routing decision, and clause-origin partition up to stable identifier renaming.*

Proposition 26.7 (Emission soundness for scalar families)**.** *Let λ be a structural obligation admitted to the scalar family. If the scalar emitter produces clause bundle $\mathcal{C}_{\mathrm{scalar}}$ from $N_{\mathrm{scalar}}(\lambda)$, then every model of $\mathcal{C}_{\mathrm{scalar}}$ corresponds to a satisfying assignment of the normalized scalar content of λ under the same guard bundle and imported assumptions, and every lifted countermodel falsifies the claimed scalar clause when replayed through the same normalization pipeline.*

Proposition 26.8 (Boundary conservativity)**.** *If a caller uses only exact preconditions, postconditions, frame clauses, and imported summaries from $\mathcal{I}_f$, then any scalar discharge at the caller remains valid under replay so long as the replay stamps of those boundary components are unchanged. In particular, cache reuse across unchanged boundaries is sound only through the boundary record, not through incidental textual sameness.*

The non-theorems should be equally explicit. This layer does not claim completeness for Python numeric behavior, because floating-point bit-precise semantics, reflection-heavy value flow, and library-defined arithmetic escape its exact fragment. It does not claim that nonlinear arithmetic is solved by clever normalization. It does not claim that every public guarantee admits a useful scalar shadow. And it does not allow a scalar countermodel to settle a semantic dispute whose bridge theorem has not been declared.

Chapter 27

Exact Z3 encodings II: collections, finite maps, heap summaries, alias partitions, and interface abstractions

This chapter should govern the exact fragment whose natural coordinates are keyed neighborhoods, root stars, and call-boundary overlaps. A collection or heap summary is not exact as an undifferentiated Python object. It is exact only as a family of local sections over a declared finite support cover together with overlap laws strong enough to support descent. The AG claim is therefore operational: the encoder chooses an atlas, the solver checks compatibility on intersections and admitted higher overlaps, and witness lifting glues local assignments back into a support-local countermodel only when the associated Čech data is exact. Family II should therefore expose authority centers for finite-support collection atlases, heap-summary atlases, alias hypercovers, and interface-summary sheaves rather than hiding them behind ad hoc helper functions.

27.1 Collection encodings should be family-specific

Collections should be routed by their observable geometry, not by the fact that they all happen to be "iterable" in Python. The exact question is which local observations are semantically relevant on the current support region. For a normalized collection obligation at coordinate c, the implementation should build a family record

$$\mathfrak{A}_{\mathrm{coll}}(c) = \big(\mathrm{kind},\, D,\, \mathcal{U}_{\mathrm{coll}},\, \mathrm{pres},\, \mathrm{mult},\, \mathrm{val},\, \Gamma_{\mathrm{shape}},\, \Gamma_{\mathrm{obs}},\, \mathcal{R},\, \chi\big),$$

where $\mathrm{kind} \in \{\mathsf{set}, \mathsf{bag}, \mathsf{fmap}\}$ is the chosen family, D is the declared finite support domain, $\mathcal{U}_{\mathrm{coll}}$ is the observation cover, pres the presence predicate on D, mult a multiplicity map when the family is bag-like, val a value selector when the family is map-like, Γ_{shape} the local shape facts, Γ_{obs} the actually requested observations, and $\mathcal{R}, \chi$ support and provenance.

A useful base cover is

$$\mathcal{U}_{\mathrm{coll}} = \{U_{\mathrm{dom}}, U_{\mathrm{pres}}, U_{\mathrm{val}}, U_{\mathrm{card}}\}.$$

A local section on U_{dom} names the admissible key or index support D. A local section on U_{pres} assigns membership or multiplicity facts for elements of D. A local section on U_{val} assigns values for present keys. A local section on U_{card} carries only those aggregate laws that are exact over the declared support, such as finite cardinality or finite support sums. The overlap laws are the actual implementation discipline:

1. on $U_{\mathrm{pres}} \cap U_{\mathrm{val}}$, value clauses may only apply where presence holds;
2. on $U_{\mathrm{dom}} \cap U_{\mathrm{card}}$, aggregate claims may mention only elements of D;
3. on $U_{\mathrm{dom}} \cap U_{\mathrm{pres}}$, extensionality is finite-support extensionality, never global extensionality over the ambient Python universe.

The point is that exactness lives on the chosen cover. If the needed observation is not admitted on that cover, the obligation is residual, not approximately encoded.

The family specializations should be explicit:

$$\mathfrak{A}_{\mathrm{set}} = \big(K,\, D,\, \mathrm{mem},\, \Gamma_{\mathrm{card}},\, \Gamma_{\mathrm{ext}},\, \mathcal{R},\, \chi\big),$$

$$\mathfrak{A}_{\mathrm{bag}} = \big(K,\, D,\, \mu,\, \Gamma_{\mathrm{count}},\, \Gamma_{\mathrm{sum}},\, \mathcal{R},\, \chi\big),$$

$$\mathfrak{A}_{\mathrm{fmap}} = \big(K,\, V,\, D,\, \mathrm{pres},\, \mathrm{val},\, \Gamma_{\mathrm{req}},\, \Gamma_{\mathrm{opt}},\, \Gamma_{\mathrm{ext}},\, \mathcal{R},\, \chi\big).$$

The encoder should route to $\mathfrak{A}_{\mathrm{set}}$ only when multiplicity and keyed values are semantically irrelevant, to $\mathfrak{A}_{\mathrm{bag}}$ only when finite multiplicity is essential, and to $\mathfrak{A}_{\mathrm{fmap}}$ only when the obligation genuinely depends on finite-domain keyed lookup. This is not taxonomic neatness. It determines which clauses are legal, which witnesses can be lifted, and which theorem schemas are even true.

Construction 27.1 (Collection-family routing)**.** *For a normalized collection obligation, the routing pass should:*

1. *compute the observation signature* $\mathrm{Obs}(o)$ *consisting of membership, multiplicity, lookup, required-key, optional-key, and finite-cardinality queries actually used;*
2. *infer the minimal declared support domain D from syntax, imported summaries, and explicit interface packs;*
3. *choose the smallest family whose atlas contains all charts required by* $\mathrm{Obs}(o)$*;*
4. *emit only those finite-support extensionality clauses justified by D and the chosen family;*
5. *residualize any obligation whose truth would require undeclared iteration order, undeclared infinite support, or whole-object extensionality.*

In AG terms, the error to avoid is collapsing all collections to one ambient array-like object and thereby destroying the overlap structure that makes descent honest. A missing required key, an inconsistent multiplicity count, and an illegal aggregate are not the same obstruction. They live on different charts, and the implementation should preserve that fact.

Proposition 27.2 (Finite-support descent for collection atlases)**.** *Let $\mathfrak{A}_{\mathrm{coll}}$ be an admitted collection atlas over declared support D. If the emitted local clauses are satisfiable and their restrictions agree on every pairwise overlap in $\mathcal{U}_{\mathrm{coll}}$, together with any admitted higher overlaps used for nested keyed values, then the lifter reconstructs a collection witness faithful on D. The witness is unique up to unconstrained behavior outside D.*

27.2 Heap summaries and object identity

Heap reasoning in this family should be support-local from the beginning. Exactness does not mean "the whole heap has been encoded." It means "the tracked root stars and their overlaps have been encoded with explicit identity and epoch discipline." The summary object should therefore be

$$\mathfrak{H}_S = \big(O,\ \{\rho_r\}_{r\in\mathrm{Roots}},\ \{\sigma_f^e\}_{f\in\mathrm{Fields},\, e\in\mathrm{Epochs}},\ \mathrm{Reach},\ \Gamma_{\mathrm{shape}},\ \Gamma_{\mathrm{frame}},\ \mathcal{P}_{\mathrm{alias}},\ \mathcal{U}_{\mathrm{heap}},\ \mathcal{R},\ \chi\big),$$

where O is the object-identity sort, ρ_r maps each tracked root r to an object token, σ_f^e is the field selector for field f at epoch e, Reach is the admitted finite reachability skeleton, Γ_{shape} are shape and type-like summary laws, Γ_{frame} are non-touch laws across epochs, $\mathcal{P}_{\mathrm{alias}}$ is the alias object imported from the next section, and $\mathcal{U}_{\mathrm{heap}}$ is the root-star cover.

The natural cover is

$$\mathcal{U}_{\mathrm{heap}} = \{U_r \mid r \in \mathrm{Roots}\},$$

where a local section on U_r records the object token at root r, the tracked reachable tokens beneath it, and the values of tracked fields on those tokens. The overlap $U_r \cap U_{r'}$ is semantically nonempty only when the alias object permits shared identity. On that overlap, descent requires agreement on object tokens, field selectors, and epoch transition laws for the shared slice. This is the exact place where stale summaries appear: a caller-local section and an imported callee summary both exist, but their restrictions disagree on the overlap.

The implementation should make object identity first-class. If identity is flattened away into only field equalities, the system loses the ability to distinguish true equality of objects from accidental equality of visible fields, and it loses the geometric distinction between a disagreement on a field chart and a disagreement on an identity overlap. A stale summary, an alias collapse, and a plain field mismatch must be different witness kinds.

Construction 27.3 (Epoch-indexed heap emission)**.** *Given a tracked heap summary $\mathfrak{H}_S$, the emitter should:*

1. *create one object sort O per summary world, not one per field;*
2. *create root maps ρ_r and tracked selectors σ_f^e only for declared roots, fields, and epochs;*
3. *emit reachability and shape clauses only for the admitted finite skeleton* Reach*;*
4. *emit frame clauses only on locations outside the declared footprint but inside the tracked stars;*
5. *record every overlap clause with the corresponding root-star pair so that a later witness can say which local summaries failed to glue.*

This chapter should also say plainly that a heap summary is a sheaf of observable heap facts, not a hidden concrete heap. Local sections contain only what the support region authorizes. What matters is whether those local sections glue on overlaps. When they do not, the resulting Čech 1-cocycle is the implementation-facing obstruction: it tells us whether the problem is a missing frame law, a stale imported summary, or an alias mistake.

Proposition 27.4 (Heap-summary faithfulness on tracked stars)**.** *If a model satisfies the emitted clauses for $\mathfrak{H}_S$ and the overlap laws of $\mathcal{U}_{\mathrm{heap}}$, then the reconstructed heap witness is faithful on the tracked roots, tracked fields, and tracked epochs of $\mathfrak{H}_S$. No claim is made about untracked objects or fields.*

27.3 Aliasing obligations

Alias reasoning should not be treated as a bag of pairwise equalities. Pairwise data alone is often not enough to decide whether local identity claims actually glue into a coherent global quotient. The implementation should therefore represent alias state by a finite path cover together with admitted higher simplices:

$$\mathcal{P}_{\mathrm{alias}} = \big(P,\, \approx,\, \#,\, \Omega,\, \mathcal{H}_{\bullet}^{\mathrm{alias}},\, \Gamma_{\mathrm{eq}},\, \Gamma_{\mathrm{neq}},\, \mathcal{R},\, \chi\big),$$

where P is the finite family of normalized access paths, $\approx$ the definite-alias relation, $\#$ the definite non-alias relation, Ω the unresolved may-alias pairs, $\mathcal{H}_{\bullet}^{\mathrm{alias}}$ the alias hypercover carrying triple and higher coherence simplices, and $\Gamma_{\mathrm{eq}}, \Gamma_{\mathrm{neq}}$ the emitted equality and disequality clauses.

The cover here is indexed by tracked paths:

$$\mathcal{U}_{\mathrm{alias}} = \{U_p \mid p \in P\}.$$

A local section on U_p assigns an object token to p. A pairwise overlap $U_p \cap U_q$ asks whether those two sections can agree. But a coherent alias partition needs more than pairwise satisfiability. If p agrees with q and q agrees with r, the triple simplex $U_p \cap U_q \cap U_r$ must witness transitivity. Likewise, field-sensitive aliasing may require higher simplices that combine root equality with selector compatibility. This is exactly where hypercovers stop being ornamental language. Without them the implementation cannot distinguish a locally consistent pair graph from a globally incoherent quotient candidate.

Algorithmically, the alias pass should:

1. normalize access paths into a stable root-selector form;
2. generate candidate alias pairs only from co-support, shared summaries, or explicit mutation interaction, not from an ambient whole-program universe;
3. emit equality and disequality clauses over object tokens for definite cases;
4. construct triple and higher coherence clauses only for admitted simplices in $\mathcal{H}_{\bullet}^{\mathrm{alias}}$;
5. leave genuinely open-world may-alias questions in Ω as residuals unless a finite case split has been explicitly authorized.

This section should also be blunt about obstruction language. A failed pairwise equality is not the only alias failure. There are also higher alias obstructions: pairwise identifications may each look satisfiable while no global partition exists that respects transitivity and imported frame laws. Such a failure should be reported as a nontrivial alias cocycle on the chosen hypercover, not hidden as "solver instability."

Proposition 27.5 (Alias hypercover coherence)**.** *If the emitted alias clauses for $\mathcal{P}_{\mathrm{alias}}$ are satisfiable on every admitted simplex of $\mathcal{H}_{\bullet}^{\mathrm{alias}}$, then the induced quotient on tracked paths is a coherent partial alias partition on the declared support. Replay across revisions is sound only while the normalized path family and the hypercover seal remain unchanged.*

27.4 Interface summaries as uninterpreted but constrained relations

Interface summaries should be treated as imported sheaves of boundary facts, not as prose and not as bodies silently inlined at call sites. The core summary object for a callable boundary should be

$$\mathcal{I}_f^{\sharp} = \big(R_f,\, \vec{x},\, \vec{y},\, h_{\mathrm{in}},\, h_{\mathrm{out}},\, \Gamma_{\mathrm{pre}},\, \Gamma_{\mathrm{post}},\, \Gamma_{\mathrm{frame}},\, \Gamma_{\mathrm{alias}},\, \mathcal{U}_f,\, \beta_f,\, \chi\big),$$

where R_f is an uninterpreted relation symbol naming the summary graph, $\vec{x}$ and $\vec{y}$ are argument and result packs, $h_{\mathrm{in}}, h_{\mathrm{out}}$ are abstract pre- and post-state heaps or summary worlds, Γ_{pre} are admissibility clauses, Γ_{post} are postcondition clauses, Γ_{frame} are frame laws, Γ_{alias} are imported alias assumptions, $\mathcal{U}_f$ is the interface cover, and β_f is the replay seal for this summary package.

The interface cover should separate at least the argument chart, the result chart, and the heap-effect chart:

$$\mathcal{U}_f = \{U_{\mathrm{arg}}, U_{\mathrm{res}}, U_{\mathrm{heap}}\}.$$

A local section on U_{arg} provides argument-side structural facts. A local section on U_{res} provides result-side structural facts. A local section on U_{heap} provides permitted heap and alias movement. The descent problem at a call site is whether the caller-local section and the imported summary section glue on the overlaps determined by actual arguments and tracked heap support. This is the precise implementation meaning of "using a contract." One is not merely asserting a formula; one is importing descent data on an overlap between two charts.

Interface abstractions for protocols or abstract base shapes should be built the same way. A protocol object should package a finite family of member summaries $\{R_m\}_{m \in M_I}$ together with a shared shape map over required members, required fields, and allowed heap effects. This makes protocol checking finite-support exact: every admitted member law is tied to a named chart, and every failure witness can say which member chart or overlap law failed.

The algorithmic consequences should be explicit:

1. summary compilation creates uninterpreted relation symbols plus a declared finite law bundle; it never silently imports implementation bodies;
2. call lowering instantiates only the declared summary charts needed by the current obligation;
3. cache keys and replay eligibility depend on β_f, not on accidental textual sameness of the callee;
4. conflicting imported summaries on the same overlap create treaty obligations, not opportunistic conjunction.

Proposition 27.6 (Interface-summary conservativity)**.** *If a discharge uses only the clauses exported by $\mathcal{I}_f^\sharp$ and the summary seal β_f is unchanged, replay of that discharge is sound. If any clause in Γ_{pre}, Γ_{post}, Γ_{frame}, or Γ_{alias} changes under the same interface name, every dependent discharge must reopen regardless of textual stability elsewhere.*

27.5 Exact boundaries and explicit non-theorems

The theorem targets for Family II should be stated as implementation obligations, not as atmospheric aspirations. The chapter should aim at:

1. family-routing determinism for collection atlases;
2. finite-support descent soundness for collection and finite-map witnesses;
3. heap-summary faithfulness on tracked stars and epochs;
4. alias-hypercover coherence for admitted path families;
5. interface-summary conservativity under stable summary seals.

Each of these is testable because each ranges over concrete records introduced above.

A useful envelope theorem is the following.

Proposition 27.7 (Family-II exactness envelope)**.** *Any obligation discharged in Family II depends only on the declared finite collection support, tracked heap stars, admitted alias simplices, and sealed interface summaries named in its normalized record. Changes outside that semantic envelope cannot invalidate the discharge. Changes inside it must either preserve the same descent data or trigger reopening.*

The non-theorems should be equally explicit. This chapter does not claim exactness for arbitrary Python iteration order, generator exhaustion, reflection-heavy attribute creation, whole-program pointer analysis, global heap extensionality, open-world dynamic dispatch, or behavioral subtyping beyond declared summary laws. It does not infer covers that were never declared. It does not pretend that pairwise compatibility implies higher descent when the needed hypercover data was not constructed. And it does not allow an interface witness to settle a semantic dispute whose bridge theorem has not been supplied.

Chapter 28

Exact Z3 encodings III: strings, symbolic text, naming laws, and documentation shadows

28.1 Why text deserves its own structural chapter

Text deserves its own structural chapter because names, paths, labels, and documentation summaries are where a large fraction of public honesty obligations actually live. A function may compute the right result and still present the wrong flag name, the wrong module path, the wrong exported identifier, or the wrong summary line. Those failures are not secondary. They are failures on the public boundary, and the public boundary is part of the semantic object JuGeo is trying to govern.

The AG role here is exact and non-ornamental. For a coordinate c, let $\mathfrak{U}_\partial(c) = \{U_i \to c\}_{i \in I}$ be a cover of the public surface into charts such as exported symbol names, filesystem or import paths, CLI or protocol labels, and documentation regions. A naming or path assignment is then a local section of a presheaf of normalized textual surfaces,

$$\mathscr{T}_\partial(U_i),$$

and a documentation clause is a local section of a presheaf of textual assertions,

$$\mathscr{D}_\partial(U_i).$$

The documentation shadow is not the documentation section itself. It is a structured projection

$$\delta_{U_i} : \mathscr{D}_\partial(U_i) \to \mathscr{S}_{\mathrm{text}}(U_i),$$

into the admitted structural fragment $\mathscr{S}_{\mathrm{text}}$, together with provenance and boundary kind. This is the right place to say explicitly that documentation shadows are projections, not paraphrases and not embeddings of full prose meaning.

Once one states the objects this way, mismatch is naturally geometric. If two charts U_i and U_j export local naming data s_i and s_j, then their disagreement on the overlap U_{ij} is a local discrepancy

$$\eta_{ij} \in \mathscr{M}_{\mathrm{text}}(U_{ij}),$$

for a discrepancy presheaf $\mathscr{M}_{\mathrm{text}}$. A family of local naming choices may look consistent chartwise and still fail to globalize because the η_{ij} form a nontrivial Čech cocycle. In that case the system

should say so. It should report that the local naming discipline does not glue, rather than pretending that the failure is a mere string typo. When the cocycle is a coboundary, the inconsistency is repairable by renormalization or a compatible rename. When it is not, the right semantic object is an obstruction class in

$$\check{H}^1\big(\mathfrak{U}_\partial(c), \mathscr{M}_{\text{text}}\big).$$

The DTT role is equally direct. Public text is not free-floating metadata; it is indexed by exported artifacts and therefore induces dependent obligations. If e is a public export in context $\Gamma_\partial(c)$, then the system should treat its name, path, and shadow clauses as dependent inhabitants attached to that export:

$$\Gamma_\partial(c) \vdash e : \mathrm{Export}(k) \quad \Longrightarrow \quad \Gamma_\partial(c) \vdash n_e : \mathrm{Name}(e),\ p_e : \mathrm{Path}(e),\ \delta(d_e) : \Omega_{\text{text}}(e, n_e, p_e).$$

A public guarantee can therefore depend on textual boundary data without collapsing that data into free-form prose. This is one reason text cannot simply be relegated to the semantic oracle layer.

The AI role should remain controlled and subordinate. A controlled oracle may propose a candidate stem correspondence, a likely rename relation, or a candidate structured shadow for a prose sentence when the theory has no declared bridge yet. But the proposal enters the pipeline only as a candidate relation. It becomes structural only if it is normalized into an admitted family with declared provenance and replay conditions. Otherwise it remains a residual semantic obligation. The non-theorem should be stated at once: this chapter does not solve full natural-language semantics, does not prove documentation equivalence from embedding similarity, and does not let an oracle quietly rewrite public law.

The implementation consequence is that the system needs a dedicated text authority layer rather than a handful of helper utilities for string comparison. The authority center should remember local charts, normalization policy, admitted family membership, shadow provenance, overlap structure, and invalidation stamps. Without that dedicated layer, exact text obligations will be either over-approximated into useless heuristics or under-reported as mere cosmetic mismatches.

28.2 The normalized text environment

Text deserves its own normalized environment because names, paths, generated identifiers, CLI surfaces, and documentation shadows behave differently from arithmetic and heap facts. For a coordinate c, the text encoder should operate over

$$\Sigma_{\text{text}}(c) = \big(V_{\text{text}}, T, P, D, N, \delta, \mathfrak{U}_\partial, \Gamma_\partial, \chi\big),$$

where V_{text} is the family of symbolic string variables and structured path components, T is the tokenization and segmentation policy, P the active guard and path-condition family over those variables, D the imported documentation clauses, N the naming-law environment for symbols, paths, and exported labels, δ the declared shadowing map from documentation clauses to structural text predicates, $\mathfrak{U}_\partial$ the chosen cover of public textual charts, Γ_∂ the dependent public-surface context, and χ the provenance and origin partition. The important point is that the environment contains both normalization policy and chart structure. A text clause without its chart and policy is not yet an exact clause.

The normalizer should produce canonical atoms only relative to that environment. It should not erase the distinction among identifier normalization, path normalization, and documentation-shadow projection. A convenient grammar for normalized atoms is

$$a ::= x = y \mid x \preceq_T y \mid x \succeq_T y \mid \mathrm{TokIn}_T(x, y) \mid \mathrm{StemEq}_N(x, y) \mid \mathrm{PathEq}_N(x, y) \mid \mathrm{Fmt}_k(x, \bar{z}) \mid \phi_d,$$

where ϕ_d ranges over the image of δ. Here x and y may denote identifiers, path strings, normalized token sequences, or finite-format fields, depending on their family tags. The design constraint is that every emitted atom must belong to a declared exact family. If a clause requires unrestricted regex semantics, language-specific doc inference, or discourse-level entailment, it should remain outside this grammar and be routed to a residual family.

Algorithm sketch. The normalization pipeline should be explicit.

1. Build chart-local raw sections for names, paths, labels, and documentation spans from $\mathfrak{U}_\partial(c)$, preserving source spans and imported boundary records.
2. Apply declared normalization in family order: Unicode normalization policy, case policy, separator policy, escape handling, path component normalization, identifier segmentation, and token-boundary policy.
3. Classify each local clause into an admitted family of equality, prefix or suffix, token-membership, finite-format, path law, or documentation shadow. If classification fails, emit a residual text obligation with the same provenance.
4. Compute overlap maps on each U_{ij} and record any local discrepancy η_{ij} together with its origin in names, paths, or shadow clauses.
5. Emit exact clauses, residual clauses, and cache keys from the normalized environment. If there is a missing but plausible correspondence, a controlled oracle may propose a candidate stem or shadow relation, but the proposal must either be translated into an admitted exact clause under declared policy or remain explicitly residual.

The theorem target for this environment is not that normalization recovers meaning in general. It is that normalization is deterministic, family-respecting, and conservative with respect to the admitted comparison relation. The implementation consequence is straightforward: the cache key for text discharge must include the stamps of T, N, δ, and the relevant boundary imports. Text reuse without those stamps is unsound.

28.3 Encoding families

The text encoder should distinguish families instead of pretending that all text is one solver sort with ad hoc helper lemmas. Equality and disequality over normalized identifiers are one family. Prefix, suffix, token-membership, and reserved-token exclusion are another. Finite-format obligations are a third, covering relations such as qualified names, generated file names, module paths, CLI flags, and anchor strings that arise from templates with finitely many fields. Path laws are a fourth, because separator normalization, component equality, extension preservation, and relative-versus-absolute discipline are structural but should not be confused with arbitrary substring reasoning. Documentation shadows are a fifth family, because they enter as projections from D rather than as native string clauses.

A documentation shadow should therefore be a concrete record, not a slogan. For a documentation clause $d \in D$, the emitted structural artifact should have the form

$$\mathrm{Shadow}(d) = \big(\phi_d,\, \sigma_d,\, \kappa_d,\, \tau_d\big),$$

where ϕ_d is the structural predicate in the admitted fragment, σ_d is the source span or document coordinate, κ_d is the public-boundary kind to which the shadow applies, and τ_d records the translation

policy or bridge identifier by which the shadow was admitted. This record is what makes shadow-soundness testable. One can ask whether ϕ_d really came from a declared projection and whether replay is still valid under the same policy stamp.

The AG story continues at the level of gluing. A local name law in U_i and a local documentation shadow in U_j should glue only if their restrictions agree on U_{ij}. If they do, they define compatible descent data for a larger public surface. If they do not, the discrepancy presheaf should record whether the failure is a plain equality clash, a tokenization clash, a path-law clash, or a shadow mismatch. This matters for repair: different cocycles demand different gluing moves.

Proposition 28.1 (Text normalization determinism)**.** *If two text obligations differ only by alpha-renaming of symbolic variables, admissible path sugar, separator spelling covered by T, permutation of conjuncts, or equivalent presentation of the same declared shadow projection, then they normalize to the same N_{text} and therefore yield the same family classification, cache key, and origin partition up to stable identifier renaming.*

Proposition 28.2 (Shadow soundness for documentation projections)**.** *If the emitter produces* $\text{Shadow}(d) = (\phi_d, \sigma_d, \kappa_d, \tau_d)$ *from a documentation clause d, then ϕ_d lies in the image of the declared projection δ under policy stamp τ_d. In particular, the emitted structural clause is not permitted to strengthen, weaken, or paraphrase d beyond the declared projection schema.*

Proposition 28.3 (Fragment exactness)**.** *Let λ be a normalized text obligation all of whose clauses lie in the admitted families of equality, token law, finite format, path law, or declared documentation shadow. Then the emitted Z3 bundle is equisatisfiable with the normalized structural content of λ under the same guard bundle and imported boundary assumptions.*

The non-theorems should be equally explicit. This layer does not claim full natural-language understanding. It does not claim semantic equivalence of documentation bodies, rhetorical adequacy of summaries, or intent recovery from prose alone. It does not claim exact treatment of unrestricted regexes, arbitrary locale-sensitive normalization, or every filesystem convention in every environment. It does not allow a controlled oracle proposal to become law merely because it looks plausible. And it does not let a documentation shadow settle a broader semantic dispute unless a separate bridge theorem says that the shadow is adequate for that dispute.

The implementation consequence is that each family should have its own emitter, witness-lifter, and invalidation discipline, even if several families share the underlying solver string sort. Reusing a single generic emitter would destroy the provenance needed for replay and would blur the difference between name-law failure, path-law failure, and shadow failure.

28.4 Countermodels and clausewise explanation

Countermodels in the text layer should explain more than a failed string equation. They should explain whether the failure is local to one chart, imported across a boundary, or genuinely obstructed on overlaps. The witness object should therefore retain both model data and overlap discrepancy:

$$\mathcal{X}_{\text{text}} = \big(M_{\text{text}}, \eta, \Pi, \mathcal{R}, \chi\big),$$

where M_{text} is the satisfying assignment for the emitted text bundle, $\eta = \{\eta_{ij}\}$ is the computed discrepancy cochain on overlaps, Π is the active guard bundle, $\mathcal{R}$ is the public-surface support region, and χ is clause provenance. If η is trivial, the witness globalizes as a boundary-visible counterexample. If η is nontrivial, the system should say that the local witnesses do not glue and should surface the obstruction rather than forcing a fake global string assignment.

The witness-lifting recipe should be algorithmic. For a satisfying model that falsifies a wanted text implication, the lifter should:

1. read off symbolic assignments from M_{text} for all normalized names, path components, token variables, and shadow atoms in the active clause bundle;

2. replay inverse normalization chartwise, producing canonical source-local spellings and path forms under the same T and N policy stamps rather than guessing prettier strings;

3. compute the violated local clauses and the overlap discrepancy family η_{ij}, classifying each discrepancy as name-law, path-law, finite-format, or documentation-shadow mismatch;

4. attempt gluing across the cover $\mathfrak{U}_\partial(c)$, recording whether the local witness descends to a global public-surface witness or instead yields a nontrivial obstruction class;

5. emit a clausewise explanation object

$$\text{Explain}_{\text{text}}(\mathcal{X}_{\text{text}}) = \{(\lambda_i, \text{status}_i, \xi_i, \omega_i)\}_i,$$

   where status_i is one of used, violated, guarded_out, imported, or obstructed, ξ_i is the normalized witness fragment, and ω_i is the source-local origin and chart information.

This is where the AG language earns its keep. A counterexample that exists separately in the API chart and the documentation chart but fails to glue across a shared exported identifier is not merely two local bugs. It is a failed descent datum. Reporting the overlap cocycle makes visible whether the repair should be a local rename, a boundary treaty change, a shadow projection update, or a cover refinement. Without that distinction, text diagnostics become noisy and strategically useless.

Proposition 28.4 (Countermodel faithfulness for text families)**.** *Let λ be an admitted text obligation and let $\mathcal{C}_{\text{text}}$ be the emitted clause bundle from $N_{\text{text}}(\lambda)$. Every satisfying model of $\mathcal{C}_{\text{text}} \wedge \neg\lambda$ lifts to a replay-valid text witness under the same normalization policies, and every replay-valid lifted witness falsifies the corresponding normalized clause or overlap compatibility condition recorded in χ.*

Proposition 28.5 (Obstruction honesty)**.** *If chart-local text witnesses satisfy all local emitted clauses but fail to globalize because their overlap discrepancies define a nontrivial class in $\check{H}^1(\mathfrak{U}_\partial(c), \mathscr{M}_{\text{text}})$, then the explanation layer must report an obstruction witness rather than a fabricated global counterexample. In particular, local naming consistency is not permitted to be reported as global consistency when descent fails.*

The implementation consequences are immediate. The text layer must store chart-aware provenance, normalization-policy stamps, shadow records, and overlap graphs as first-class authority objects. Invalidation must trigger not only on source edits but also on changes to naming policy, path policy, imported interface summaries, or documentation-shadow bridges. Public diagnostics should distinguish violations of exact text law from residual semantic disagreements, and the orchestrator should treat unresolved text obstructions as real frontier items rather than cosmetic cleanup. This is the exact structural family for text: strong enough to verify and refute boundary-critical surface claims, modest enough not to counterfeit full natural-language semantics, and explicit enough to participate honestly in locality, gluing, and obstruction throughout the larger AG+DTT+AI machine.

Chapter 29

Exact Z3 encodings IV: sequences, finite maps, heap slices, and support-aware mutation

This chapter should make precise how richer structural data can still participate in the AG story. Sequence windows, finite maps, and heap slices are not arbitrary backend data structures; they are local charts on structured supports. Exact encoding here matters because these are the neighborhoods where local mutation, replay, and repair witnesses actually live.

29.1 Structured data should not be flattened too early

Sequence, map, and heap-slice obligations should preserve structural shape until the family boundary that actually needs flattening. Premature flattening destroys origin information, makes countermodels harder to lift, and obscures which support regions really matter for replay. The authority record for this chapter should therefore be

$$\mathcal{D}_{\text{struct}} = \big(\Pi_{\text{path}},\, \Pi_{\text{shape}},\, \Pi_{\text{mut}},\, \Pi_{\text{frame}},\, \mathcal{R},\, \chi\big),$$

where Π_{path} is the family of symbolic access paths, Π_{shape} the family of shape facts such as lengths, key domains, and tracked fields, Π_{mut} the family of mutation candidates, Π_{frame} the family of non-touch or support constraints, and $\mathcal{R}, \chi$ are support and provenance.

A symbolic access path should be an object in its own right:

$$p ::= x \mid p[i] \mid p[k] \mid p.f \mid p[\ell{:}r].$$

The normalizer should compare paths modulo alpha-renaming of roots and normalization of constant indices or keys, but should not eagerly compile every path into unrelated scalar atoms. This gives the emitter a chance to choose between sequence, finite-map, and heap-slice families with the right witness discipline.

The non-theorem for this section is important: the chapter does not claim that all structured data admits one solver encoding. It claims that the IR should preserve enough shape so that each exact subfamily can decide what it owns. In AG terms, premature flattening is a bad choice of local chart that destroys the overlap structure needed for honest gluing.

29.2 Sequence windows

The sequence family should center on windows rather than on monolithic sequence objects, because most support-local obligations mention only bounded slices, point updates, or prefix-suffix decompositions. The normal form should be

$$\mathcal{W} = \big(s,\, \ell,\, r,\, v,\, \Gamma_{\mathrm{idx}},\, \Gamma_{\mathrm{shape}},\, \mathcal{R},\, \chi\big),$$

where s is the base sequence, ℓ, r are symbolic or constant bounds, $v = s[\ell{:}r]$ the window term, Γ_{idx} the bound and indexing constraints, Γ_{shape} the local length or concatenation facts, and $\mathcal{R}, \chi$ support and provenance.

Exact sequence obligations in this chapter should include:

- point lookup under exact in-bounds guards;
- prefix, suffix, and concatenation decomposition;
- bounded equality of windows;
- exact single-update laws expressed as prefix-update-suffix decomposition;
- finite insertion and deletion obligations over declared windows.

Construction 29.1 (Window emission)**.** *Given a sequence update at index i,*

$$s' = \mathrm{update}(s, i, a),$$

the emitter should not flatten this directly into arbitrary extensional axioms. It should instead emit

$$s' = \mathrm{concat}\big(s[0{:}i],\, [a],\, s[i+1{:}|s|]\big)$$

under the exact index guard

$$0 \le i < |s|.$$

Likewise, a deletion should be emitted as concatenation of left and right windows, and an insertion as concatenation of prefix, inserted window, and suffix. If the surrounding obligation mentions only one resulting window, the emitter may project directly to that window instead of materializing the full sequence, but it must record that projection in χ.

The implementation authority center here should be a window normalizer that canonicalizes equivalent slices and computes support-local cache keys. Two obligations involving the same root sequence and the same normalized window should compare equal even if they arose from different surface syntax.

29.3 Finite maps and interface dictionaries

Finite maps in this chapter should be exact only over declared finite supports. The family record should be

$$\mathcal{M}_{\mathrm{fin}} = \big(K,\, V,\, \mathrm{Dom},\, \mathrm{val},\, \bot,\, \Gamma_{\mathrm{req}},\, \Gamma_{\mathrm{opt}},\, \Gamma_{\mathrm{ext}},\, \mathcal{R},\, \chi\big),$$

where K and V are key and value sorts, Dom is a finite symbolic domain set, val is the totalized lookup operator, $\bot$ the distinguished absence sentinel, Γ_{req} required-key laws, Γ_{opt} optional-key laws, Γ_{ext} finite extensionality clauses over Dom, and $\mathcal{R}, \chi$ support and provenance.

This family should be used for interface dictionaries, keyword packs, typed JSON-like summaries, and local configuration maps when the key universe is finite and declared. Exactness should be finite-support exactness:

$$m_1 =_{\mathrm{Dom}} m_2 \quad \Longleftrightarrow \quad \forall k \in \mathrm{Dom}\,.\ \mathrm{val}(m_1, k) = \mathrm{val}(m_2, k).$$

No claim of global map extensionality is needed or permitted.

A required-key violation should reconstruct as a map witness naming the missing or malformed key, its support region, and the interface clause it violated. Optional keys should carry a presence predicate rather than overloading $\perp$ as semantic absence when a real value could coincide with the sentinel. This distinction is essential for honest witness lifting.

29.4 Heap slices and mutation support

Mutation should be analyzed over heap slices with declared footprints. The slice object should be

$$\mathcal{H}_S = \big(\mathrm{Roots},\ \mathrm{Reach},\ \mathrm{Fields},\ \mathrm{Footprint},\ e_{\mathrm{in}},\ e_{\mathrm{out}},\ \mathcal{P}_{\mathrm{alias}},\ \Gamma_{\mathrm{frame}},\ \mathcal{R},\ \chi\big),$$

where Roots are slice roots, Reach the admitted reachability skeleton, Fields the tracked field set, Footprint the exact set of locations allowed to change, $e_{\mathrm{in}}, e_{\mathrm{out}}$ the pre- and post-mutation epochs, $\mathcal{P}_{\mathrm{alias}}$ the imported alias partition, Γ_{frame} the non-touch laws, and $\mathcal{R}, \chi$ support and provenance.

The write discipline should be explicit:

1. if a location lies in Footprint, writes are emitted as tracked updates from e_{in} to e_{out};
2. if a location lies outside Footprint but inside the slice, frame clauses require equality across epochs;
3. if alias uncertainty could pull an outside location into the effective footprint, the obligation must depend on the alias partition seal and reopen when that seal changes.

This is the chapter where support-aware mutation becomes concrete. The support of a frame theorem is not the whole function body; it is the footprint, the alias partition that justifies it, and the imported interface clauses that say which substructures are tracked. Invalidation should therefore proceed through those semantic dependencies, not through mere file adjacency.

29.5 Mutation countermodels as repair guides

A mutation countermodel should be a repair guide, not merely a failed satisfiability result. The artifact should be

$$G_{\mathrm{mut}} = \big(k,\ p,\ v_{\mathrm{before}},\ v_{\mathrm{after}},\ g_{\mathrm{gap}},\ w_{\mathrm{alias}},\ \mathcal{R},\ \chi\big),$$

where k classifies the failure kind, p is the minimal path or slice location, $v_{\mathrm{before}}, v_{\mathrm{after}}$ are reconstructed pre- and post-values, g_{gap} is a missing or false guard, w_{alias} an alias witness when relevant, and $\mathcal{R}, \chi$ support and provenance.

The lifter should classify mutation failures into at least four exact cases:

1. *missing guard*: the update is legal only under an index, key, or alias guard not established;
2. *frame violation*: a location outside the declared footprint changed;

3. *alias collapse*: two supposedly distinct locations became equal in the model;

4. *stale summary*: a replayed interface or heap summary no longer matches the current slice.

Proposition 29.2 (Support-local mutation replay)**.** *If a mutation discharge depends only on a heap slice record, a finite-map domain record, and an alias partition seal that all remain unchanged, then the discharge may be replayed. If any of those semantic dependencies changes, the mutation judgment must reopen even if the surrounding text is unchanged.*

The non-theorems should also be stated. This chapter does not claim global heap extensionality, whole-program mutation completeness, or precise reachability for arbitrary Python objects. It claims exactness only for declared slices, windows, finite domains, and tracked fields.

Chapter 30

Exact Z3 encodings V: tensor extents, affine legality, quantifier discipline, and witness extraction

This chapter should explain why affine geometric structure matters so much for implementation even when the user-facing claim sounds semantic. Tensor shapes, index legality, broadcasting, and non-overlapping writes are local geometric facts on coordinate neighborhoods. When they fail, they often fail by exposing a concrete witness simplex—an extent assignment and index tuple—whose obstruction is entirely structural. This is an explicit AG point: local legality regions are polyhedral charts, and witness extraction identifies where a candidate section leaves the allowed chart.

30.1 Why this family matters disproportionately

Tensor and affine-shape obligations matter disproportionately because they concentrate many of the structural claims that are expensive to debug later: bounds safety, broadcasting legality, loop-index compatibility, non-overlapping writes, and layout-preserving transformation laws. A large fraction of failures that look "semantic" at the API level are structurally affine at the support level. The authority record for this family should therefore be

$$\mathcal{T}_{\text{aff}} = \big(\vec{d},\, \vec{s},\, \vec{i},\, \Gamma_{\text{shape}},\, \Gamma_{\text{access}},\, \Gamma_{\text{layout}},\, \mathcal{R},\, \chi\big),$$

where $\vec{d}$ are extent variables, $\vec{s}$ strides or layout coefficients, $\vec{i}$ index variables, Γ_{shape} shape and broadcast constraints, Γ_{access} access expressions, Γ_{layout} legality constraints about overlap and storage, and $\mathcal{R}, \chi$ support and provenance.

This family should own only those obligations whose legality can be presented as affine or guarded quasi-affine arithmetic over finite-rank tensor metadata. It should not attempt to absorb arbitrary numeric code merely because indices appear.

30.2 Affine and quasi-affine normal forms

The normal form for an indexed access should be

$$a(\vec{i}) = A\vec{i} + \vec{b},$$

together with a guard polyhedron

$$G(\vec{i}) \equiv C\vec{i} \leq \vec{c},$$

where A, C are integer matrices and $\vec{b}, \vec{c}$ integer vectors or symbolic extent expressions. For each access site, the family record should store

$$\mathcal{A}_{\text{idx}} = \big(A,\, \vec{b},\, G,\, \vec{d},\, \text{mode},\, \mathcal{R},\, \chi\big),$$

with mode $\in \{\mathsf{read}, \mathsf{write}\}$.

A quasi-affine access should be normalized as a finite guarded family

$$\{(G_j, A_j, \vec{b}_j)\}_{j=1}^{m},$$

not as a single approximate nonlinear expression. This allows exact routing of piecewise-affine index code while preserving guard structure for explanation and replay.

Construction 30.1 (Affine legality emission)**.** *For each access family:*

1. *emit bound legality clauses*
$$G(\vec{i}) \Rightarrow 0 \leq A\vec{i} + \vec{b} < \vec{d};$$
2. *for two writes with access forms $(A_1, \vec{b}_1, G_1)$ and $(A_2, \vec{b}_2, G_2)$, emit non-overlap legality as*
$$G_1(\vec{i}) \wedge G_2(\vec{j}) \Rightarrow A_1\vec{i} + \vec{b}_1 \neq A_2\vec{j} + \vec{b}_2$$
when the transform requires disjoint writes;
3. *for broadcast obligations, emit exact extent equalities or singleton-dimension clauses rather than informal compatibility claims;*
4. *residualize any access whose legality depends on nonlinear arithmetic, data-dependent symbolic indirection, or undeclared layout summaries.*

This section should also name its implementation authority centers: an affine normalizer, a shape-environment builder, a broadcast-law compiler, and a legality emitter. Without these concrete centers the theory remains untestable.

30.3 Quantifier discipline

Quantifiers should be admitted only through finite, schema-generated disciplines. The authority record should be

$$\mathcal{Q} = \big(\vec{u},\, D(\vec{u}),\, \varphi(\vec{u}),\, \pi,\, \omega,\, \mathcal{R},\, \chi\big),$$

where $\vec{u}$ are bound variables, $D(\vec{u})$ the explicit finite or Presburger domain restriction, $\varphi(\vec{u})$ the quantified body, π the instantiation policy, ω the witness policy, and $\mathcal{R}, \chi$ support and provenance.

The admissible schemes should be narrow:

1. bounded universal quantification over tensor axes or loop indices with explicit extent bounds;
2. existential witnesses that can be extracted as violating index tuples;
3. finite extensionality over a declared support set;

4. schema-generated quantifiers from domain packs whose bridge theorems say exactly how they instantiate.

Unrestricted quantification should be a non-theorem. The chapter should not rely on solver heroics to make such formulas workable. If a quantifier cannot be tied to a declared finite or Presburger domain, it should be left as a higher-level obligation rather than smuggled into this family.

30.4 Witness extraction and proof burden

The distinctive product of this family is a geometric witness: a concrete extent assignment, index tuple, or pair of aliasing writes showing why a legality theorem failed. The extracted witness should have shape

$$W_{\text{aff}} = \big(\vec{d}^{\,*},\, \vec{i}^{\,*},\, \vec{j}^{\,*},\, \text{AccessTrace},\, \text{GuardTrace},\, \mathcal{R},\, \chi\big),$$

where $\vec{d}^{\,*}$ is the concrete extent assignment, $\vec{i}^{\,*}$ and $\vec{j}^{\,*}$ violating index tuples when relevant, AccessTrace records the normalized affine forms appealed to, and GuardTrace records which branch or broadcast guards were active.

The proof burden should therefore be precise.

Proposition 30.2 (Affine witness faithfulness)**.** *If an affine or quasi-affine legality clause is falsified by a satisfying model of the emitted structural bundle, then the extracted witness W_{aff} reconstructs an actual violating extent and index assignment for the same normalized access family and guard bundle.*

Proposition 30.3 (Quantifier-schema conservativity)**.** *Any quantified legality result admitted in this chapter depends only on a declared quantifier schema $\mathcal{Q}$ with explicit domain restriction. Reuse across replay is sound only while the schema, its domain pack seals, and its support region remain unchanged.*

The non-theorems should be plain. This family does not claim completeness for nonlinear tensor arithmetic, data-dependent sparse indirection, arbitrary symbolic layouts, or general quantified array properties. It claims exactness for declared affine and guarded quasi-affine fragments with explicit witness extraction.

Chapter 31

Exact Z3 encodings VI: partial functions, algebraic surfaces, exception-valued semantics, and model reconstruction

This chapter should handle the exact fragment in which structural truth depends on where evaluation is defined, which exceptional stratum a computation lands in, and whether a solver assignment can be glued back into a Python-shaped witness. Partiality is not a nuisance to be erased by default values. It is the local geometry of definability. Likewise, an exception is not just a Boolean side channel; it is a section landing in a different fiber. Family VI should therefore make definability charts, exception strata, algebraic surface atlases, and reconstruction algorithms first-class authority objects. Otherwise the implementation will keep faking totality and then emitting witnesses that do not correspond to any honest local section.

31.1 Why Python obligations are full of partiality

Python obligations are saturated with partial functions: indexing, key lookup, attribute access, numeric operations with side conditions, conversions, protocol methods whose domains are not ambiently total, and user summaries with explicit admissibility clauses. The exact fragment should therefore not encode a partial operation $f : X \rightharpoonup Y$ as a total function plus folklore. It should encode the graph and its domain chart:

$$\mathfrak{P}_f = \big(X,\, Y,\, \mathrm{Def}_f,\, \mathrm{Graph}_f,\, \mathcal{U}_f^{\partial},\, \Gamma_{\mathrm{dom}},\, \Gamma_{\mathrm{graph}},\, \mathcal{R},\, \chi\big),$$

where $\mathrm{Def}_f(x)$ is the exact definability predicate, $\mathrm{Graph}_f(x, y)$ is the value relation on the domain, and $\mathcal{U}_f^{\partial}$ is the cover by admissible domain charts.

For composite expressions, the implementation should build a definability hypercover rather than a single monolithic guard. If

$$e = e_n \circ \cdots \circ e_1,$$

then each e_i contributes a local domain chart, and the admitted hypercover records which combinations of those charts are semantically meaningful. This matters because partiality failures are often not single missing guards but failed descent between local domains. One branch may establish

lookup presence, another may establish attribute existence, and the intended global claim may still fail because no common domain chart was constructed on the overlap.

A useful lowering discipline is:

$$\begin{aligned} \mathrm{lookup}(m,k) &\rightsquigarrow \mathrm{pres}(m,k) \wedge \mathrm{val}(m,k) = v, \\ \mathrm{getattr}_f(o) &\rightsquigarrow \mathrm{has}_f(o) \wedge \mathrm{fld}_f(o) = v, \\ \mathrm{call}_g(\vec{x}) &\rightsquigarrow \mathrm{Adm}_g(\vec{x}) \wedge \mathrm{Graph}_g(\vec{x}, v). \end{aligned}$$

The point is not to proliferate predicates; it is to make domain obligations explicit so that missing guards become first-class structural failures.

Algorithmically, the partiality pass should:

1. hoist every definability side condition into a named domain clause;
2. split each partial obligation into a domain component and a graph component;
3. route domain clauses through exact families already developed for keys, heaps, aliases, and interfaces when possible;
4. record whether a later witness should reconstruct a missing domain proof, a value mismatch on a valid domain, or a genuine overlap failure between local domains.

The AG reason for this discipline is that a partial computation is a local section on an open domain, not a global section on the whole ambient state space. A missing guard is therefore not merely a failed precondition. It is the witness that the claimed global section never existed on the chosen cover.

Proposition 31.1 (Domain extraction soundness)**.** *Let $\mathfrak{P}_f$ be an admitted partial-function record. If the emitted domain clauses establish $\mathrm{Def}_f(x)$ and the emitted graph clauses establish $\mathrm{Graph}_f(x,y)$ on the same normalized support, then the corresponding structural claim about $f(x)$ is sound on that support. If either component fails, the lifted witness correctly classifies the failure as domain failure or graph failure.*

31.2 Exception-valued structural semantics

Once partiality is explicit, many Python obligations want a total semantic object again, but not by pretending exceptions are values of the same kind. The right exact object is an exception-valued fiber:

$$\mathrm{Exc}_E(V) = \mathsf{ok}(V) + \mathsf{err}(E),$$

with finite admitted exception tag family E. A normalized expression should therefore carry a result record

$$\mathcal{X}_e^\sharp = \big(r_e,\, V,\, E,\, \Gamma_{\mathrm{ok}},\, \Gamma_{\mathrm{exc}},\, \Gamma_{\mathrm{flow}},\, \mathcal{U}_{\mathrm{exc}},\, \mathcal{R},\, \chi\big),$$

where r_e has sort $\mathrm{Exc}_E(V)$, Γ_{ok} are value-side clauses, Γ_{exc} are exception-tag and payload clauses, Γ_{flow} are control-flow clauses, and $\mathcal{U}_{\mathrm{exc}}$ is the exception-stratified cover.

The cover should refine into one non-exceptional chart and one chart per admitted exceptional stratum:

$$\mathcal{U}_{\mathrm{exc}} = \{U_{\mathrm{ok}}\} \cup \{U_{\mathrm{err},t} \mid t \in E\}.$$

A local section on U_{ok} provides a value witness. A local section on $U_{\mathrm{err},t}$ provides an exception payload witness for tag t. Overlaps between distinct exceptional strata are empty; overlaps between

producer and handler charts are nontrivial only when the handler claims to catch the relevant tag. This is the implementation meaning of exception handling as descent. A `try/except` block glues the producer chart to a handler chart on the exceptional overlap it claims to cover.

The emission discipline should be exact:

1. emit tag disjointness and exhaustiveness only for the finite tag family E;
2. emit value clauses under the guard $\mathrm{isOk}(r_e)$;
3. emit exceptional clauses under the guard $\mathrm{isErr}_t(r_e)$ for each admitted tag t;
4. lower handlers as gluing maps between the relevant exceptional chart and the handler's result chart, not as ambient control-flow folklore.

If a totality claim says "this call returns a value," then an exception witness is not merely a counterexample value of the wrong shape. It is a failure of descent from the producer's section family into the claimed total-value chart. That is why exception-valued semantics belongs in this exact family.

Proposition 31.2 (Exception-tag disjointness and handler soundness)**.** *For any admitted exception-valued record $\mathcal{X}_e^\sharp$, the emitted clauses guarantee that a satisfying model selects exactly one admitted stratum of $\mathcal{U}_{\mathrm{exc}}$. If a handler summary is used to eliminate a subset of exceptional strata, replay is sound only when the same handled-tag set and handler law bundle remain in force.*

This section should also name its non-theorem boundary. It does not claim a full CPython operational semantics for exceptions, traceback objects, or arbitrary foreign-function error channels. It claims exactness only for finite admitted exception tags and explicitly modeled payload fields.

31.3 Algebraic data surfaces without pretending Python is ML

Python is not ML, but many support-local obligations still live on finite tagged surfaces: `None`-versus-value states, result objects with status tags, dataclass-like records, typed dictionaries with declared required keys, finite enum payloads, and structured exception payloads. The chapter should call these *algebraic surfaces* because they are finite atlases of product fibers glued along explicitly declared coercion overlaps, not because Python suddenly became a closed-world algebraic language.

The exact surface object should be

$$\mathcal{S}_{\mathrm{alg}} = \big(T,\, \mathrm{tag},\, \{F_t\}_{t\in T},\, \Gamma_{\mathrm{inv}},\, \Gamma_{\mathrm{coh}},\, \mathcal{H}_\bullet^{\mathrm{surf}},\, \mathcal{R},\, \chi\big),$$

where T is the finite tag family, tag the discriminant, F_t the field family on chart U_t, Γ_{inv} the per-chart invariants, Γ_{coh} the admissible coercion or compatibility laws on overlaps, and $\mathcal{H}_\bullet^{\mathrm{surf}}$ a surface hypercover used when nested surfaces interact with guards, heaps, or exception strata.

Most overlaps between distinct constructor charts should be empty. When they are not empty, the reason must be declared: subclass coercion, sentinel promotion, protocol view, or another explicit bridge theorem. This is why the section title must deny the false theorem that Python data should be encoded as if it enjoyed ML-style exhaustive pattern matching. The atlas is finite only where the support region and domain pack say it is finite.

The surface compiler should therefore:

1. create finite tagged datatypes or discriminated record packs only for declared tag families;

2. attach field selectors to the chart on which they are defined;

3. emit cross-chart coherence clauses only for declared coercions or bridge maps;

4. residualize open-world class hierarchies, reflective attribute creation, and metaclass-dependent shape changes.

The AG content is exact. A surface chart U_t is a local stratum. A reconstructed witness belongs to a chart or to an admitted overlap. If no chart or overlap can carry the witness, the failure is not "mysterious dynamic behavior" but a statement that the chosen finite atlas was insufficient for descent.

Proposition 31.3 (Finite-atlas surface faithfulness)**.** *If $\mathcal{S}_{\mathrm{alg}}$ is admitted and the emitted clauses are satisfiable, then the lifted witness reconstructs a tag $t \in T$ and field assignment on U_t, or an explicitly declared overlap witness, consistent with the same normalized support and invariants. No stronger claim is made for values outside the declared atlas.*

31.4 Effect summaries and branch-sensitive value reconstruction

Effect summaries in this family should describe how partiality, exceptions, heap movement, and value surfaces distribute across branch charts. A useful normalized object is

$$\mathcal{B}^\sharp = \big(\{(g_i, r_i, \Delta_i)\}_{i\in I},\, \Gamma_{\mathrm{join}},\, \Gamma_{\mathrm{frame}},\, \Gamma_{\mathrm{exc}},\, \mathcal{U}_{\mathrm{br}},\, \mathcal{H}^{\mathrm{br}}_\bullet,\, \mathcal{R},\, \chi\big),$$

where each g_i is a branch guard, r_i the branch-local result surface, Δ_i the branch-local heap delta or effect summary, Γ_{join} the join laws, Γ_{frame} the branch-stable frame clauses, Γ_{exc} the exception-flow clauses, and $\mathcal{U}_{\mathrm{br}}$ the branch cover.

A branch cover is not just a control-flow artifact. It is a semantic cover by local execution charts. The join point is a descent problem. If two branch-local sections disagree on the overlap where both guards can hold, then the join has a genuine Čech obstruction. That obstruction may be solvable by strengthening a guard, refining a summary, or splitting a chart; it should not be buried under ad hoc SSA plumbing.

The implementation should treat ϕ-like joins, `if`/`else` merges, and `try`/`except`/`finally` merges uniformly:

1. every branch gets its own local result surface and heap delta;

2. every join gets explicit overlap laws between branch charts that can co-occur;

3. every exceptional branch is just another chart in the branch hypercover, not a special case outside the semantics;

4. witness reconstruction chooses the minimal chart or simplex on which the solver model actually lives.

This point is important for implementation. If the model determines a single branch chart, the witness should say so. If the model determines only an overlap class because guards were abstracted, the witness should be an overlap witness, not a fabricated single-branch trace. That distinction is what keeps reconstruction honest.

Proposition 31.4 (Branch-join descent)**.** *If the local branch sections of $\mathcal{B}^\sharp$ satisfy the emitted join laws on every admitted overlap in $\mathcal{U}_{\mathrm{br}}$ and on the admitted higher simplices of $\mathcal{H}^{\mathrm{br}}_\bullet$, then the reconstructed post-join result is faithful to the same branch-sensitive effect summary. If the join laws fail, the resulting witness identifies the minimal offending overlap or higher simplex.*

31.5 Model reconstruction as a first-class algorithm

Model reconstruction should be specified as an audited semantic algorithm, not as a renderer pass that happens to read solver assignments. The reconstructed artifact should have a canonical shape

$$W_{\mathrm{recon}} = \big(k,\, U^*,\, \pi^*,\, \sigma^*,\, \Delta^*,\, \tau^*,\, \mathcal{R},\, \chi\big),$$

where k is the witness kind, U^* is the minimal chart or simplex carrying the witness, π^* is the guard or path trace, σ^* is the reconstructed value or exception surface, Δ^* is any reconstructed heap delta, τ^* is the clause-origin trace, and $\mathcal{R}, \chi$ are support and provenance.

The reconstruction algorithm should be:

1. identify the violated normalized clause family and its admissible charts from χ;
2. choose the smallest chart or simplex U^* in the relevant cover or hypercover on which the model fixes a consistent local section, preferring 0-simplices to overlaps and overlaps to higher simplices;
3. reconstruct domain witnesses, exception tags, surface tags, field values, and heap deltas only from selectors legal on U^*;
4. replay the corresponding encoder fragment on the reconstructed artifact and reject the witness if replay fails;
5. minimize the reported support and classify the result as domain failure, exception witness, surface mismatch, stale summary, or higher-overlap obstruction.

The insistence on replay is not bureaucratic. It is the theorem-protecting step that prevents a pretty printed model from masquerading as a faithful counterexample. If replay of the reconstructed witness does not reproduce the same normalized failure, the witness is invalid. Likewise, if only a higher simplex carries the failure, the artifact must say that the obstruction lives on an overlap or higher overlap. A fake point witness would destroy the AG content by pretending a nontrivial cocycle were already a point-level failure.

Two theorem targets should be made explicit.

Proposition 31.5 (Reconstruction replay soundness)**.** *If W_{recon} is accepted for a model of an admitted Family-VI clause bundle, then replay of the corresponding normalized encoder on W_{recon} reproduces the same violated clause family on the same support region. Accepted witnesses are therefore faithful semantic artifacts, not presentation-only summaries.*

Proposition 31.6 (Obstruction classification honesty)**.** *If the minimal carrier of a failure is an overlap or higher simplex in the chosen cover or hypercover, then the accepted witness is classified as an overlap or higher-overlap obstruction and not as a pointwise value failure. In particular, Family VI may not silently collapse nontrivial descent failures into fabricated branch-local traces.*

The non-theorems should be equally plain. This chapter does not claim full executable reconstruction of arbitrary Python runs, uniqueness of human-readable witnesses, exact semantics for native extensions or reflection-heavy libraries, or total classification of all dynamic dispatch behavior. It claims exactness only for admitted partial-function charts, finite exception strata, declared algebraic surfaces, branch summaries with explicit joins, and witnesses that survive replay through the same normalization and cover data.

Chapter 32

Internal representations and the IR stack

If the rewrite is to stay coherent, it should revolve around a small family of authority objects and force every subsystem to project through them. A workable basis is:

$$\begin{aligned}
\mathcal{C} &= (\text{site}, \text{region}, \text{support}, \text{epoch}, \text{pack}, \text{mode}),\\
\Lambda &= (\text{id}, \text{text}, \text{shadow}, \text{core}, \text{polarity}),\\
\mathcal{O} &= (\text{id}, \mathcal{C}, \Lambda, \text{deps}, \text{route}, \text{policy}, \text{status}),\\
\mathcal{J} &= (\mathcal{C}, \Phi^{+}, \Phi^{-}, \mathcal{O}_{\text{open}}, \mathcal{E}, \text{trust}),\\
\mathcal{E} &= (\text{kind}, \text{payload}, \text{support}, \text{provenance}, \text{validator}, \text{trust}),\\
\mathcal{X} &= (\text{kind}, \text{witness}, \text{conflict}, \text{support}, \text{repair}),\\
\mathcal{K} &= (\text{scope}, \text{claims}, \text{residuals}, \text{projection}, \text{policy}, \text{seal}).
\end{aligned}$$

Here $\mathcal{C}$ is the coordinate, Λ the clause object, $\mathcal{O}$ the obligation, $\mathcal{J}$ the judgment state, $\mathcal{E}$ the evidence item, $\mathcal{X}$ the obstruction, and $\mathcal{K}$ the certificate or public closure object. The point is not mere elegance. It is to make sure that verification, repair, orchestration, rendering, and regression all speak the same internal language.

32.1 The theory wants a small number of canonical records

The practical rule should be severe. Subsystems may own local helper views, but they should not introduce rival authority centers for semantics already represented above. A front end may compute AST-local binding records, a solver family may keep emission-local clause bundles, and a renderer may cache public summaries. But these are compiled views, not semantic authorities. Any state that must survive invalidation, replay, bridge transport, or certificate projection should be stored as one of the canonical records or as an explicitly declared projection of one.

The theorem targets are canonical-serialization determinism, projection faithfulness, and non-forgetting under transport. Canonical-serialization determinism says that semantically equal authority objects have equal stable serializations up to declared ordering conventions. Projection faithfulness says that any compiled view used by another subsystem can be traced back to the authority object from which it was derived. Non-forgetting says that a transition may residualize, reopen, or demote, but it may not silently discard an open obligation or supporting witness. The non-theorems should be plain. This section does not claim that every helper data structure must be

globally canonical, and it does not claim that one tuple shape is optimal for every runtime path. It claims only that the semantic center must remain small enough to govern the rest of the code.

32.2 Normal forms where comparison, caching, and solver routing need them

Normal forms should exist exactly where semantic equality must survive replay, cache lookup, transport, or solver routing. A usable canonical key for a routable fragment is

$$K(o) = (\text{family}, \text{support}, \text{core}, \text{guards}, \text{imports}, \text{seal}),$$

with alpha-renaming, clause-order permutation, and family-admitted syntactic sugar quotiented out before serialization. The point is not maximal canonicalization. The point is to make equality of obligations a theorem-backed fact wherever routing and replay depend on it.

The implementation should therefore separate three normalizers: one for context-sensitive names and binders, one for clause algebra inside each family, and one for dependency/seal metadata. Cache hits should require all three to agree. This is computationally stricter than ordinary memoization, but it is what prevents replay from accidentally reusing a result whose imported theorem, interface seal, or path guard has changed.

32.3 An implementation-ready theory needs a stratified intermediate language

A single IR is too blunt for JuGeo. The dissertation should specify at least four layers: a source-near elaboration IR carrying syntax-linked context, a judgment IR carrying normalized clauses and supports, a family IR carrying routable structural fragments or explicitly typed residuals, and a certificate IR carrying public claims plus their evidence provenance. Lowering between layers should be explicit and typed:

$$\text{ElabIR} \to \text{JudgmentIR} \to (\text{FamilyIR} \sqcup \text{ResidualIR}) \to \text{CertificateIR}.$$

Each arrow should preserve support, provenance, and residual visibility.

This stratification is what makes the AG+DTT+AI combination computational rather than rhetorical. The AG layer needs support and overlap data to survive lowering. The DTT layer needs contexts, residual obligations, and witness types to survive lowering. The AI layer needs proposal slots without being granted certificate authority. A stratified IR stack gives each of these a home and makes it possible to write tests about where information may or may not be forgotten.

32.4 Lowering should preserve ambiguity instead of prematurely deciding it

Lowering should produce residual ambiguity records whenever the next layer cannot represent a claim exactly. A useful sum type is

$$\mathsf{Lowered}(o) \in \{\mathsf{Exact}(x),\ \mathsf{Split}(\vec{x}, \rho),\ \mathsf{Residual}(r),\ \mathsf{Blocked}(b)\}.$$

If two families are both plausible, the system should preserve that branching as a routing choice with recorded alternatives. If no family is exact, the result should be residual, not guessed over.

This rule is computationally important because many false claims enter systems precisely at lowering boundaries. The traditional temptation is to pick one backend and hope. JuGeo should instead retain ambiguity until either a theorem-backed classifier resolves it or the controller decides which exact or mixed route to pursue. Prematurely deciding ambiguity destroys replay honesty and makes later counterexamples uninterpretable.

Chapter 33

Deduction rules and judgment transitions

The deduction chapter must make an explicit shift from "a verifier that mutates state" to "a typed machine that moves local sections over a site and updates obstruction data on overlaps." A judgment is not merely a Boolean outcome attached to a file. It is a dependent section over a support region. If U is a region of the active site or hypercover and Γ_U the local context assembled for that region, then the engine should treat

$$s_U : \Gamma_U \vdash \mathcal{J}_U$$

as the primary object, together with an overlap discrepancy cocycle

$$\omega_{UV} \in \mathcal{X}(U \cap V)$$

recording why neighboring local sections do or do not glue. A transition is therefore a movement

$$(\Gamma_U, s_U, \omega) \rightsquigarrow (\Gamma'_U, s'_U, \omega')$$

whose semantic content is twofold: it replaces a local section in a typed context and it updates the obstruction class carried by overlaps touching $\mathrm{supp}(U)$. This algebraic-geometric reading is not decorative. It is the only way to make reopening, overlap repair, and federation later in the dissertation mathematically uniform.

The implementation must therefore name exact authority centers. The semantic authorities for this chapter should be ContextAuthority for normalized dependent contexts, JudgmentAuthority for accepted local judgments, ObligationAuthority for live dependent obligations and residuals, ObstructionAuthority for overlap and higher-overlap discrepancy records, EvidenceAuthority for validated evidence items, and ReplayAuthority for the transition ledger. Everything else is a compiled view. In particular, ReadyQueue, SolverBatchView, OverlapFrontier, CertificateSnapshot, and RoutingAgenda should be described as disposable projections compiled from these authorities under stable serialization. This naming discipline matters because invalidation and replay must later target authorities, not whatever cache happened to be convenient.

Dependent type theory should appear at the level of the transition judgment itself. A useful form is

$$\Gamma_U \vdash \Xi \xrightarrow{\alpha}_{U,\chi} \Xi' : \Sigma\big(\Delta\mathcal{J}_U,\, \Delta\mathcal{O}_U,\, \Delta\mathcal{X}_{\mathrm{Star}(U)},\, \Delta\Pi\big),$$

where χ records provenance and route policy, $\Delta\mathcal{J}_U$ is the local judgment delta, $\Delta\mathcal{O}_U$ the dependent residual family, $\Delta\mathcal{X}_{\mathrm{Star}(U)}$ the induced obstruction update on the support star, and $\Delta\Pi$ the replay trace delta. The point of the Σ-form is that the engine never performs a bare state mutation.

Every accepted transition returns a typed package containing the new local section, the residual obligations still to be inhabited, the obstruction effect, and the replay material needed for later reopening. Constructors such as split, route, discharge, residualize, and glue should be presented as typed introduction forms for these transition packages, while reopening is an elimination rule for stale certificates under changed support.

AI belongs here only as a typed proposal and routing subsystem. The admissible AI records should be candidate route suggestions, search proposals for witness spaces, and typed repair proposals for unresolved residuals. They may populate RoutingAgenda or ProposalQueue, but they do not enter EvidenceAuthority or JudgmentAuthority directly. Any AI-produced object must first be turned into an ordinary candidate term or route record with declared support, expected codomain, and validator family. The non-theorem should be stated explicitly: no transition rule in this chapter licenses semantic acceptance on the basis of fluent prose, statistical confidence, or untyped suggestion.

Construction 33.1 (Support-local transition executor)**.** *For a ready obligation $o \in \mathcal{O}_U$:*

1. *restrict the current authority state to the local context Γ_U and the overlap star $\mathrm{Star}(U)$;*
2. *normalize o into structural premises, semantic premises, routing premises, and dependent residual constructors;*
3. *route each fragment through an admissible validator, recording the chosen route and its provenance in Π;*
4. *assemble the resulting evidence into either a discharged section s'_U, a residual family ρ_U, or a contradiction witness x_{UV} on overlaps;*
5. *update $\mathcal{X}$ by replacing the previous overlap discrepancy data on $\mathrm{Star}(U)$ with the new local coboundary contribution and any surviving obstruction classes;*
6. *glue the accepted local change into certificate projections without mutating any authority outside the declared dependency closure.*

Proposition 33.2 (Local transition soundness)**.** *If*

$$\Gamma_U \vdash \Xi \xrightarrow{\alpha}_{U,\chi} \Xi'$$

is admitted by the declared validators for α, then Ξ' agrees with Ξ outside the recorded dependency closure of $\mathrm{Star}(U)$, and the change in obstruction state is exactly the transported local discrepancy induced by replacing the old local section with the new one together with any residual overlap witnesses explicitly recorded in $\Delta\mathcal{X}_{\mathrm{Star}(U)}$.

Proposition 33.3 (Residual conservation)**.** *Every unresolved dependent premise produced during a transition is represented in Ξ' either as a live residual obligation, as an admitted obstruction witness, or as a contradiction entry. Silent disappearance of proof burden is not an admissible transition.*

The theorem targets for the chapter should therefore be support-locality of transition effects, replay determinism for fixed validator outcomes, residual conservation, obstruction-accounting faithfulness, and compiled-view faithfulness. The non-theorems should be equally plain: this chapter does not claim global confluence for arbitrary scheduling, completeness of residual elimination, or success of any proposal search policy. It claims only that accepted engine moves are typed local-section updates with explicit obstruction semantics and auditable replay.

Chapter 34

Incrementality, invalidation, and persistent semantic memory

Incrementality must be defined as a sheaf-theoretic update law before it is discussed as a performance optimization. A semantic change event δ with support S should be interpreted as a replacement of local sections on S together with a restriction of unaffected sections away from the support and a gluing step on the overlap frontier. If $\mathfrak{M}$ denotes the persistent semantic memory, then the intended update law is of the form

$$\mathfrak{M}' = \mathrm{Glue}\Big(\mathfrak{M}\big|_{X\setminus \mathrm{cl}(\mathrm{Star}(S))},\, \mathfrak{M}^{\delta}_{\mathrm{Star}(S)},\, \tau_{\delta}\Big),$$

where $\mathfrak{M}^{\delta}_{\mathrm{Star}(S)}$ is the recomputed local memory over the affected star and τ_{δ} is the treaty bundle governing how old and new summaries compare on the overlap boundary. This is the right formalization because an incremental system is sound only when unchanged regions are preserved by restriction, changed regions are recomputed locally, and boundary compatibility is rechecked rather than assumed.

The authority centers should therefore be explicit and persistent. The chapter should name SemanticMemoryAuthority for accepted local summaries, DependencyAuthority for support and dependency closures, InvalidationAuthority for the rules that map changes to stale semantic objects, ReplayAuthority for stable replay traces, and PackMemoryAuthority for imported pack seals, bridge dependencies, and transported theorems. A memory cell over a region U should have dependent shape

$$\mathfrak{m}_U = \big(\Gamma_U,\, \mathcal{J}_U,\, \mathcal{O}_U,\, \mathcal{X}_U,\, \mathcal{E}_U,\, \mathcal{K}_U,\, \sigma_U,\, \chi_U\big),$$

where σ_U is a stable semantic hash computed from canonical authority objects rather than from raw syntax. The compiled views should be named and demoted accordingly: HotCache, SolverMemo, RetrievalIndex, PromptSlice, and BenchmarkProjection are projections, not authorities. They may accelerate replay, but they may not carry semantic truth on their own.

Dependent type theory matters here because invalidation is not only support-based but context-based. A cached result may be stale even when syntax nearby is unchanged if its normalized context has changed. Thus replay eligibility should be typed by a context-preservation judgment

$$\Gamma_U \equiv_{\sigma} \Gamma'_U$$

together with pack-seal equality and support equality. A reused summary is admissible only when its dependent premises remain well-typed in the new context. If a binder, imported theorem, constructor family, or bridge translation changes, then every residual and certificate whose type

mentions that dependency must be invalidated, even if its text or file origin is untouched. This is the correct DTT account of incrementality: reuse is transport along preserved typing data, not textual coincidence.

AI again has a narrow role. It may rank replay candidates, search for likely reuse regions, or propose routing priorities for recomputation, but it may not authorize reuse. An AI-generated retrieval hit should compile only to a candidate memory key together with an expected type, support region, and admissibility check. The chapter should say openly that PromptSlice and retrieval memories are planning aids, not evidence classes, and that no certificate may cite them as proof material.

Construction 34.1 (Restrict–invalidate–replay–glue)**.** *Given a change event δ with semantic support S:*

1. *compute the dependency closure $D(\delta)$ from* DependencyAuthority*, including support-intersection, context dependence, pack-seal dependence, and bridge dependence;*
2. *restrict every authority object disjoint from $D(\delta)$ and retain it unchanged;*
3. *invalidate every semantic object in $D(\delta)$, demoting dependent certificates and reopening dependent obligations according to the declared reopening laws;*
4. *replay only those traces whose canonical premises, contexts, support hashes, and imported seals remain equal;*
5. *recompute the remaining local sections and glue them back across the frontier with explicit overlap checks.*

The invalidation rules should be sharp. A change in normalized syntax or exported constructor changes local context support. A change in pack seal invalidates all transported theorems whose certificate paths mention that seal. A change in bridge theorem invalidates both downstream imports and any solver memo compiled from the previous translation. A newly discovered contradiction does not erase prior evidence; it demotes every dependent public projection and records a reopening cause. A proposal cache whose type, support, or seal hash changes is simply discarded. These are not optional engineering preferences but semantic laws.

Proposition 34.2 (Incremental locality)**.** *If δ has support S, then every reopened or invalidated authority object lies in the recorded dependency closure $D(\delta)$, and every authority object outside $D(\delta)$ is preserved by restriction and reused without semantic revalidation.*

Proposition 34.3 (Persistent memory conservativity)**.** *Persistent semantic memory may strengthen by attaching new witnesses, replay traces, or bridge certificates, but it may not preserve a judgment whose typing context, support seal, or dependency certificate has changed. Reuse without transport evidence is unsound.*

The theorem targets are invalidation minimality relative to declared dependencies, replay soundness, support-local gluing under unchanged boundaries, and persistent-memory conservativity. The non-theorems should be explicit as well: this chapter does not claim maximal cache hit rate, semantic reuse from lexical similarity, or cross-epoch validity of proposal memories. It claims that incremental behavior is a support-local restriction-and-gluing semantics with typed invalidation.

Chapter 35

Domain packs, bridge theorems, and federation

A domain pack must be introduced as a local semantic theory over a region or cover, not as an informal plugin. If U is a region of the site together with its chosen cover or hypercover $\mathcal{H}_U$, then a domain pack should be treated as a record

$$\mathfrak{P} = \big(U,\, \mathcal{H}_U,\, \Gamma^{\mathfrak{P}},\, \Sigma^{\mathfrak{P}},\, \mathcal{L}^{\mathfrak{P}},\, \mathcal{T}^{\mathfrak{P}},\, \mathcal{B}^{\mathfrak{P}},\, \mathrm{seal}^{\mathfrak{P}}\big),$$

where $\Gamma^{\mathfrak{P}}$ is the admissible dependent context extension, $\Sigma^{\mathfrak{P}}$ the exported constructors and data surfaces, $\mathcal{L}^{\mathfrak{P}}$ the local theorem family, $\mathcal{T}^{\mathfrak{P}}$ the routing and validation policies, $\mathcal{B}^{\mathfrak{P}}$ the bridge slots it imports or exports, and $\mathrm{seal}^{\mathfrak{P}}$ the stable identity of the pack for replay and federation. In algebraic-geometric terms, a pack is a coefficient theory carried locally on a region of the site. Federation is then descent across overlaps equipped with explicit transport data, not a loose statement that several theories "work together."

The authority centers must be named accordingly: PackAuthority stores pack records and seals, BridgeAuthority stores admitted bridge theorems and their dependency certificates, SealAuthority governs import stability and replay identity, and FederationAuthority computes cross-pack transport and descent obligations. Compiled views should again be demoted: ImportGraphView, BridgeRoutingTable, TheoremLookupIndex, and PackPromptBundle are projections for planning and performance. They are not the semantic source of theorem transport. If this distinction is not drawn sharply, later chapters will smuggle theorem reuse through caches rather than through bridges.

A bridge theorem between packs $\mathfrak{P}$ on U and $\mathfrak{Q}$ on V should be stated as a typed morphism on the overlap region:

$$\beta_{\mathfrak{P}\to\mathfrak{Q}} : \Gamma^{\mathfrak{P}}_{U\cap V} \to \Gamma^{\mathfrak{Q}}_{U\cap V}$$

together with transport on constructors, obligations, and witnesses. Concretely, the bridge record should include context translation, constructor translation, judgment translation, witness translation, residual translation, and a family of descent obligations that certify compatibility with local gluing laws. In DTT terms, bridge transport is not just a map on formulas; it is a dependent map that must preserve typing, premises, and the meaning of exported witnesses. In AG terms, the bridge must preserve the local descent data required to glue transported sections on overlaps. This is why bridge theorems should be first-class deliverables rather than hidden implementation lemmas.

AI may assist only by proposing candidate alignments, searching for relevant prior bridges, or ranking possible route choices for bridge proof obligations. A candidate alignment between constructor families is not a bridge theorem. It becomes one only after the typed transport maps and descent obligations are materialized and validated. The non-theorem should be stated directly:

federation does not grant automatic equivalence between domain packs, and there is no license to transport claims across packs merely because the natural-language descriptions look similar.

Construction 35.1 (Bridge admission algorithm)**.** *To admit a bridge $\beta_{\mathfrak{P}\to\mathfrak{Q}}$:*

1. *normalize the source and target pack seals and reject any transport path with unresolved seal ambiguity;*
2. *compile candidate translations for contexts, constructors, residuals, and witness shapes on the overlap $U \cap V$;*
3. *generate descent obligations stating that transported local sections remain compatible with the target pack's overlap laws and certificate projections;*
4. *discharge these obligations using the ordinary validator family of the target pack rather than a special bridge shortcut;*
5. *register the bridge together with its support scope, imported dependencies, and replay invalidation law.*

Proposition 35.2 (Bridge soundness)**.** *If $\beta_{\mathfrak{P}\to\mathfrak{Q}}$ is admitted, then every transported judgment, residual, and witness remains well-typed in the target context on the declared overlap support, and any transported local section that glued in $\mathfrak{P}$ continues to glue in $\mathfrak{Q}$ up to the explicitly discharged descent obligations.*

Proposition 35.3 (Federation conservativity)**.** *Federation extends the space of admissible transports and reusable theorems, but it does not alter pack-local truth. A theorem proved in $\mathfrak{P}$ remains a theorem of $\mathfrak{P}$ for the same reasons as before; federation only adds typed ways to re-express or reuse it elsewhere.*

The replay and invalidation rules should also be spelled out: changing a pack seal invalidates all imports whose certificate path mentions that seal; changing a bridge theorem invalidates every transported judgment and public projection downstream of that bridge; changing a local pack theorem invalidates only those bridges whose transport proof depended on it. The theorem targets are bridge soundness, descent adequacy, federation conservativity, and reuse locality under seal-preserving replay. The non-theorems are automatic interoperability, unrestricted theorem transport, and global completeness of bridge search.

Chapter 36

Subsystem theorem schemas

The subsystem chapter should not read as a list of aspirational properties. It should impose one theorem schema that every subsystem section is required to instantiate. The right unit is a local-to-global theorem package parameterized by support region, dependent context, authority center, allowed evidence classes, obstruction effect, replay law, and public projection. A usable schema record is

$$\Theta = \big(\text{name}, U, \Gamma_U, \mathsf{Auth}, \text{Premises}, \text{Move}, \text{Conclusion}, \Delta\mathcal{X}_{\mathrm{Star}(U)}, \text{WitnessShape}, \text{ReplayLaw}, \text{PublicProjection}\big).$$

Each subsystem theorem should assert not merely that a local move is sound, but that it transforms local sections in a way that controls overlap obstructions and composes into a global statement after gluing. This is how the foundation, front end, analysis, synthesis, runtime, fleet, and public layers become parts of one theorem machine rather than separate informal stories.

The authority centers for the later sections should therefore be fixed at the chapter opening. The foundation subsystem should be governed by FoundationAuthority for core typing and judgment formation. The front end should be governed by FrontendAuthority for parsing, binding, elaboration, and normalization. The analysis subsystem should be governed by AnalysisAuthority for abstract states, solver clauses, and witness extraction. The synthesis subsystem should be governed by SynthesisAuthority for typed candidate construction, hole inhabitation, and residual management. The runtime and orchestration subsystem should be governed by RuntimeAuthority and ReplayAuthority for traces, probes, and reopening decisions. The fleet and public surface subsystem should be governed by FleetAuthority and CertificateAuthority for treaty exchange, benchmark transport, and public claim rendering. Parser forests, CFGs, solver batches, search frontiers, runtime trace summaries, and dashboards are compiled views attached to these authorities, not substitutes for them.

AG must be explicit in the theorem form. A local subsystem theorem is a claim about a section over U and its effect on the obstruction cocycle on overlaps. A global subsystem theorem is obtained only after proving compatibility on overlaps and, where needed, on higher simplices. In other words, each subsystem section should present both a local soundness theorem and a gluing theorem. DTT must be equally explicit: each theorem is parameterized by a dependent context, names the constructors and residuals it preserves, and states the type of its witnesses. Without this, later claims about interoperability and replay will be impossible to state cleanly.

AI again should be treated as a restricted contributor to subsystem theorems. In synthesis it may propose typed inhabitants for holes. In analysis it may rank decomposition or routing candidates. In runtime it may route probes or search historical traces. But every such role belongs under Move as a proposal or search step whose output is then validated by the subsystem's ordinary rules. No

subsystem theorem should have "AI judged that the property seems true" as a premise. The chapter should mark that as a non-theorem once and for all.

Construction 36.1 (Subsystem theorem compiler)**.** *For each subsystem authority* Auth*:*

1. *choose the local support region U and the dependent context Γ_U on which the subsystem actually operates;*
2. *state the admissible move family and the local witness shape it produces;*
3. *prove a local preservation or progress theorem in that context;*
4. *prove an overlap theorem describing how the move changes, preserves, or exposes obstruction data on* $\mathrm{Star}(U)$*;*
5. *state the replay and invalidation law under which the theorem remains reusable;*
6. *define the public projection and its honesty condition.*

Proposition 36.2 (Local-to-global subsystem composition)**.** *Suppose a cover $\{U_i\}$ is equipped with subsystem theorem schemas Θ_i whose conclusions agree on overlaps up to their declared bridge and treaty obligations. Then the family $\{\Theta_i\}$ composes to a global subsystem theorem, and the resulting global obstruction state is exactly the glued image of the local obstruction deltas $\Delta\mathcal{X}_{\mathrm{Star}(U_i)}$ together with any explicitly retained residual cocycle classes.*

Proposition 36.3 (Obstruction honesty)**.** *Every subsystem theorem must classify its effect on obstruction data as elimination, preservation, transport, exposure, or demotion. Silent absorption of obstruction classes is not an admissible theorem form.*

The theorem targets are therefore local preservation, overlap compatibility, gluing soundness, replay conservativity, obstruction honesty, and public projection honesty. The non-theorems are equally important: the chapter does not promise globally optimal parsing, best possible analyses, complete synthesis, optimal orchestration, or the wisdom of fleet heuristics. It promises only that each subsystem exports theorems in a uniform local-to-global form with explicit obstruction control.

Chapter 37

Implementation-complete thesis doctrine

The dissertation should adopt an implementation-complete doctrine: no theoretical object counts as central unless the text specifies how it compiles into an authority center, what transitions create and mutate it, how it is invalidated and replayed, how it is projected publicly, and what theorem and non-theorem accompany that compilation. The doctrine can be stated as a compilation contract

$$\mathrm{Comp}(T) = \big(\mathrm{carrier}(T),\ \mathrm{authority}(T),\ \mathrm{constructors}(T),\ \mathrm{validators}(T),\ \mathrm{compiledViews}(T),\ \mathrm{invalidation}(T),\ \mathrm{replay}($$

A chapter is implementation-complete only if every object it treats as semantically primary is assigned such a contract. This doctrine is what prevents the dissertation from introducing beautiful notions that never become executable semantic authority.

The AG+DTT+AI center of the thesis should be expressed through this contract rather than through slogans. Algebraic-geometric objects such as sites, covers, local sections, treaties, cocycles, and obstruction classes must compile to stored carriers with support and overlap laws. Dependent-type-theoretic objects such as contexts, constructors, judgments, residual obligations, and witness terms must compile to normalized records with validators and replay conditions. AI objects must compile only to proposal, search, or routing records with expected types and validator families. The implementation should therefore name, at minimum, SiteAuthority, ContextAuthority, JudgmentAuthority, ObstructionAuthority, MemoryAuthority, PackAuthority, BridgeAuthority, ReplayAuthority, CertificateAuthority, and AIRoutingAuthority. These are the centers to which later code modules should answer.

The doctrine must also distinguish authority centers from compiled views in a chapter-independent way. Parser outputs, normalized IRs, solver clause bundles, retrieval indices, benchmark summaries, dashboards, and rendered reports are legitimate and often necessary, but only as compiled views generated from authority centers under stable serialization. Every such view must declare its dependency tokens and invalidation law. The thesis should be explicit that a compiled view may be thrown away and regenerated without semantic loss, whereas an authority center carries the durable object of record. This distinction is what makes replay and invalidation tractable across the whole dissertation.

A universal replay and invalidation discipline follows immediately. If an authority center changes, every compiled view and downstream certificate whose dependency path mentions that authority must be invalidated or replayed according to its declared law. If only a compiled view changes, no semantic reopening occurs. The doctrine should insist that every chapter name this law rather than postpone it to implementation. Otherwise the dissertation will repeatedly reintroduce hidden semantic state under the name of optimization.

AI must be bounded here with unusual clarity because this chapter sets doctrine. Typed proposal, search, and routing are admissible compiled services. Semantic acceptance, theorem transport, and public certification are not AI roles. An AI suggestion becomes relevant only after it has been compiled into an ordinary typed candidate and passed through the same validators, bridge rules, and replay laws as any other proposal. This should be stated as a thesis-wide non-theorem, not negotiated chapter by chapter.

Construction 37.1 (Implementation-completeness checklist)**.** *For every chapter-central object T:*

1. *specify its mathematical carrier and support semantics;*
2. *assign one authority center responsible for durable storage and mutation;*
3. *give its typed constructors and admissible validators;*
4. *enumerate the compiled views that may be derived from it;*
5. *state its invalidation and replay law under support, context, and seal change;*
6. *define its certificate or public projection form;*
7. *name the theorem target and the corresponding non-theorem.*

Proposition 37.2 (Implementation-complete adequacy)**.** *If every central object in a chapter satisfies the compilation contract above, then the chapter is implementation-complete in the precise sense that its theoretical claims can be represented, updated, invalidated, replayed, and publicly rendered without introducing undeclared semantic authorities.*

Proposition 37.3 (Compiled-view conservativity)**.** *A compiled view may accelerate search, routing, rendering, or benchmarking, but it may not add semantic commitments beyond those already present in its source authorities. Any public claim justified only by a compiled view is doctrinally invalid.*

The theorem targets for the doctrine chapter are implementation-complete adequacy, compiled-view conservativity, replay adequacy, public honesty, and chapter-wise contract coverage. The non-theorems should be equally sharp: the doctrine does not require every concept to receive its final optimized implementation now, nor does it require every auxiliary convenience structure to be elevated into an authority center. It requires only that every semantically central object in the dissertation compile into one.

Part VI

Large-Scale Code Generation

Chapter 38

Semantic decomposition and cover design

A large-scale generation system should begin by choosing a semantic cover, not by choosing a folder hierarchy. The question is which regions can carry local judgment formation with manageable overlap law, manageable replay, and informative obstruction localization. In algebraic-geometric language, decomposition chooses the site on which the construction problem will be posed. A bad site hides the true support of failure, forces spurious reopening, and makes every later treaty look ad hoc. A good site makes local sections easy to construct, overlap laws explicit, and nontrivial cocycles visible at the right simplicial level.

The implementation consequence is that cover design must itself be a first-class algorithm with persistent records, scores, and falsification traces. The system should not hard-code "module" or "file" as the only decomposition units. It should admit region constructors for files, APIs, effect basins, data-shape clusters, runtime surfaces, documentation shadows, and donor fragments, and it should score candidate covers by the judgment geometry they induce rather than by textual neatness.

38.1 Semantic decomposition criteria: cover quality, obligation distribution, and overlap minimization

A semantic cover should be judged by whether it makes the right local-to-global problem visible. Let $\mathfrak{U} = \{U_i \to X\}$ be a candidate cover of the project state X. The correct objective is not to minimize overlap absolutely, because zero overlap often means that shared law has been hidden rather than eliminated. The objective is instead to create overlaps that are semantically meaningful, low-entropy, and stable under replay. A good cover exposes the interfaces on which global coherence actually depends. A bad cover creates either overlap explosion or the illusion that there are no overlaps until integration fails.

A useful evaluation record is

$$\mathfrak{q}(\mathfrak{U}) = \big(A(\mathfrak{U}), E_{\mathrm{ov}}(\mathfrak{U}), R_{\mathrm{reopen}}(\mathfrak{U}), M_{\mathrm{obs}}(\mathfrak{U}), D_{\mathrm{donor}}(\mathfrak{U}), C_{\mathrm{plan}}(\mathfrak{U})\big),$$

where A measures authority-center clarity, E_{ov} overlap entropy, R_{reopen} predicted reopening radius under local edits, M_{obs} predicted obstruction localizability, D_{donor} donor transport quality, and C_{plan} decomposition cost. The planner should rank covers by a phase-dependent functional such as

$$\mathrm{Score}(\mathfrak{U}) = \alpha A - \beta E_{\mathrm{ov}} - \gamma R_{\mathrm{reopen}} + \delta M_{\mathrm{obs}} + \varepsilon D_{\mathrm{donor}} - \zeta C_{\mathrm{plan}}.$$

The point is not a final universal formula. The point is to force the system to say what geometric virtues a cover is expected to provide.

The overlap entropy term should be explicit. If an overlap $U_i \times_X U_j$ carries many unstable clauses, hidden guards, or mixed evidence routes, then its entropy is high. High entropy is not always bad at the beginning of construction, but it is a strong predictor of treaty debt and replay fragility later. The planner should therefore record whether entropy comes from genuine shared semantics or from poor region choice. Only the first justifies keeping the overlap large.

Definition 38.1 (Cover quality dossier)**.** *For each candidate cover $\mathfrak{U}$, a cover quality dossier is a record*

$$\mathfrak{d}_{\mathfrak{U}} = \big(\mathfrak{U}, \{\Gamma_i\}, \{\Lambda_i\}, \{\Omega_i\}, \{\partial_{ij}\}, \mathfrak{q}(\mathfrak{U}), \Pi_{\mathfrak{U}}\big),$$

where Γ_i are local contexts, Λ_i local clause families, Ω_i routed obligation bundles, ∂_{ij} overlap summaries, and $\Pi_{\mathfrak{U}}$ the evidence explaining why the cover was proposed and retained.

The design algorithm should be concrete.

1. Normalize the initial project obligations into clause families with supports, authority centers, and likely evidence routes.
2. Cluster clauses by authority center and by required shared law rather than by file adjacency alone.
3. Propose base regions whose internal obligations have high mutual dependence and whose exported clauses are small enough to stabilize.
4. Materialize pairwise overlaps from shared exported clauses, shared state, shared effects, shared data-shape invariants, and shared documentation or benchmark surfaces.
5. Estimate obstruction visibility by asking which recurrent failures would become 1-cochains on this cover rather than diffuse noise.
6. Reject covers that merely defer shared law into integration, even if they minimize immediate overlap count.

The failure modes are equally important. The first is *pseudo-modularity*: regions look separate syntactically but are coupled by hidden mutation, global registries, or undocumented normalization conventions. The second is *overlap laundering*: a planner lowers measured overlap by omitting exported law and forcing downstream repair to rediscover it. The third is *reopening blow-up*: a local edit causes nearly global invalidation because the cover ignored the true support of shared semantics. The fourth is *donor misprojection*: reused fragments appear convenient textually but carry incompatible overlap law, so they inject obstruction mass rather than reducing it.

Proposition 38.2 (Cover honesty target)**.** *A retained cover is semantically honest only if every public integration or decomposition claim is accompanied by a dossier $\mathfrak{d}_{\mathfrak{U}}$ whose predicted overlap and reopening behavior can later be compared against realized treaty debt and replay traces.*

38.2 Initial cover synthesis: obligation bundles, authority centers, and decomposition cost

Initial cover synthesis should begin from obligation bundles, not code volume. The planner is not partitioning text; it is partitioning judgment burden. Each region should own the obligations for

which it has the narrowest and most faithful authority center, while shared obligations should be placed on overlaps from the beginning rather than being falsely assigned to one side and rediscovered later as "integration work."

A useful region record is

$$\kappa_u = \big(u, \Gamma_u, \Lambda_u, \Omega_u, \partial u, a_u, c_u, r_u\big),$$

where u is the region identifier, Γ_u its dependent local context, Λ_u its local law family, Ω_u its unresolved obligation bundle, ∂u its exported overlap boundary, a_u its authority-center summary, c_u its decomposition cost, and r_u its predicted replay radius. These records should be built before any large construction loop begins, because later orchestration depends on them.

The synthesis algorithm should be staged.

1. Form an obligation hypergraph whose nodes are normalized clauses and whose hyperedges connect clauses sharing support, data invariants, effect summaries, or public-surface claims.
2. Annotate each node with an authority center estimate: which region, pack, or interface is naturally responsible for discharging or exporting the clause.
3. Seed provisional regions around high-authority clusters.
4. Promote clauses with competing authority centers to overlap obligations rather than forcing premature ownership.
5. Estimate decomposition cost from expected treaty count, expected replay fanout, donor translation burden, and predicted higher-overlap load.
6. Select the lowest-cost family whose overlaps remain semantically legible.

This process should also admit cover refinement pressure. If later replay reveals that a region owns too many unrelated obligations, then the original bundling was wrong. If later integration reveals that most failures arise on one specific overlap, then the base regions may be correct but the overlap requires finer internal structure or higher simplices. Initial cover synthesis is therefore the first step of a continuing geometric adjustment process, not a one-shot planning ritual.

A practical decomposition objective is to minimize

$$\mathcal{L}_{\text{cover}} = \sum_u c_u + \lambda_1 \sum_{u \neq v} \text{Entropy}(\partial_{uv}) + \lambda_2 \sum_u r_u + \lambda_3 \,\text{Debt}_{\text{treaty}} - \lambda_4 \,\text{LocalizationGain}\,.$$

The last term matters. A cover that exposes likely obstruction supports may be worth a higher immediate planning cost because it reduces later blind search.

The theorem burden here is not full optimality. It is decomposition conservativity and support honesty. Conservativity says that reorganizing obligation ownership may not silently discharge or erase obligations. Support honesty says that if a clause is assigned to a region or an overlap, the assignment must be traceable to the normalized support analysis rather than to convenience. Those are implementable theorems because they range over explicit records.

38.3 Module boundaries: overlap quality, interface explicitness, and donor reuse

A module boundary should be treated as a deliberate overlap design decision. The important question is not how little two modules know about each other in prose, but what law must live on

their overlap for the global artifact to glue. In categorical terms, the boundary is valuable only if the induced restriction maps and overlap law are compact, explicit, and replay-stable. When boundaries are chosen this way, interfaces become descent data rather than comments around import statements.

A boundary dossier should therefore record

$$\mathfrak{b}_{uv} = \big(u, v, \Gamma_{uv}, \Phi_{uv}^{\mathrm{exp}}, \Phi_{uv}^{\mathrm{hid}}, \tau_{uv}^{\mathrm{cand}}, \chi_{uv}\big),$$

where Γ_{uv} is the overlap context, Φ_{uv}^{exp} the exported law family, Φ_{uv}^{hid} the suspected hidden law family, $\tau_{uv}^{\mathrm{cand}}$ candidate treaties, and χ_{uv} the challenge history. A boundary is good when Φ_{uv}^{hid} stays small, the treaty family stabilizes quickly, and challenges reveal local rather than diffuse failure modes.

Donor reuse belongs to the same geometry. A donor fragment is not reusable merely because it solves a similar local problem. It is reusable when its exported law can be transported into the new overlap structure with bounded treaty debt. The reuse planner should therefore compute a donor transport record

$$\mathfrak{t}_{\mathrm{donor}} = \big(d, u, \psi, \mathcal{P}_{\mathrm{pres}}, \mathcal{X}_{\mathrm{risk}}, C_{\mathrm{bridge}}\big),$$

where d is the donor region, u the target region, ψ a proposed transport map, $\mathcal{P}_{\mathrm{pres}}$ explicit preservation claims, $\mathcal{X}_{\mathrm{risk}}$ predicted obstruction risks, and C_{bridge} bridge cost. If the preservation claims are weak or the overlap law changes drastically, reauthoring is often cheaper than transport.

Three failure modes recur at boundaries. The first is *hidden invariant traffic*: important laws are used across the boundary but are not exported, so repeated repair recreates them as folklore. The second is *guard leakage*: laws are exported without the context parameters and effect guards on which they actually depend. The third is *replay asymmetry*: one side changes often while the other is treated as stable, causing repeated support-nonlocal reopening. These are exactly the signs that the boundary dossier should expose.

Proposition 38.3 (Boundary admissibility target)**.** *A module boundary is admissible only if its exported clauses suffice to reconstruct the accepted treaty family on the induced overlap up to recorded guards and replay policy. Repeated reliance on hidden clauses is evidence that the boundary is geometrically misdrawn.*

Chapter 39

Local construction loops, interface discipline, and coordinated elaboration

Once a semantic cover exists, generation proceeds through local construction loops that elaborate partial sections region by region while exporting the information necessary for later gluing. The point of a local loop is not simply to produce code inside one region. It is to move the project from a weaker 0-cochain to a stronger one while simultaneously refining the 1-cochain of overlap law. Every accepted local move must therefore leave behind semantic compression artifacts that other regions, the orchestrator, and later replay can actually use.

39.1 Local construction loops: proposal, checking, semantic compression, and export

A local construction loop should operate on an explicit goal record

$$g_u = \big(u, \Gamma_u, \Lambda_u, \Sigma_u, \Omega_u, \mathcal{T}_{\partial u}, \mu_u\big),$$

where u is the target region, Γ_u the local context, Λ_u the target law family, Σ_u the current local partial section, Ω_u the unresolved obligations, $\mathcal{T}_{\partial u}$ the active overlap treaties touching the boundary, and μ_u the admissible move family. The loop should never operate directly on raw text alone; text is only one surface realization of a proposed section update.

Each iteration should emit a semantic compression record

$$\chi_u = \big(\Delta\mathcal{S}_u, \Delta\mathcal{O}_u, \Delta\mathcal{E}_u, \Delta\mathcal{X}_u, \Delta\mathcal{K}_u, \mathrm{supp}(\Delta_u)\big),$$

where the Δ terms describe section change, obligation change, evidence change, obstruction change, and certificate change respectively. This record is the memory of the loop. Without it, the system knows only that something changed. With it, the system knows what semantic capital was gained, what residuals remain, and what neighboring overlaps must be reopened.

The four-phase loop should be explicit.

1. **Proposal.** Produce one or more candidate local sections by local synthesis, donor transport, normalization, or theorem instantiation.

2. **Checking.** Validate the candidate against local obligations, overlap treaties, routed evidence channels, and support guards.

3. **Semantic compression.** Summarize the checked result into judgments, residuals, obstructions, and public-surface deltas.

4. **Export.** Push only the compressed overlap-relevant consequences to neighboring regions, frontier state, and archive memory.

The export step is where algebraic geometry becomes operational. A local success is not "done" when the local file looks correct. It is done when the restriction of the new section to each relevant overlap has been computed and attached to the corresponding overlap object. A local failure is equally informative when its support and witness structure show that the region cannot be completed without treaty change, hypercover refinement, or coefficient change.

The theorem burden should include *local export conservativity*: semantic compression may forget raw operational detail, but it may not forget overlap-relevant law, open obligations, or evidence provenance. One also wants *support-local reopening*: if two local loop updates are support-disjoint modulo recorded higher overlaps, then accepting one may not silently invalidate the other's stabilized summaries.

39.2 Interface discipline: overlap objects, provisional treaties, and stabilization rules

Interface discipline should be expressed through overlap objects rather than through unstructured API summaries. If u and v are neighboring regions, then the overlap object should have form

$$\omega_{uv} = \big(u, v, \Gamma_{uv}, \Lambda_{uv}^{\mathrm{exp}}, \Theta_{uv}, \tau_{uv}, \Xi_{uv}, \nu_{uv}\big),$$

where Γ_{uv} is the shared context, $\Lambda_{uv}^{\mathrm{exp}}$ the exported clause family under negotiation, Θ_{uv} the active guards and parameters, τ_{uv} the current treaty status, Ξ_{uv} the challenge and witness history, and ν_{uv} the replay policy. This object is the true interface authority center.

A treaty should begin provisional when either the guards are still moving, the witness package is incomplete, or neighboring regions have not stabilized enough to justify replay commitments. Provisionality is not a social status. It is a semantic mode. The overlap object should therefore carry an explicit stabilization state

$$\tau_{uv}.\mathit{status} \in \{\mathrm{proposed}, \mathrm{challenged}, \mathrm{provisional}, \mathrm{stabilized}, \mathrm{reopened}\}.$$

Once stabilized, the treaty becomes a law on the overlap, and future local elaboration must either preserve it, strengthen it by explicit guard refinement, or reopen it with a named counter-witness.

This is the correct sense in which treaty formation is overlap-law stabilization. What begins as repeated local friction is abstracted into a candidate law on $U_u \times_X U_v$. Challenge loops test whether that law is real, guarded correctly, and transportable across replay. Stabilization is therefore the act of converting empirical overlap regularity into explicit descent data.

Typical failure modes should be named in the chapter. *Oscillating treaty law* occurs when the overlap appears stable only because local representatives keep changing. *Guard erasure* occurs when a law is exported without the assumptions that make it true. *Summary underfitting* occurs when the overlap object compresses away distinctions needed by later challenge or replay. These are not minor engineering mistakes; they are failures of interface semantics.

Proposition 39.1 (Treaty stabilization target)**.** *If an overlap treaty is marked stabilized, then any later local section whose support is disjoint from the treaty's invalidation support may be replayed without changing the treaty status. If this condition fails repeatedly, the treaty was never semantically stabilized; it was only operationally convenient.*

39.3 Coordination with semantic accounting: bids, risk, and expected global delta

Parallel elaboration should be coordinated by semantic accounting. A worker should not ask merely for permission to edit a region. It should submit a bid describing an expected global semantic delta. A useful bid record is

$$b = \big(w, m, \mathrm{supp}(b), \widehat{\Delta\mathcal{S}}, \widehat{\Delta\mathcal{O}}, \widehat{\Delta\mathcal{T}}, \widehat{\Delta\mathcal{E}}, r_b, c_b\big),$$

where w is the worker identity or strategy class, m the proposed move, $\mathrm{supp}(b)$ the targeted support region, the $\widehat{\Delta}$ terms are predicted semantic deltas, r_b is overlap and replay risk, and c_b is projected cost.

Acceptance should be based on expected global leverage rather than local busyness:

$$\mathrm{Value}(b) = \alpha\,\mathbb{E}[\Delta_{\mathrm{closure}}] + \beta\,\mathbb{E}[\Delta_{\mathrm{stability}}] + \gamma\,\mathbb{E}[\Delta_{\mathrm{treaty}}] - \delta\,\mathbb{E}[\Delta_{\mathrm{reopen}}] - \varepsilon c_b.$$

The terms should be conditioned on project phase and support region. Early in a project, a bid that raises overlap clarity may be worth more than a bid that closes one local obligation. Late in a project, replay fragility may dominate everything else.

Challenges should target the semantic prediction, not the worker. A challenge may assert that the claimed section delta is ill-scoped, that the obligation delta hides new residuals, that the treaty delta depends on unexported guards, or that the evidence delta exceeds what the cited channel can warrant. This makes worker competition mathematically legible. The system is choosing among predicted semantic futures and counterarguments, not among personalities or styles.

The theorem target here is *bid-accounting honesty*: after validation, every accepted bid must be comparable to its realized compression record. That comparison is how bidder calibration becomes a scientific object rather than intuition. Without it, a fleet architecture will reward charisma or verbosity instead of semantic reliability.

Chapter 40

Integration, regression, and semantic closure

Integration, regression, and closure are three views of one local-to-global problem. Integration asks whether the present family of local sections descends. Regression asks which previously accepted descent witnesses survive a support-local change. Closure asks whether the resulting global section is stable enough, explicit enough, and honest enough to count as a completed semantic state. These notions should not be separated into build, test, and release folklore. They should be stated as properties of the same gluing object under replay.

40.1 Regression as semantic memory: retained judgments, changed supports, and replay

Regression should be treated as semantic replay against retained judgments, not as indiscriminate rerunning of past checks. The system should preserve a regression memory record

$$\mathfrak{r} = \big(\lambda, \operatorname{supp}(\lambda), W_\lambda, \nu_\lambda, \Pi_\lambda, \sigma_\lambda\big),$$

where λ is a retained judgment or treaty claim, $\operatorname{supp}(\lambda)$ its support, W_λ its witness package, ν_λ its replay policy, Π_λ its provenance trace, and σ_λ its current replay status. A regression event is then not "run the suite again," but "re-evaluate precisely those retained claims whose support intersects the closure of the changed star under the current hypercover."

The replay algorithm should be support-aware.

1. Normalize the change event δ into a support region and a semantic compression delta.
2. Compute the potentially affected star $\operatorname{Star}_{\mathcal{H}}(\operatorname{supp}(\delta))$ and extend only by declared nonlocal dependency edges.
3. Select retained judgments λ whose support intersects this region or whose replay policy explicitly depends on changed treaties or bridge theorems.
4. Revalidate those judgments using their original witness schema whenever possible.
5. Promote failures to typed obstructions naming the support, witness mismatch, and likely repair basin.

This makes regression a form of memory-preserving descent checking. Previously accepted gluing witnesses, treaty witnesses, and local proof artifacts are not dead history. They are reusable cochains whose validity is conditional on support, guards, and evidence channel. The system should therefore remember not only *that* something passed, but what kind of local-to-global compatibility that pass established.

Failure modes are instructive. *Stale witness success* occurs when a claim appears to replay only because the system no longer checks the exact guard family or overlap law under which it was accepted. *Support drift* occurs when the real dependency region has widened but the stored support has not been updated. *Treaty drift* occurs when neighboring law changed but old regression witnesses still cite the old treaty version implicitly. A serious regression chapter should force all three to be impossible or at least explicitly reported.

Proposition 40.1 (Replay-locality target)**.** *Let δ be a local change. Any retained judgment whose support is disjoint from the declared replay region of δ and from all declared nonlocal dependencies must preserve its replay status. A regression system that violates this condition is not support-aware; it is only rechecking by habit.*

40.2 Semantic closure: completion criteria, trust profile, and reopening rules

Semantic closure should be defined as an honest global section together with an honest account of what could still reopen it. A closure record should therefore have form

$$\mathfrak{c} = \big(s, W_{\mathrm{glue}}, \mathcal{X}_{\mathrm{res}}, \mathcal{F}_{\mathrm{frag}}, \kappa_{\mathrm{trust}}, \nu_{\mathrm{reopen}}, \Pi_{\mathrm{close}}\big),$$

where s is the integrated global section, W_{glue} the gluing witness family, $\mathcal{X}_{\mathrm{res}}$ any residual obstruction family, $\mathcal{F}_{\mathrm{frag}}$ fragility regions, κ_{trust} the trust profile partitioned by evidence class, ν_{reopen} the reopening law, and Π_{close} the closure provenance trace. Closure is therefore not binary. It is a typed semantic report.

A reasonable completion doctrine should require at least the following.

1. All public-surface obligations have either been discharged or residualized explicitly.
2. Every stabilized treaty names its guards, witness class, and invalidation support.
3. Every retained obstruction is localized and classified as removable by local repair, cover refinement, treaty synthesis, or theory growth.
4. Reopening policy is support-sensitive rather than global by default.
5. The trust profile distinguishes structural proof, runtime witness, semantic judgment, and human ratification rather than aggregating them into one score.

The gluing perspective clarifies why this matters. A project may have a valid H^0-like assembly for the current cover while still carrying concentrated fragility on specific overlaps. In that case the correct public report is not "fully solved" but "globally glued with specified fragility support and replay contingencies." Closure without fragility honesty is an overclaim. Conversely, localized residual obstruction is not failure if it is precisely recorded and its repair basin is known.

The theorem burden should include *closure conservativity*, *fragility honesty*, and *reopening monotonicity*. Closure conservativity says that adding new witnesses may strengthen trust or shrink

residuals, but it may not silently erase existing residuals or fragility regions. Reopening monotonicity says that if a support region is declared outside the invalidation closure of a change, then it may not reopen without a newly recorded dependency. These are the rules that keep completion claims from turning into motivational prose.

Chapter 41

Large-scale generation as a judgement-geometry-type-theoretic problem

Large-scale code generation should be stated as a section-construction problem over a changing semantic site, not as a problem of extending a single token stream. The relevant object is the project judgment state: local contexts over a cover, overlap laws on intersections, higher compatibility obligations on multiple overlaps, evidence objects of mixed provenance, and an archive recording how all of these were proposed, challenged, accepted, or reopened. Once this is made explicit, generation, repair, integration, and theorem growth become different typed moves on the same state space.

This point matters because project-scale construction is governed simultaneously by algebraic geometry, dependent type theory, and controlled AI. The algebraic-geometric contribution is not merely the words "locality" and "overlap." It is the claim that generation proceeds by constructing local sections over a hypercover, that interface and treaty data form descent data on overlaps, that successful construction is a passage from local cochains to a global section, and that persistent generation failure is measured by nontrivial obstruction classes. The dependent-type-theoretic contribution is that every local move is parameterized by context, assumptions, and evidence class, so there is no such thing as an untyped patch. The AI contribution is that proposal pressure, analogy, and search over admissible futures become necessary once the local search space is too large for hand-authored tactics alone. JuGeo should therefore be written as one semantic machine whose large-scale behavior is expressed in this common language.

The implementation consequence is immediate. The code should have authority centers for cover objects, overlap objects, treaty objects, goal objects, candidate objects, routing judgments, replay traces, and archive records. A large-codebase orchestrator that lacks these objects will necessarily fall back to prose coordination and ad hoc prompts. That is precisely what this chapter should forbid.

41.1 Generation as section construction: state space, dependent moves, and gluing criteria

The right abstract state for large-scale generation is not a file tree plus a task queue, but a typed tuple

$$\mathcal{G} = (\mathcal{U}, \mathcal{H}_\bullet, \mathcal{C}, \mathcal{S}, \mathcal{T}, \mathcal{O}, \mathcal{E}, \mathcal{R}, \mathcal{A}),$$

where $\mathcal{U}$ is the chosen base cover of the project, $\mathcal{H}_\bullet$ a hypercover refinement recording higher overlaps, $\mathcal{C}$ the family of dependent local contexts, $\mathcal{S}$ the current partial sections, $\mathcal{T}$ the treaty family on overlaps, $\mathcal{O}$ the unresolved obligations, $\mathcal{E}$ the accumulated evidence objects, $\mathcal{R}$ the routing state for obligations, and $\mathcal{A}$ the semantic archive. This state is the real object of construction. Source files, prompts, test harnesses, and proof scripts are projections of it.

This state should be read cohomologically. $\mathcal{S}$ is a family of local 0-cochains; $\mathcal{T}$ and neighboring compatibility objects live on 1-simplices and higher simplices; gluing attempts compute whether the present descent data lands in a genuine global section; and the unresolved-obligation state records where the current local family has not yet achieved descent. A repair or theorem-growth move should therefore be understood as either altering the current representative cocycle, refining the hypercover, or changing the coefficient theory in which the obstruction is being measured.

The move language should likewise be explicit. A useful core grammar is

$m ::=$ refineCover | spawnGoal | inhabit | proposeTreaty | challenge | glue | replay | reopen | route | archive.

Each move should be interpreted as a dependent transition

$$(\mathcal{G}, m) \rightsquigarrow \mathcal{G}'$$

with a declared support region, precondition on local context, admissible evidence classes, expected semantic delta, and invalidation law. This is how local elaboration, interface law discovery, solver discharge, runtime probing, and LLM proposal can be compared inside one semantics rather than as unrelated workflow steps.

The gluing criterion should also be stated at the project level. A family of local sections $\{s_u\}_{u \in \mathcal{U}}$ is not enough. One needs treaty-mediated compatibility on overlaps, replay-stable witness attachment, and a controlled reopening law under later edits. In practical terms, the system should only claim a global construction when it can produce a gluing report that names the local sections used, the treaties appealed to, the unresolved residuals if any, the evidence classes behind each compatibility claim, and the support region on which future replay may invalidate the assembly.

Equivalently, the system should be able to say when the current descent cocycle is exact. If the overlap discrepancy lives in the image of a coboundary operator after admissible local change, then local revision suffices. If not, the engine should expose the nontrivial class rather than endlessly proposing more local patches. That is the mathematically disciplined route from generation into repair, cover refinement, and mathematical ideation.

One wants theorem targets that match this account. There should be a move-admissibility theorem saying that accepted moves preserve the invariants of the global transition system; a gluing-soundness theorem saying that a produced gluing report really induces a global project section up to the declared treaty relations; and a replay-locality theorem saying that support-disjoint changes cannot force undeclared reopening outside the recorded dependency closure. These are exactly the theorem schemas that keep large-scale generation from degenerating into a narrative of good intentions.

41.2 The core state space for generation

A useful abstract state is

$$\mathcal{G} = (\mathcal{U}, \mathcal{C}, \mathcal{S}, \mathcal{T}, \mathcal{O}, \mathcal{E}, \mathcal{A}),$$

where $\mathcal{U}$ is the cover, $\mathcal{C}$ the dependent local contexts, $\mathcal{S}$ the partial sections, $\mathcal{T}$ the treaty family, $\mathcal{O}$ the open obligations, $\mathcal{E}$ the accumulated evidence, and $\mathcal{A}$ the archive of proposals, challenges, and semantic decisions. The code should have authority centers for each of these rather than scattering them across orchestration helpers.

41.3 Generation moves as dependent transitions

A generation move should be represented as

$$(\mathcal{G}, m) \rightsquigarrow \mathcal{G}'$$

with explicit preconditions, support scope, expected semantic delta, and replay rules. This formalization is what lets the orchestrator compare local elaboration, treaty refinement, theorem growth, reopening, and bridge discovery inside one semantic space.

41.4 Implementation consequences

The code path implied by the generation chapters is already explicit enough to name. The runtime should own generation-state records, cover and overlap objects, treaty stores, candidate-inhabitant records, semantic-compression outputs, gluing reports, and archive entries linking accepted moves to later replay behavior. These are not convenience abstractions. They are the minimum authority centers required if large-scale generation is to remain a judgment-geometry problem rather than collapsing back into prompt choreography.

Chapter 42

Hypercover synthesis, dependent treaty formation, and overlap law discovery

Base covers are only the first approximation to project geometry. Many persistent failures do not live on pairwise overlaps alone. They live on triples, on API-plus-runtime-plus-documentation simplices, on proof-plus-benchmark-plus-implementation simplices, or on other higher interaction surfaces. Hypercover synthesis and treaty formation are therefore not optional refinements. They are how the system learns where shared law really lives and how that law should stabilize into reusable descent data.

42.1 Overlap law discovery: friction mining, treaty proposals, and challenge loops

Overlap law discovery should begin from friction records rather than from aesthetic desires for abstraction. If a simplex σ in the current hypercover repeatedly accumulates challenge, replay churn, or duplicated repair, then the system should ask whether a latent law on σ is missing. A useful friction record is

$$f_\sigma = \big(\sigma, \Gamma_\sigma, \mathcal{W}_\sigma, \mathcal{X}_\sigma, \Pi_\sigma, h_\sigma\big),$$

where Γ_σ is the simplex context, $\mathcal{W}_\sigma$ the witness family extracted from successful and failed interactions, $\mathcal{X}_\sigma$ the obstruction family, Π_σ the negotiation and replay history, and h_σ summary statistics such as frequency, entropy, and support concentration.

The law-discovery loop should be explicit.

1. Mine recurrent friction motifs from overlap and higher-overlap histories.
2. Normalize those motifs into candidate clause schemas with explicit guards and parameters.
3. Propose treaty candidates or higher-simplex laws that would explain the recurrent patterns.
4. Challenge those proposals by replaying them against held-out successful and failing cases, neighboring simplices, and donor or benchmark shadows.
5. Promote only those proposals that reduce overlap entropy and reopening radius without overclaiming their support.

This is the right place to state that treaty formation is stabilized overlap law discovery. A treaty is not merely a negotiated interface sentence. It is a law discovered from repeated local interaction and then tested strongly enough to become part of the gluing relation. The system should therefore archive both pre-treaty friction and post-treaty stabilization behavior. Otherwise it cannot tell whether a treaty truly simplified the geometry or merely renamed the old trouble.

A candidate law on a simplex σ should be scored by

$$\mathrm{Gain}(\phi_\sigma) = \alpha \Delta H_{\mathrm{ov}} + \beta \Delta R_{\mathrm{reopen}} + \gamma \Delta C_{\mathrm{glue}} - \delta C_{\mathrm{challenge}} - \varepsilon R_{\mathrm{overclaim}},$$

where the Δ terms measure entropy reduction, reopening reduction, and gluing simplification. This makes overlap law discovery implementation-guiding: a law matters if it changes the future geometry, not if it is rhetorically satisfying.

The theorem burden should include *law-discovery traceability*: every promoted treaty or overlap law must be traceable to the friction motifs and challenge results that justified it. One also wants *guard completeness*: a promoted law may not omit guards that were necessary in the motivating data. Finally, one wants *challenge sufficiency*: a law is not stabilized until it has survived the challenge families declared relevant by its support and intended replay scope.

42.2 Implementation consequences

The implementation path should follow directly from the mathematics. The code should contain persistent records for simplices, friction histories, treaty proposals, challenge sessions, promoted overlap laws, and invalidation supports. A minimal authority layer would include:

1. `HypercoverCell` records with face maps, support projections, and current law families;
2. `FrictionRecord` objects keyed by simplex and witness class;
3. `TreatyProposal` objects carrying guards, law text, witness package, predicted gain, and replay scope;
4. `ChallengeSession` objects with counterexamples, rebuttals, and resolution status;
5. `TreatyLedger` objects recording promotion, reopening, strengthening, and retirement.

Without such records, higher-overlap reasoning will collapse back into pairwise folklore. With them, the system can ask implementation-level questions that mirror the dissertation: where does obstruction mass concentrate, which simplices repeatedly generate law, which treaties reduced replay churn, and which promoted laws later decayed. Those are the questions from which an actual hypercover-aware generator can be built.

Chapter 43

Local inhabitant synthesis, AI fleets, and semantic backpressure

Local inhabitant synthesis is the constructive heart of generation. A project-scale system succeeds only if it can turn localized goals into inhabitants quickly, compare multiple proposed inhabitants semantically rather than stylistically, and regulate proposal pressure when the overlap structure is becoming unstable. AI assistance matters here, but only as controlled proposal inside a typed local loop whose outputs are judged by gluing consequences, not by fluency.

43.1 Local inhabitant synthesis: goal records, candidate normalization, and unresolved obligations

A local inhabitant problem should be represented as

$$\mathfrak{g} = \big(u, \Gamma_u, T_u, \Lambda_u, \Omega_u, \mathcal{T}_{\partial u}, \mathcal{B}_u, \pi_u\big),$$

where u is the region, Γ_u the local context, T_u the target JuGeo type or judgment family, Λ_u the active laws, Ω_u the unresolved obligations, $\mathcal{T}_{\partial u}$ the neighboring treaty family, $\mathcal{B}_u$ the available evidence and computation budget, and π_u the current synthesis policy. This object should drive both hand-authored tactics and AI-generated candidates.

Candidate normalization is decisive. Two locally similar text proposals may induce very different semantic futures once normalized. The system should therefore reduce each candidate to a semantic candidate record

$$c = \big(p, \widehat{\Delta\mathcal{S}}, \widehat{\Delta\mathcal{O}}, \widehat{\Delta\mathcal{T}}, \mathrm{Norm}(p), \mathcal{R}_c\big),$$

where p is the surface artifact, $\mathrm{Norm}(p)$ the normalized core representation, and $\mathcal{R}_c$ residual obligations that remain if the candidate is accepted. Candidate comparison should occur primarily at this normalized level.

Unresolved obligations should never be hidden under a locally useful inhabitant. If a candidate closes the main target but introduces new dependent assumptions, effect side conditions, runtime probes, or overlap guard requirements, those must be emitted as explicit residuals in $\mathcal{R}_c$. This is the difference between inhabitant synthesis and patch theater. The system should count a candidate as progress only if the new residual profile is globally preferable.

The failure modes are familiar and should be named: *locally inhabitable but globally ungluable* candidates, *guard-smuggling* candidates that depend on undeclared preconditions, and *residual laundering* candidates that appear successful because they convert obligations into undocumented assumptions. Each is precisely a failure of semantic normalization.

43.2 Fleet search over admissible inhabitants: specialization, calibration, and comparison

A fleet should be understood as a diversified search distribution over inhabitant proposals. Different workers may specialize by obligation family, effect regime, donor transport skill, treaty proposal skill, or theorem-carrying synthesis. The important point is that all proposals enter a common comparison language. A worker is useful only insofar as its proposals predict correct semantic deltas in the relevant region class.

A bidder profile should therefore be a calibration record

$$\mathfrak{p}_w = \big(w, \mathcal{F}_w, \mathcal{D}_w, \theta_w, \epsilon_w, \Pi_w\big),$$

where $\mathcal{F}_w$ is the worker's specialization family, $\mathcal{D}_w$ its realized delta history, θ_w its current reliability parameters, ϵ_w its uncertainty model, and Π_w its challenge history. Bid weighting should be conditioned on support region, phase, and obligation family rather than learned as one global score.

Comparison among candidates should optimize expected semantic gain under risk:

$$\mathrm{Select}(c) = \alpha\,\mathbb{E}[\Delta_{\mathrm{closure}}(c)] + \beta\,\mathbb{E}[\Delta_{\mathrm{treaty}}(c)] + \gamma\,\mathbb{E}[\Delta_{\mathrm{replay}}(c)] - \delta\,\mathbb{E}[\Delta_{\mathrm{fragility}}(c)] - \varepsilon\ \mathrm{Cost}(c).$$

The comparison function should also preserve diversity across candidate law shapes and support regions, because early convergence on one family of proposals can hide simpler or more stable inhabitants nearby.

Calibration is the core theorem burden. The system should continually compare predicted semantic compression to realized compression. A fleet member that is excellent at local structural closure but poor at overlap stability should be trusted for the former and challenged aggressively on the latter. This is how AI search becomes disciplined: not by assuming a universal worker, but by forcing worker quality to become a region-sensitive empirical object.

43.3 Semantic backpressure: congestion signals, treaty debt, and pacing rules

Semantic backpressure should be a first-class orchestration signal because generation quality collapses when proposal rate exceeds stabilization capacity. The system should compute a pressure field over support regions, for example

$$P(R) = \alpha H_{\mathrm{ov}}(R) + \beta D_{\mathrm{treaty}}(R) + \gamma B_{\mathrm{chal}}(R) + \delta R_{\mathrm{reopen}}(R) - \varepsilon C_{\mathrm{closure}}(R),$$

where H_{ov} is overlap entropy, D_{treaty} treaty debt, B_{chal} challenge backlog, R_{reopen} replay fragility, and C_{closure} recent closure gain. High pressure means that the region is producing unresolved shared law faster than the system can stabilize it.

Backpressure should change behavior, not just dashboards. When $P(R)$ exceeds a threshold, the controller should throttle pure inhabitant search in R and reallocate effort toward treaty discovery, witness extraction, support refinement, or higher-overlap modeling. This is support-aware reopening in proactive form: rather than waiting for a replay explosion, the system recognizes that the geometric conditions for stable gluing are deteriorating and shifts phase accordingly.

The pacing rules should be explicit.

1. If challenge backlog dominates, prioritize challenge resolution and law sharpening.
2. If treaty debt dominates, prioritize overlap-law discovery and guard completion.

3. If replay fragility dominates, prioritize support refinement and gluing witness reinforcement.

4. If closure gain remains high while pressure is moderate, continue local inhabitant expansion.

A mature system should also record *pressure provenance*: which clauses, overlaps, and recent moves caused the pressure signal to rise. Otherwise backpressure becomes a mystical orchestration heuristic rather than a scientifically inspectable control variable.

43.4 Implementation consequences

The implementation architecture implied here is concrete. The synthesis subsystem should own goal records, normalized candidate records, bidder profiles, calibration ledgers, and regional pressure maps. The orchestrator should be able to ask, for any proposed inhabitant, what section delta it claims, what residuals it leaves, what overlaps it stresses, and how that compares with the worker's past performance on the same obligation family. A repository that lacks these records cannot honestly say it is doing large-scale inhabitant synthesis. It is only sampling patches.

Chapter 44

Integration as global gluing under replay

Integration should be where the generation chapters cash out. If local construction, treaty formation, and frontier control are correct, then integration produces a global section together with explicit witness, replay scope, fragility map, and residual obstruction field. If they are not, integration should expose precisely where gluing fails rather than flattening all failure into build or test red bars.

44.1 Global gluing under replay: integration criteria, fragility regions, and support scope

A gluing witness should be a first-class record

$$W_{\mathrm{glue}} = \big(\mathfrak{U}, \mathcal{H}_\bullet, \{s_u\}, \{\tau_\sigma\}, \{w_\sigma\}, \mathcal{X}_{\mathrm{res}}, \nu_{\mathrm{replay}}\big),$$

where $\mathfrak{U}$ is the base cover, $\mathcal{H}_\bullet$ the active hypercover, $\{s_u\}$ the accepted local sections, $\{\tau_\sigma\}$ the treaty family on simplices, $\{w_\sigma\}$ the compatibility witnesses, $\mathcal{X}_{\mathrm{res}}$ residual obstructions, and ν_{replay} the replay law. Integration should succeed exactly when this witness exhibits the family $\{s_u\}$ as descent data for a genuine global section under the declared treaty relations.

Fragility should be localized rather than averaged. A fragility region is a support region whose compatibility depends on low-margin witnesses, provisional treaties, recent guard changes, sparse evidence, or unstable higher-overlap summaries. The integration artifact should therefore include a fragility map

$$\mathcal{F}_{\mathrm{frag}} : \mathrm{Supp}(X) \to [0,1] \times \mathrm{Reasons},$$

so that later replay and closure reports can say not only that the project currently glues, but where that gluing is delicate.

The important failure distinction is between removable discrepancy and persistent obstruction. If local representative changes can make the current discrepancy cocycle exact, then the failure is still local repair. If not, then integration has revealed a genuine obstruction class and should route repair toward treaty synthesis, hypercover refinement, or theory growth. Integration is therefore not the end of the pipeline. It is the main measurement point at which the geometry of the remaining problem becomes visible.

44.2 Cumulative generation memory: assembly provenance, treaty history, and accepted candidates

Generation memory should preserve how the current global assembly came to exist. A useful assembly provenance record is

$$\mathfrak{a} = \big(W_{\mathrm{glue}}, \Pi_{\mathrm{sections}}, \Pi_{\mathrm{treaties}}, \Pi_{\mathrm{candidates}}, \Pi_{\mathrm{reopen}}, t\big),$$

where the Π components record provenance of accepted local sections, treaty evolution, candidate competition, and reopening events up to time t. This record lets the system answer implementation-critical questions: which candidate family usually produces stable sections on this region, which treaties have been repeatedly reopened, and which integration wins depended on fragile bridge theorems.

Cumulative memory matters because project-scale construction is path dependent. Two projects may have the same current source text and very different replay properties because the semantic capital behind that text differs. One may rest on stabilized overlap law and replay-tested witnesses; the other may rest on provisional treaties and recent unexplained repairs. The assembly provenance record is how the system preserves that difference.

This chapter should therefore insist that accepted candidates remain identifiable after integration. An accepted section should not disappear into a final blob. Its origin strategy, challenge history, treaty interactions, and later replay behavior are all semantically relevant. The same is true of retired candidates, because repeated near-miss candidates often signal a latent missing law or a poorly drawn cover.

44.3 Implementation path for cumulative generation: archives, gluing reports, and witness attachment

The implementation path is straightforward if the dissertation remains serious about authority centers. The runtime should maintain:

1. a `GluingReport` object for each integration event;
2. an `AssemblyLedger` linking gluing reports to accepted local sections and treaty versions;
3. a `ReplayLedger` tracking which retained witnesses survived which change families;
4. a `FragilityIndex` keyed by support region and cause;
5. a `CandidateArchive` connecting accepted and rejected candidates to later integration outcomes.

Witness attachment is particularly important. A compatibility edge in a gluing report should not merely say "compatible." It should name whether compatibility was established by proof, solver discharge, runtime witness, controlled semantic judgment, or human ratification, and under which guards. That information is required both for public trust reporting and for support-aware reopening. If the evidence channel changes, the replay consequences change.

44.4 Theorem and falsification burden for generation closure

The theorem burden should be named precisely. One wants *gluing-soundness*: a successful gluing report induces a global section up to the declared treaty relations. One wants *archive faithfulness*: the assembly archive preserves which witnesses and treaties were actually used. One wants *replay conservativity*: replay may reopen a claim, but it may not silently strengthen it. One wants *candidate provenance honesty*: public closure summaries may not imply that a stabilized section was obtained more directly or more strongly than the archive shows.

The falsification burden is equally important. The theory is weakened if gluing reports repeatedly fail to predict replay behavior, if fragility maps do not localize future failures, if candidate provenance is too expensive to preserve, or if integration success correlates poorly with later semantic stability. These are not side notes. They are the empirical conditions under which the chapter's mathematical promises should be judged.

Part VII

Orchestration, Frontier Control, and Fleet Semantics

Chapter 45

Project-scale orchestration as semantic control

Project-scale orchestration should be presented as control over a changing semantic state, not as a scheduler wrapped around tools. The orchestrator acts on a state whose coordinates are covers, hypercovers, local contexts, partial sections, overlap treaties, unresolved obligations, evidence channels, frontier nodes, archives, and resource budgets. Its task is to choose the next admissible move so as to improve global attainability, not merely to maximize local activity. Once this is said clearly, long-horizon generation stops looking like a bag of prompts and starts looking like a typed control system.

The chapter should also make the unification claim explicit: orchestration is specification verification lifted from a static judgment to a dynamic search over partial judgments. The specification chapter defined a target geometry and asked whether a proposed section satisfies it. The orchestration chapter keeps the same target geometry fixed but adds a control law over partial sections, residual ledgers, and treaty states. In that sense the orchestrator does not sit outside verification. It is the component that decides which local verification problem, treaty negotiation, bridge search, or theorem-growth move should be attempted next so that the same target section can eventually descend globally.

A useful slogan is that verification answers "does this section satisfy the target geometry now?" while orchestration answers "which admissible move most improves the probability that some future section will satisfy that same target geometry?" Both questions are asked in the same judgment presheaf and use the same support discipline, evidence algebra, and obstruction language. This is what makes JuGeo one math rather than a verifier plus an unrelated agent layer.

A useful project-scale control state is

$$\mathcal{X} = (\mathcal{G}, \mathcal{F}, \mathcal{B}, \Phi),$$

where $\mathcal{G}$ is the semantic generation state, $\mathcal{F}$ the active frontier of admissible futures, $\mathcal{B}$ the budget state across evidence channels and workers, and Φ the current semantic phase estimate. The controller chooses actions

$$a \in \mathrm{Act}(\mathcal{X})$$

subject to admissibility, locality, trust, and replay constraints. This formulation is what allows local elaboration, theorem mining, treaty stabilization, routing repair, and archive reuse to be compared by one objective.

The objective should be stated semantically. A policy π should optimize expected semantic leverage,

$$J(\pi) = \mathbb{E}_\pi \sum_t \Big(w_1 \Delta_{\text{closure}} + w_2 \Delta_{\text{stability}} + w_3 \Delta_{\text{bridge}} + w_4 \Delta_{\text{optionality}} - w_5 \text{cost} - w_6 \text{overclaim} \Big)_t,$$

with weights allowed to vary by phase. This objective makes precise why some actions with small immediate output are still valuable: they may reduce overlap entropy, expose a reusable bridge theorem, or shift the frontier into a more tractable region.

The AG layer should appear directly in that objective. A semantically good controller is not merely one that closes many obligations. It is one that reduces obstruction mass, shrinks the support of persistent cocycles, selects hypercovers whose Čech data is informative rather than noisy, and decides when a nontrivial class should trigger theorem growth or coefficient change rather than more local search. In other words, orchestration is partly a control problem over the changing cohomology of the project state.

The theorem burden should be assigned to orchestration itself. One wants non-overclaim, replay-coherence, locality of declared support, and challenge reproducibility. These results need not all be global at first, but the theory should insist that the orchestrator preserve the records from which such claims could eventually be proved.

45.1 Orchestration is a control problem on a changing semantic state

The controller should be specified as a receding-horizon policy over semantic state rather than as a prompt scheduler. At each epoch it observes a state

$$\mathcal{X}_t = (\mathcal{G}_t, \mathcal{F}_t, \mathcal{B}_t, \Phi_t, \mathcal{A}_t)$$

and chooses an admissible move that changes local sections, treaties, proof burden, or theory inventory. The important computational point is that local fixed-point engines may already have their own convergence guarantees—for example, POPL-style bidirectional section synthesis on finite-height lattices converges in bounded iterations relative to the local cover—but the orchestrator decides where those bounded computations are spent.

This makes orchestration a meta-algorithm over several proof-producing subroutines. The controller must know which frontier nodes are structurally decidable now, which require theorem growth, which are blocked by treaty debt, and which are likely to collapse under replay. A purely activity-driven agent loop cannot express these distinctions. A semantic controller can.

45.2 The controller should optimize semantic leverage rather than local activity

A useful control objective should measure semantic leverage directly:

$$\mathcal{J}_{\text{ctrl}}(a \mid \mathcal{X}) = \alpha \, \widehat{\Delta H^0} - \beta \, \widehat{\Delta \text{rk} H^1} + \gamma \, \widehat{\Delta \text{Equiv}} + \eta \, \widehat{\Delta \text{VCRed}} - \delta \, \widehat{\Delta \text{Replay}} - \varepsilon \, \text{Cost}(a).$$

Here $\widehat{\Delta H^0}$ is expected gain in descended global sections, $\widehat{\Delta \text{rk} H^1}$ expected reduction in independent obstruction mass, $\widehat{\Delta \text{Equiv}}$ expected progress on relational descent, and $\widehat{\Delta \text{VCRed}}$ expected product-cover VC reduction. This imports the strongest POPL-style advantages directly into the control loop.

The practical consequence is that a small theorem or treaty refinement may outrank a larger local patch if it is predicted to shrink obstruction rank, discharge many product-cover verification conditions, or turn a local equivalence into a global one. Activity count is therefore the wrong objective. Semantic leverage is the right one.

45.3 Search should proceed on a frontier of semantically admissible futures

A frontier node should be retained only if it is semantically admissible: the proposed future must preserve typing, name its support, specify its expected evidence route, and state what kind of failure would falsify it. A concrete admissibility predicate is

$$\mathrm{Adm}_{\mathrm{front}}(n) \iff \mathrm{Typed}(n) \wedge \mathrm{Scoped}(n) \wedge \mathrm{ReplayNamed}(n) \wedge \mathrm{FailureSchema}(n)\downarrow .$$

Nodes that violate this predicate should be demoted to ideation backlog or discarded, not treated as ordinary work items.

This is how JuGeo avoids turning the frontier into a wish list. The frontier is a set of candidate future semantic states that the runtime knows how to test, challenge, or replay. That restriction is what keeps global search compatible with local proof discipline.

45.4 Proof obligations for orchestration itself

The orchestrator should carry its own theorem burden. At minimum one wants support-scope honesty for declared moves, replay conservativity for accepted control decisions, and challenge reproducibility for any high-leverage promotion. One also wants a bounded-starvation condition relative to the active phase policy: if a frontier class remains admissible and repeatedly high-value under the objective, the scheduler may not postpone it indefinitely behind cosmetically active work.

The POPL-style metatheory suggests stronger medium-term goals. If local subroutines are sound and cover-relative complete on their admitted fragments, then orchestration should be proved conservative with respect to them: changing control policy may change when an obstruction is found, but not what those subroutines certify once run under the same premises. This is the right proof obligation for a semantic controller.

Chapter 46

Routing across Z3, controlled LLMs, runtime evidence, and humans

This chapter should make routing explicit as part of the semantics of judgment management. The point is not simply to decide which tool to call next. It is to define what kind of answer is being requested, what evidence form would count as success, what form of failure is informative, and under what conditions the routing choice must later be revisited.

46.1 The router is a semantic judgment, not a convenience heuristic

Routing should be a judgment on obligations, not an undocumented heuristic about which backend feels appropriate. Every unresolved obligation has a coordinate, a support region, a trust target, a current normal form, and a family of admissible evidence channels. The router's job is to expose this structure and choose a channel for principled reasons. Some obligations are already structural and should be canonicalized for Z3. Some are mixed and should be split by evidence family. Some require runtime witnesses before any formal discharge attempt is meaningful. Some require human intervention or theory growth because the current bridge inventory is insufficient.

A routing judgment for an obligation o should therefore be a record

$$\rho(o) = (k, \tau, \eta, \kappa, \nu),$$

where k is the chosen evidence channel, τ the target evidence form, η the expected success witness schema, κ the expected failure or counterexample schema, and ν the invalidation policy. This makes routing legible. The router is not merely choosing Z3 or an LLM. It is asserting what kind of answer would count, what form of failure is informative, and when the decision must be recomputed.

The mixed case is especially important. A common failure mode is to route a mixed obligation wholesale to whichever subsystem appears strongest. JuGeo should instead factor mixed obligations into structural, semantic, empirical, and treaty-clarification parts, route those separately, and then recompose the result in the common judgment language. This is better for soundness, better for explanation, and better for calibration, because each evidence channel is judged only on claims inside its admitted scope.

The implementation consequence is that routing should have its own authority center, with tests and theorem targets. One wants router soundness, disciplined fallback, and not-yet-routable honesty. The last of these is crucial: when no current backend family is admissible, the system should say so explicitly and feed that fact into ideation and theory-pack growth.

46.2 Canonicalized fragments for Z3

46.3 Mixed obligations should be split, not guessed over

46.4 Routing proofs and failure modes

Chapter 47

Fleet semantics

Fleet semantics should be described as competitive search over semantic moves rather than as mere parallel patch production. This makes the fleet mathematically legible and gives the implementation a reason to preserve bid deltas, challenge records, and calibration traces as first-class objects.

47.1 A fleet member should propose semantic moves, not mere patches

A fleet architecture becomes intellectually defensible only when each member proposes semantic moves rather than mere textual patches. A patch is one possible surface realization of a move, but the real proposal should include intended judgment deltas, expected obligation effects, anticipated treaty interactions, and a self-declared uncertainty profile. This is what turns a fleet from parallel noise into a competitive search over admissible futures.

A fleet bid should therefore be normalized into a record

$$b = (\Delta\mathcal{S}, \Delta\mathcal{O}, \Delta\mathcal{T}, \Delta\mathcal{E}, \Delta\mathcal{A}, p, q),$$

where the Δ terms describe proposed semantic deltas, p is the concrete artifact proposal, and q is the justification trace stating why the bidder expects those deltas to occur. A challenge is then a typed counterclaim against one or more of these components: the section delta may be ill-scoped, the obligation delta may hide new residuals, the treaty delta may overstate compatibility, or the evidence delta may exceed what the cited channel can warrant.

This representation gives a clean calibration target. The fleet controller should learn which bidders are reliable by comparing predicted semantic deltas to realized deltas after validation. Reliability should be conditioned by obligation family, support region, domain pack, and project phase. A bidder that is excellent at local structural repairs may be poor at treaty proposals; a bidder that is strong in early decomposition may be weak in late replay stabilization. Fleet semantics should preserve this granularity.

The theorem targets are correspondingly concrete. One wants a no-unearned-certificate theorem, a challenge-exposure theorem for declared invariant families, and a calibration theorem saying that bidder weighting converges toward semantically reliable proposal classes under suitable assumptions. These are the results that make fleet semantics more than folklore.

47.2 Challenges should be typed counterclaims

47.3 Accepted competition should improve search quality over time

47.4 Proof targets for fleet semantics

Chapter 48

Frontier algorithms and phase transitions

The frontier should be treated as a search object over semantically admissible futures. Managing it well means balancing expected closure gain, stability, diversity, and backpressure rather than merely consuming tasks in dependency order.

48.1 The frontier should be managed as a budgeted search over semantic states

The frontier should be managed as a budgeted search over semantic states, not as a list of tasks waiting for workers. A frontier node should represent a semantically admissible future: a successor state together with its predicted closure gain, stability gain, theorem yield, treaty impact, cost, uncertainty, and support scope. Search then becomes a choice among future semantic worlds rather than among arbitrary units of work.

A useful node record is

$$n = (\mathcal{X}_n, \hat{V}_n, \hat{U}_n, C_n, D_n, P_n),$$

where $\mathcal{X}_n$ is the candidate successor state, $\hat{V}_n$ the predicted semantic value, $\hat{U}_n$ the uncertainty estimate, C_n the projected cost, D_n a diversity contribution, and P_n the backpressure penalty inherited from affected support regions. A principled selection score should therefore combine value, uncertainty, diversity, and pressure rather than letting any one of them dominate by accident.

48.2 Bandit-style allocation across heterogeneous evidence channels

48.3 Search should preserve diversity across support regions and proof modes

48.4 Large projects move through distinct semantic phases

48.5 Phase changes should be triggered by semantic statistics

Chapter 49

Treaty synthesis, negotiation memory, and archival semantics

Treaties are the stabilized overlap laws that make global gluing possible at project scale. The system therefore needs a dedicated account of how treaties are discovered, negotiated, remembered, reused, challenged, and retired. Raw history is not enough. What matters is semantic negotiation memory: which friction patterns produced which law proposals, what challenges shaped them, what support regions they govern, and how they behaved under replay.

49.1 Interfaces should be discovered as well as checked

Interfaces should not be treated as fully given in advance. Much of large-scale construction consists in discovering which overlap laws are actually needed for neighboring regions to glue. An interface is therefore a discovered semantic object before it is a checked one. The system should mine recurrent dependence patterns, repair motifs, and stable behavioral correlations on overlaps and use them to propose interface laws with explicit guards and invalidation support.

This is the right place to say that interface discovery is a form of local law induction over overlap simplices. The AG layer contributes the support geometry on which the law lives. The DTT layer contributes the dependent parameters and guards. The AI layer contributes proposal pressure over candidate law shapes. A treaty is accepted only when this candidate law has survived challenge strongly enough to count as reusable descent data.

49.2 Negotiation memory

Negotiation memory should preserve the semantic history of a treaty, not just its current text. A useful record is

$$m_\tau = \big(\tau, \Pi_{\text{motif}}, \Pi_{\text{proposal}}, \Pi_{\text{challenge}}, \Pi_{\text{replay}}, \Pi_{\text{retire}}\big),$$

where the Π components record motivating friction motifs, proposal variants, challenge sessions, replay outcomes, and retirement or strengthening events. This record is how the system later explains why a treaty exists and whether it still deserves authority.

Negotiation memory is implementation-critical because treaty failure is often path dependent. A law that looks simple in its final form may have required several failed versions before the correct guards were exposed. If that history is discarded, later reopening will repeat the old mistakes and the system will not know whether the law is genuinely decaying or merely being misunderstood.

49.3 Law discovery as a search problem

Law discovery should be treated as frontier search over candidate overlap laws. The search space contains strengthened or weakened guards, generalized relational forms, effect-conditional clauses, representation-level equalities, and higher-simplex compatibility laws. The value of a candidate law should be measured by future geometric effect: reduction in overlap entropy, reduction in challenge rate, reduction in replay churn, and increase in local section transportability.

A good law-discovery engine should also preserve negative results. A repeatedly proposed law that keeps failing under the same challenge family is evidence about the shape of the true overlap law or about the need for a finer hypercover. Negative treaty history is therefore semantic capital, not clutter.

49.4 Semantic archives versus raw history

Raw history stores actions. Semantic archives store meaning-preserving compression of those actions. For treaty work, the archive should store friction motifs, candidate laws, challenge schemas, replay outcomes, support scopes, and realized geometric effects. This lets later orchestration ask not "what did we do?" but "which law shapes helped on which overlaps, under which guards, and with what replay cost?"

The archive entry for a treaty event should therefore be typed:

$$a_\tau = \big(\text{simplex}, \text{law schema}, \text{guards}, \text{witness class}, \text{challenge outcome}, \text{replay outcome}, \text{geometric gain}\big).$$

Such entries are much more valuable than chat logs or commit messages when the system needs to reason about future treaty work.

49.5 Archival value, semantic capital, and theorem decay

Treaties, bridge theorems, and replay witnesses accumulate semantic capital, but they also decay. A law that once reduced overlap entropy may later become a source of fragility if the surrounding support geometry changes. The archive should therefore record realized reuse, replay survival, and decay indicators for each asset. The important architectural consequence is that semantic memory must support retirement and demotion as well as promotion.

A serious dissertation chapter should say that theorem and treaty decay are not embarrassing anomalies. They are expected features of a changing project geometry. The system's obligation is to detect and localize that decay honestly rather than to preserve authority by inertia.

Chapter 50

Exploration, exploitation, and frontier control

Frontier control should explicitly balance exploration and exploitation over semantic futures. Exploration means spending budget to reduce uncertainty about covers, overlap laws, higher simplices, theorem opportunities, and worker strengths. Exploitation means deepening a known productive basin to harvest closure, replay stability, or treaty consolidation. Both are necessary, and both should be judged by their effect on future attainable semantic states.

50.1 The frontier as a controlled search object

The frontier is not a queue. It is a controlled family of candidate semantic futures together with uncertainty and geometric metadata. A good frontier object should include candidate support region, move class, predicted value, uncertainty, diversity contribution, pressure footprint, and expected falsification trace. This lets the controller ask not only "what seems promising?" but "what would we learn if this move failed?"

That last question matters because exploration is often valuable precisely when it falsifies a tempting but wrong picture of the project geometry. A failed hypercover refinement, an unsuccessful treaty candidate, or a donor transport rejection can still be high-value frontier moves if they sharply reduce uncertainty about where obstruction really lives.

50.2 A frontier control objective

A useful frontier control objective is

$$\mathcal{J}_{\text{frontier}} = \alpha\,\mathbb{E}[\Delta_{\text{closure}}] + \beta\,\mathbb{E}[\Delta_{\text{stability}}] + \gamma\,\mathbb{E}[\Delta_{\text{information}}] + \delta\,\mathbb{E}[\Delta_{\text{optionality}}] + \varepsilon D - \zeta P - \eta C,$$

where $\Delta_{\text{information}}$ measures uncertainty reduction, D diversity contribution, P semantic backpressure, and C cost. This objective makes exploration a mathematically respectable part of the same control law as exploitation, because both contribute to future attainable descent.

The AG content should remain explicit: exploration may refine the cover, expose a hidden overlap, or localize an obstruction class more sharply; exploitation may shrink a known obstruction or stabilize a treaty. Both are geometric operations on the state space.

50.3 Exploration pressure

Exploration pressure should rise when the project displays any of the following: repeated failure under locally different repairs, unstable treaty proposals on the same simplex, poor calibration across workers in one region class, or persistent residual obstruction whose support is diffuse under the current cover. In such cases the system should spend budget on alternative covers, higher simplices, challenge families, or theory routes rather than only proposing more local inhabitants.

A mature controller should also separate *cover exploration*, *law exploration*, *worker exploration*, and *theory exploration*. These are different uncertainty classes and should not be collapsed into one generic "try something new" move.

50.4 Exploitation pressure

Exploitation pressure should dominate when one basin already has low overlap entropy, stable replay, reliable bidders, and strong closure gradients. In that regime, the controller should harvest the basin aggressively because the expected global delta is high and the risk of semantically wasted effort is low. Exploitation is therefore not mere greed. It is the disciplined completion of a region whose geometry is already well understood.

The failure mode is overexploitation: pushing deeper into a basin whose apparent productivity is masking unresolved higher-overlap law or accumulating replay fragility. This is why exploitation pressure must be checked against semantic backpressure and fragility maps rather than measured by local throughput alone.

Part VIII

Ideation, Theorem Growth, and Mathematical Discovery

Chapter 51

Ideation as search over semantic futures

Ideation should be treated as search over semantic futures rather than as unstructured brainstorming. The objective of this chapter is to define what an idea is in JuGeo, how an idea should be valued before implementation, and why ideation belongs inside the same AG+DTT+AI judgment geometry as verification, repair, and orchestration. An idea is valuable here only if it changes the reachable semantic future of a project under an explicit purpose and an explicit evidence discipline.

A useful state object for this chapter is

$$\mathcal{I} = (\mathcal{S}_{\text{now}}, \mathcal{P}, \mathcal{F}, \mathcal{B}, \mathcal{A}),$$

where $\mathcal{S}_{\text{now}}$ is the current semantic state, $\mathcal{P}$ the declared purpose object, $\mathcal{F}$ a family of candidate future states or regime changes, $\mathcal{B}$ the current ideational budget, and $\mathcal{A}$ the archive of earlier proposals, rejections, and realized outcomes. This state object matters because ideation should not be asked to produce isolated conjectures. It should be asked to propose transitions from one judgment state to another, with predicted leverage, predicted proof burden, and predicted support scope.

The AG contribution is that future value is local-to-global: some ideas reduce obstruction mass only in one support basin, while others alter overlap coherence or bridge availability across a wide region of the cover. The DTT contribution is that candidate futures are not mere stories about what might become true; they are possible future judgment states with contexts, obligations, inhabitants, and residuals. The AI contribution is controlled proposal over a large search space of decompositions, conjectures, bridges, and new semantic regimes. Put differently, AI proposes futures, DTT specifies what it would mean for those futures to count as attained, and AG explains where in the project those futures would propagate.

The implementation path should therefore include explicit future-state records, ideation ledgers, and replayable prediction traces. A mature system should store not only that an idea was proposed, but what future semantic delta it claimed, what purpose it served, what support scope it targeted, and which later evidence confirmed or falsified that forecast.

51.1 Idea objects: future attainability, predicted leverage, and support scope

An idea object should be a typed forecast, not a slogan. A useful record is

$$\mathfrak{i} = (\Delta_{\text{target}}, \widehat{H^0}, \widehat{H^1}, \mathcal{R}_{\text{supp}}, \rho_{\text{proof}}, \rho_{\text{transport}}, \kappa_{\text{fail}}, \tau_{\text{archive}}),$$

where Δ_{target} is the claimed semantic delta, $\widehat{H^0}$ and $\widehat{H^1}$ are predicted global-section gain and obstruction reduction, $\mathcal{R}_{\text{supp}}$ is the support region affected, ρ_{proof} the planned validation route, $\rho_{\text{transport}}$ any bridge or donor dependence, κ_{fail} the anticipated falsification shape, and τ_{archive} the archive key. This record is what makes future-attainability computational.

An idea should enter the frontier only after these fields are instantiated. Otherwise JuGeo has no way to compare a theorem-growth proposal against a cover refinement, a bridge theorem, or a local repair. The record also enforces support-scope honesty: an idea that only affects one overlap basin should not be reported as a project-wide advance.

51.2 Pre-implementation valuation: expected semantic value, uncertainty, and auditability

Before implementation, valuation should separate expected value from uncertainty and from auditability. A convenient tuple is

$$V(\mathfrak{i}) = (\widehat{\Delta H^0}, \widehat{\Delta \mathrm{rk} H^1}, \widehat{\Delta \mathrm{VC}}, \widehat{\Delta \mathrm{Equiv}}, U, A),$$

where U measures forecast uncertainty and A measures how replayable and auditable the forecast is. High-value, low-auditability ideas should usually remain exploratory rather than policy-setting.

This is where JuGeo should distinguish itself from ordinary brainstorming. Every valuation must be replayable against later outcomes. The archive should remember which forecasts were accurate, which systematically overstated global benefit, and which failed because their support scope or transport burden was misunderstood.

51.3 Ideation signals: obstruction rank, overlap entropy, and bottleneck geometry

The ideation subsystem should draw from explicit signals rather than stylistic frustration. The strongest signals are persistent nonzero obstruction classes after local repair, stagnating or rising cover-relative obstruction rank, overlap basins whose entropy remains high across replay, and theorem-growth candidates whose expected Mayer–Vietoris gain exceeds local patch gain. These signals let the system tell the difference between "keep repairing" and "change the mathematics."

A practical signal vector is

$$\sigma_{\text{idea}} = (\mathrm{rk} H^1, E_{\text{ov}}, \Delta_{\text{MV}}, \text{ReplayDrag}, \text{BridgeScarcity}).$$

When several components remain high under stable normalization, JuGeo should escalate from local coding to ideation or theorem growth rather than continuing blind search.

Chapter 52

Mathematical ideation optimization

Mathematical ideation should be optimized as a purpose-conditioned search over theory space rather than as an unconstrained request for clever conjectures. The objective of this chapter is to say how JuGeo should search for new mathematics, including new kinds, new coefficient theories, new bridge theorems, and novel mathematical regions, when a project has a declared purpose and a measured obstruction field. This makes the repeated question "what new mathematics should be explored next for this purpose?" into a judgment-geometric optimization problem rather than an aesthetic choice.

A useful state object is

$$\mathcal{M}_{\mathrm{idea}} = (\mathfrak{U}_{\Theta,\bullet}, \mathcal{O}_\Theta, \mathcal{P}, \mathcal{Q}_{\mathrm{dep}}, \mathcal{A}, \mathcal{B}),$$

where $\mathfrak{U}_{\Theta,\bullet}$ is a chosen cover or hypercover of theory space, $\mathcal{O}_\Theta$ is a purpose-weighted obstruction field over that space, $\mathcal{P}$ is the declared purpose object, $\mathcal{Q}_{\mathrm{dep}}$ is the family of dependent goals whose attainability matters for that purpose, $\mathcal{A}$ is the archive of prior proposals and realized outcomes, and $\mathcal{B}$ is the ideational budget. Candidate ideas should be represented as local sections of a sheaf of candidate regimes,

$$\mathcal{R}_{\mathrm{cand}}(U) = \{\text{conjecture schemas, new kinds, coefficient changes, bridge templates, and transport maps admissibl}$$

for theory patches $U \in \mathfrak{U}_{\Theta,\bullet}$. The central AG point is that ideation is local-to-global. A proposal may look promising on one patch of theory space and still fail to glue across overlaps. That failure is informative: a nontrivial Čech compatibility defect is evidence of missing mathematics, not merely of poor brainstorming.

The optimization target should therefore be explicit. For a candidate regime move r, one should maximize

$$\mathcal{J}_{\mathrm{idea}}(r \mid \mathcal{P}, \mathcal{S}) = \alpha\, \widehat{\Delta \mathrm{Obs}}_{\mathcal{P}}(r) + \beta\, \widehat{\Delta H^0_{\mathcal{P}}}(r) + \gamma\, \mathrm{NoveltyRegion}(r, \mathcal{A}) + \eta\, \mathrm{DescentGain}(r, \mathfrak{U}_{\Theta,\bullet}) - \delta\, \mathrm{ProofBurden}(r) - \varepsilon\, \mathrm{Tr}$$

where $\widehat{\Delta \mathrm{Obs}}_{\mathcal{P}}$ estimates reduction in purpose-relevant obstruction mass, $\widehat{\Delta H^0_{\mathcal{P}}}$ estimates gain in reachable globally coherent artifacts, NoveltyRegion measures movement into underexplored semantic regions rather than rhetorical surprise, DescentGain estimates how much easier gluing becomes under the proposal, and the penalty terms measure proof burden, bridge fragility, and archive redundancy. The point is not exact numerics. The point is to force the engine to explain why a new area of mathematics is predicted to matter now, for this purpose, under this proof budget.

The AG contribution is central because the system should search over covers, hypercovers, coefficient choices, and regime changes in response to the geometry of the obstruction field. Some failures call for a finer cover, some for a different coefficient theory, some for a new semantic region

entirely. The DTT contribution is that admissibility is not optional. A mathematically interesting proposal counts only if it can be linked to a dependent goal family with explicit contexts, inhabitants, residual obligations, and proof routes. The AI contribution is controlled proposal over candidate future regimes, analogical donor areas, and nonlocal bridge opportunities. AI should not decide mathematical authority; it should propose local sections and transport suggestions that the AG and DTT layers then pressure-test.

The algorithmic picture should be concrete. The engine should first estimate a purpose-weighted obstruction density on theory space, then choose or refine a cover adapted to that density, then ask the proposal model for local regime sections near high-pressure and semantically adjacent novel regions. A DTT admissibility filter should reject proposals whose claimed outputs cannot inhabit the relevant dependent goals. The surviving proposals should be tested for pairwise and triple-overlap compatibility, with change-of-cover and change-of-coefficients probes available when the first gluing attempt fails. If local promise persists but Čech-style compatibility defects remain nontrivial under multiple covers, that should be archived as a cohomological signal for missing mathematics rather than silently collapsed into "bad idea."

The theorem targets should be correspondingly explicit. One wants no-false-promotion for ideational proposals, support-scope honesty for local gains, refinement stability under cover changes, and purpose monotonicity stating that strengthening the purpose object cannot reduce the recorded proof burden of a proposal without added evidence. The implementation consequences are equally clear. JuGeo should maintain a theory-space atlas, a sheaf-like store of candidate regime sections, a novelty archive indexed by semantic region, and replayable traces recording which proposals changed covers, coefficients, or regimes before they either paid off or failed. That is the infrastructure required if ideation is to discover genuinely new mathematics rather than merely rename intuition.

Chapter 53

Mathematical research assistance inside a codebase-scale verifier

Mathematical research assistance inside a codebase-scale verifier should arise from the observed geometry of failure in the codebase itself. The objective of this chapter is to make the verifier a research director that can tell when repeated coding difficulty is really a sign of missing mathematical language, missing bridges, or missing kinds. Research help is therefore not an external oracle appended to engineering work. It is an internal response to persistent obstruction patterns across a large semantic hypercover of the project.

A useful state object is

$$\mathcal{R}_{\text{assist}} = (X, \mathcal{H}_{X,\bullet}, \mathcal{O}_X, \mathfrak{U}_{\Theta,\bullet}, \Lambda, \mathcal{P}, \mathcal{A}),$$

where X is the current codebase judgment state, $\mathcal{H}_{X,\bullet}$ is a project hypercover over modules, interfaces, tests, benchmarks, and documentation surfaces, $\mathcal{O}_X$ is the obstruction field over that hypercover, $\mathfrak{U}_{\Theta,\bullet}$ is a cover of candidate theory regions, Λ is a linkage map from code-side obstruction motifs to theory-side candidate regimes, $\mathcal{P}$ is the declared purpose, and $\mathcal{A}$ is the archive of earlier research suggestions and their outcomes. The key claim is that a mature verifier should not merely say "proof failed here." It should say whether the failure looks local, whether it recurs on overlaps, whether it survives ordinary repair, and which mathematical regions appear semantically adjacent to the missing capability.

A research-assistance action should be valued by a functional such as

$$\mathcal{V}_{\text{research}}(\Theta \mid X, \mathcal{P}) = \alpha\, \widehat{\Delta\text{Closure}}(\Theta) + \beta\, \text{ReuseBreadth}(\Theta) + \gamma\, \text{CounterexampleCompression}(\Theta) + \eta\, \text{BridgeValue}(\Theta) -$$

where $\widehat{\Delta\text{Closure}}$ estimates purpose-relevant future closure gain, ReuseBreadth estimates how many pressure regions could benefit, CounterexampleCompression measures whether many failures become explainable by one theorem family or new kind, BridgeValue measures compatibility with existing judgment language, and the penalties track cost, latency, and destabilization risk. This is how one chooses between another local repair, a bridge theorem, a coefficient change, a new theory pack, or an entirely new mathematical region.

The AG layer is what makes research assistance more than heuristic recommendation. Persistent pairwise and higher-overlap failures in $\mathcal{H}_{X,\bullet}$ are evidence that the present law language does not support descent. Repeated cocycles that do not die under routine repair should trigger theory-side search, not just more patching. The DTT layer governs admissibility by turning a research suggestion into a typed ticket: a proposed theorem, bridge, constructor, or regime only counts if it links to a

dependent goal family with explicit contexts, witness forms, and proof obligations. The AI layer supplies controlled proposals, analogical donor areas, and candidate future regimes that may be nonlocal relative to the current code. In this architecture, AI suggests possible mathematics, AG says where the pressure comes from and how broadly it propagates, and DTT says what it would mean for the suggestion to become semantically real.

The algorithmic path should be explicit. The verifier should cluster failed obligations by support shape, witness structure, overlap locus, and failed-repair trajectory; lift those clusters to candidate deficiency descriptions; retrieve neighboring theory regions and archived analogies; and then emit typed research tickets with conjecture templates, theorem targets, and planned evidence routes. Those tickets should be tested first on held-out pressure points drawn from the same obstruction family and then on nearby but distinct support regions to estimate generality. A good research assistant should therefore help discover new mathematics from code pressure, but it should do so under controlled evidence and with an archive that records whether a suggestion solved a local bottleneck, opened a reusable theorem ecology, or merely sounded elegant.

The implementation consequences follow directly. JuGeo should maintain pressure-point ledgers, theory-pack growth queues, counterexample-to-theorem clustering, transportability probes, and theorem-payback archives that track whether proposed mathematics later reduced proof burden, overlap entropy, or replay churn. The theorem targets should include pressure-faithfulness, meaning that research tickets reflect real obstruction families rather than noise, and compression honesty, meaning that the system is not credited for "discovering" mathematics unless it actually compresses a family of failures into a smaller and more reusable law package.

Chapter 54

Theorem-growth economics and semantic research scheduling

A serious codebase-scale verifier needs an explicit account of when local coding should give way to theory growth. The objective of this chapter is to make that decision principled. There are periods in which local repair has high marginal value, and there are periods in which repeated local repair merely confirms that the project lacks a sufficiently strong invariant language, bridge theorem, or domain pack. The theorem-growth question is therefore economic in a strict semantic sense: when does a unit of theory work buy more future closure than a unit of local elaboration?

A useful state object is

$$\mathcal{G} = (\mathcal{O}_{\text{rep}}, \Delta_{\text{local}}, \Delta_{\text{theory}}, C_{\text{proof}}, C_{\text{integration}}, \mathcal{B}),$$

where $\mathcal{O}_{\text{rep}}$ records recurring obstruction families, Δ_{local} is the expected semantic gain from further local work, Δ_{theory} the expected semantic gain from a theorem, bridge, or new pack capability, C_{proof} the validation cost of the theory move, $C_{\text{integration}}$ the expected cost of integrating it, and $\mathcal{B}$ the available budget. The chapter should argue that theorem-growth scheduling is justified when the expected future reduction in obstruction mass, replay churn, or bridge scarcity dominates the cost of proving and integrating the result.

The AG layer matters because repeated obstruction motifs and overlap instability are geometric signals that the current semantic basis is too weak. The DTT layer matters because the cost of theory growth depends on what new judgment forms, clauses, and proof obligations the proposed mathematics introduces. The AI layer matters because theory growth requires proposing abstractions, bridges, and lemma families that are not syntactically adjacent to the current code. This is exactly the place where purpose-conditioned AG+DTT+AI reasoning becomes more than a slogan: it becomes a scheduler for research effort inside engineering work.

The implementation path should include theorem-growth queues, repeated-obstruction clustering, cost forecasters for proof and integration burden, and an archive of realized theorem payback. A mature implementation should be able to say not only that a theorem was expensive, but whether its expense was repaid by later reductions in semantic work.

54.1 When coding should stop and theory should begin

Coding should pause for theory work when repeated local repairs stop buying semantic closure. A sharp operational trigger is a window of epochs in which local work changes surface artifacts but fails to reduce any of: cover-relative obstruction rank, product-cover verification condition count,

overlap entropy on the affected basin, or replay spread after local change. If those metrics plateau while theory-side candidates show plausible leverage, the system should schedule theorem growth.

This criterion is stricter than "the proof feels hard." It is based on measured local-to-global stagnation. Traditional verification pipelines rarely expose such a decision rule because they do not retain obstruction geometry explicitly. JuGeo should.

54.2 The growth signal

A useful growth signal is

$$G = \frac{\text{Recurrence} \cdot \text{Stability} \cdot \text{ReusePotential}}{1 + \text{LocalYield}},$$

where recurrence measures how often an obstruction family reappears, stability measures whether the family survives benign cover refinement or normalization change, reuse potential estimates how many nearby obligations would benefit from a shared theorem, and local yield measures the recent success of ordinary repair. Large G means the system is seeing the same mathematically structured pain repeatedly and should invest in new law.

The POPL-style Mayer–Vietoris result suggests an especially useful sub-signal: if a local branch or region keeps reintroducing global obstruction through the connecting map after ordinary fixes, then the missing asset is probably not another line edit but a stronger theorem or bridge.

54.3 Scheduling principle

Theory-growth scheduling should be queue-based and explicit. Each candidate theorem, bridge, or regime move should carry predicted payback, proof burden, integration burden, and replay risk. The scheduler should choose the highest expected long-run reduction in obstruction mass per unit of proof cost subject to budget and phase constraints.

In implementation terms, JuGeo needs a `TheoryTicket` queue, a `ProofBudgetLedger`, and a `PaybackArchive`. Without those objects, theory growth remains inspirational prose rather than a schedulable computational action.

Chapter 55

Experiment design for mathematical ideation optimization

Experiment design for mathematical ideation optimization should treat ideation as an evaluated subsystem, not as a source of impressive anecdotes. The objective of this chapter is to determine whether AG+DTT+AI ideation actually finds more useful mathematics, more purpose-fit novel regions, and better new kinds than local theorem mining, unguided novelty search, or free-form prompting. The central claim to test is that purpose-conditioned descent over theory space can discover optimal novel areas to explore for a declared goal more reliably than either conservative proof search or unconstrained mathematical imagination.

A useful experimental record is

$$e = (\mathcal{P}, \mathfrak{U}_{\Theta,\bullet}, \mathcal{D}_{\text{train}}, \mathcal{D}_{\text{hold}}, \mathcal{A}, \Pi_{\text{abl}}, \mathcal{T}_{\text{targets}}),$$

where $\mathcal{P}$ is the purpose object, $\mathfrak{U}_{\Theta,\bullet}$ is the evaluated cover or hypercover of theory space, $\mathcal{D}_{\text{train}}$ is a historical archive of obstruction episodes and realized theorem work, $\mathcal{D}_{\text{hold}}$ is a held-out set of future episodes, $\mathcal{A}$ is the proposal archive available to the system, Π_{abl} is the ablation family, and $\mathcal{T}_{\text{targets}}$ is the suite of theorem, bridge, kind, and regime targets against which usefulness will be judged. The benchmark must contain not only accepted theorems but also rejected proposals, failed transports, abandoned theory directions, and cases where the right answer was a change of cover or coefficient theory rather than a new theorem.

The evaluation objective should be explicit. For an ideation policy M, one should measure

$$\text{Score}(M) = \lambda_1 \,\text{RealizedDelta}_{\mathcal{P}}(M) + \lambda_2 \,\text{NovelUsefulAreaRate}(M) + \lambda_3 \,\text{TransportSuccess}(M) + \lambda_4 \,\text{TheoryPayback}(M$$

where $\text{RealizedDelta}_{\mathcal{P}}$ measures actual purpose-relevant semantic gain, NovelUsefulAreaRate measures how often the system enters a genuinely new mathematical region that later pays off, TransportSuccess measures whether analogical or bridge proposals survive validation, TheoryPayback measures downstream reuse and obstruction reduction, FalseFrontier measures promotion of mathematically attractive but semantically idle directions, and ProofWaste measures cost spent on directions whose realized leverage is poor. These metrics should be reported both globally and by obstruction family, theory region, and proposal class.

The AG structure of the experiment matters. Evaluation should not be done on one flat task list. It should be stratified by support region, overlap depth, and cover change. One should perform leave-one-obstruction-family-out tests, leave-one-theory-region-out tests, and change-of-cover robustness tests in which the same historical pressure is viewed under coarser and finer covers. One should also perform change-of-coefficients experiments to see whether the engine is correctly distinguishing "new

theorem needed" from "same phenomenon, wrong coefficient theory." This is where cohomological signals become experimentally meaningful: the system should be rewarded for discovering when persistent incompatibilities indicate missing mathematics and penalized when it mistakes ordinary local complexity for deep novelty.

The DTT role in evaluation is equally central. A proposal should only count as successful if it reduces proof burden for a typed dependent goal family or expands the family of inhabitants that can be constructed under the declared purpose. This avoids the usual failure mode in mathematical-assistance benchmarks, where a system is credited for producing sophisticated but semantically detached mathematics. The AI role should be evaluated through controlled ablations: archive-only retrieval, LLM-only novelty search, AG-only obstruction heuristics, DTT-only admissibility filtering, and the full AG+DTT+AI stack. The key question is not whether the model can say something mathematically interesting. It is whether the combined system can discover more useful mathematics, faster and more honestly, than these weaker alternatives.

The archive requirements are nonnegotiable. Every experimental item should retain the purpose object, the selected cover or hypercover, the active coefficient theory, the proposal trace, the predicted objective values, the realized outcomes, and the rejection reason when a proposal fails. Only with such archives can the chapter report calibration, failure modes, and negative results faithfully. The theorem targets for the evaluation layer should include calibration of support-scope forecasts, stability under archive replay, and robustness of the usefulness ranking under moderate cover refinement. The implementation consequence is that JuGeo needs an ideation replay harness, a long-lived proposal archive, and benchmark suites whose unit of analysis is the semantic future changed by a proposal rather than the beauty of the proposal itself.

Chapter 56

Theory-space navigation and purpose-conditioned mathematical exploration

Once theorem growth is admitted as a real action class, the dissertation must explain how the system moves through theory space. The objective of this chapter is to define candidate mathematical regimes, to show how navigation among them should be purpose conditioned, and to prevent the system from confusing mathematical novelty with semantic usefulness. Theory-space navigation is not a detour away from the main thesis. It is the large-scale extension of the same judgment geometry into the region where missing mathematics becomes the dominant bottleneck.

A candidate regime should be treated as a structured object,

$$\Theta = (K, L, B, T, M, N),$$

where K is a family of kinds or carriers, L a law family, B a bridge family into the current judgment language, T theorem templates suggested by the regime, M the initial evidence modes by which the regime could be tested, and N a novelty profile relative to the current archive. Theory-space navigation then becomes movement over such objects under a purpose object

$$\mathcal{P} = (\mathcal{O}_{\text{target}}, \mathcal{R}_{\text{desired}}, \mathcal{B}, \mathcal{H}),$$

with target obstruction family $\mathcal{O}_{\text{target}}$, desired capability gain $\mathcal{R}_{\text{desired}}$, ideational budget $\mathcal{B}$, and historical archive $\mathcal{H}$.

The AG contribution is that the present obstruction field tells the system where mathematical reinforcement would matter. The DTT contribution is that a candidate regime is acceptable only insofar as it can be entered into dependent judgment form with explicit contexts, clauses, and evidence routes. The AI contribution is controlled proposal over regimes that may be nonlocal relative to the present code and proof state. The implementation path should therefore include a theory frontier, purpose-conditioned search over candidate regimes, transportability probes, and archive traces explaining why a regime was rejected, deferred, federated, or elevated.

56.1 Theory-space navigation: moving over candidate regimes, bridges, and novelty profiles

Theory-space navigation should be implemented as search over a graph whose nodes are candidate regimes and whose edges are admissible moves such as refine-cover, change-coefficients, introduce-

bridge, add-constructor, federate-pack, or retire-regime. Each node should store its novelty profile, theorem inventory, bridge inventory, and the obstruction families it appears to relieve. Search then becomes a shortest-path or best-first problem over mathematically meaningful state rather than a list of topics.

This graph should support replay. If a regime was rejected earlier because of transport failure or proof cost, JuGeo should remember that and revisit it only when the relevant bridge inventory, support region, or purpose object has changed.

56.2 Purpose conditioning: target obstruction families, desired capability gains, and budget

Purpose conditioning should act as a type-level filter on theory search. A purpose object should specify the target obstruction families, the acceptable proof budget, the required witness forms, the maximum tolerated destabilization of existing certificates, and any deadlines imposed by the enclosing project phase. Candidate regimes that cannot speak to that purpose should not consume first-line theory budget no matter how elegant they are.

This is the main defense against ornamental mathematics. JuGeo should refuse to promote a regime whose novelty is high but whose transport into the active dependent goals is weak or whose proof burden is grossly out of scale with the declared purpose.

56.3 Mathematical areas as candidate semantic regimes: kinds, laws, bridges, and theorem templates

Each candidate regime should be compiled into a concrete schema

$$\Theta = (K, L, B, T, M, N, \Sigma_{\text{impl}}),$$

where K are carrier kinds, L the law family, B the bridge signatures into existing judgment form, T theorem templates, M admissible initial evidence modes, N the novelty record, and Σ_{impl} the implementation obligations induced by the regime. The last component is crucial: a candidate area is not ready for search until the system can say what new records, validators, or constructors it would force into code.

This keeps the chapter computational. “Category theory,” “resource semantics,” or “effect algebra” are not actionable by themselves. Only typed regime schemas are.

Chapter 57

Discovering new mathematical kinds from obstruction fields

This chapter should make a stronger claim than ordinary theorem mining. Sometimes the correct response to repeated failure is not another local lemma and not even another bridge theorem. It is the recognition that the current semantic language lacks the right kind of object. The objective here is therefore to move from obstruction fields to kind discovery: from repeated failure motifs to proposed carriers, laws, and dependent forms that were previously absent from the project.

A useful state object is

$$\mathcal{K} = (\mathcal{C}_{\mathrm{obs}}, \mathcal{D}_{\mathrm{def}}, \mathcal{R}_{\mathrm{cand}}, \Pi_{\mathrm{probe}}),$$

where $\mathcal{C}_{\mathrm{obs}}$ is a clustering of obstructions by support shape, witness structure, and failed-repair trajectory, $\mathcal{D}_{\mathrm{def}}$ is a deficiency analysis explaining what the present language cannot express cleanly, $\mathcal{R}_{\mathrm{cand}}$ is a family of candidate new kinds or regimes, and Π_{probe} is a set of structural or held-out tests for those candidates. A new mathematical kind should therefore be understood not as a grand announcement but as a disciplined response to a persistent representational deficiency.

The theorem burden is correspondingly specific. One wants cluster-faithfulness, so that the obstruction clusters used for kind discovery are not artifacts of poor normalization. One wants deficiency-traceability, so that the move from recurring failure to proposed kind can be inspected rather than mystified. One wants bridge-feasibility honesty, so that candidate kinds are not credited with authority before there is evidence that they can interact with the existing judgment language. The implementation path should include obstruction clustering, deficiency analysis, candidate-kind stores, bridge probes, and held-out simplification tests. Only then can "discovering new mathematical kinds" become an implementable research program rather than a gesture toward creativity.

57.1 Obstruction fields as evidence of missing mathematics rather than missing code

JuGeo should distinguish missing mathematics from missing code by testing the stability of an obstruction family under ordinary repairs. If a family persists after representative local edits, after cover refinement, and after ordinary bridge lookup, then the deficiency is likely representational rather than merely implementational. A useful diagnostic loop is: normalize the family, attempt local repair, refine the cover, change coefficients if applicable, and measure whether the same obstruction signature survives. Persistent survival is evidence that the current semantic language lacks an adequate kind.

This rule is important because otherwise every hard bug looks "deep." The system should reserve kind discovery for failures that remain nontrivial after the usual computational moves have been exhausted or shown insufficient.

57.2 Candidate new mathematical kinds: effect algebras, resource logics, relational type formers, and more

Candidate kinds should be proposed as typed interfaces, not just named areas of mathematics. For example, an effect-algebra candidate should specify a carrier of effects, a partial sum, an order or compatibility relation, and a bridge into obligation splitting or interference control. A resource-logic candidate should specify resource atoms, composition, framing laws, and transport rules into replay and mutation obligations. A relational type former candidate should specify binary or n-ary judgment carriers, restriction maps, and witness forms for equivalence or refinement obligations.

These candidate schemas let the system test whether a mathematical area is actually implementable inside JuGeo. If the area cannot yet be expressed as carriers, laws, bridges, and witness routes, it remains a research note rather than a regime candidate.

57.3 The obstruction-to-kind pipeline: clustering, deficiency analysis, and regime proposals

The obstruction-to-kind pipeline should be a concrete three-stage algorithm. First, cluster failures by support shape, witness schema, replay behavior, and failed-repair trajectory. Second, infer a deficiency description that states what cannot currently be expressed, transported, or glued. Third, generate candidate regime schemas and probe them on held-out obligations from the same cluster. Only proposals that reduce cluster pressure on held-out items should graduate to theory tickets.

A practical record for a pipeline run is

$$\mathfrak{p} = (\mathcal{C}_{\mathrm{obs}}, \mathcal{D}_{\mathrm{def}}, \mathcal{R}_{\mathrm{cand}}, \Pi_{\mathrm{probe}}, \mathcal{O}_{\mathrm{held}}).$$

This is the minimal object that turns "discovering new kinds" into an auditable computation.

57.4 Implementation consequences

Chapter 58

Optimal novelty search for mathematical purpose

Novelty search should be formulated against purpose, not against surprise alone. A mathematically unfamiliar idea is not automatically useful, and a mathematically conservative idea is not automatically uninteresting. The right question is whether a candidate regime would improve the reachable semantic future of the project relative to a declared purpose and under a manageable bridge and proof burden.

58.1 Novelty versus useful novelty: leverage, tractability, and semantic relevance

Novelty alone measures distance from the archive. Useful novelty measures distance that also buys leverage, has a plausible bridge, and does not explode proof burden. JuGeo should therefore explicitly reject proposals whose novelty is high but whose transportability or expected obstruction reduction is negligible. This is the mathematical analogue of avoiding impressive but useless abstractions.

Operationally, a proposal is usefully novel only if it simultaneously clears thresholds on predicted semantic leverage, admissible bridgeability, and proof tractability. The archive should record which proposed directions failed this filter so that the system learns what kinds of novelty are repeatedly seductive but unhelpful.

58.2 A purpose-conditioned novelty functional: leverage, transportability, proof cost, and risk

Let Θ be a candidate mathematical regime, let $\mathcal{P}$ be a purpose object, and let $\mathcal{S}$ be the current semantic state. A useful scoring functional is

$$\mathcal{J}_{\mathrm{nov}}(\Theta \mid \mathcal{P}, \mathcal{S}) = \alpha\,\mathrm{Leverage}(\Theta, \mathcal{P}, \mathcal{S}) + \beta\,\mathrm{Novelty}(\Theta, \mathcal{A}) + \gamma\,\mathrm{Transportability}(\Theta, \mathcal{S}) - \delta\,\mathrm{ProofCost}(\Theta) - \varepsilon\,\mathrm{IntegrationR}$$

where $\mathcal{A}$ is the archive of previously explored regimes. The point of this functional is not numerical exactness. The point is to force the search procedure to say why a mathematically novel direction is worth taking now, for this purpose, in this semantic state.

Each term should be interpreted operationally. Leverage estimates expected reduction in important obstruction mass or expected gain in capability. Novelty measures semantic distance from the current archive, not mere rhetorical unfamiliarity. Transportability estimates whether the

regime can be bridged into existing judgment form. Proof cost estimates the burden of making the proposal honest. Integration risk estimates the chance that the regime destabilizes established semantic capital. Redundancy penalizes discovery of what the current pack federation already knows how to express.

The evaluation metrics for this section should include predicted-versus-realized leverage, novelty-to-reuse ratio, transport success rate, and false-frontier rate, meaning how often the system promotes a mathematically attractive direction that later proves semantically idle. The implementation path is equally clear: a novelty scorer, an archive-sensitive distance model, transportability probes, proof-cost estimators, and failure analyses for proposals that were novel but not useful.

58.3 Why AG, DTT, and AI each matter in novelty search

AG matters because novelty search is local-to-global: one wants to know where a regime helps, what overlaps it changes, and whether its local benefits glue. DTT matters because every novelty claim must be linked to typed goals, witness forms, and residual obligations. AI matters because the search neighborhood of useful regimes is often nonlocal relative to the present code or theorem pack, so some proposal mechanism must jump to semantically adjacent but syntactically distant regions.

The roles are therefore complementary and non-substitutable. AG without AI localizes pressure but does not search broadly enough. AI without DTT proposes attractive but often inadmissible ideas. DTT without AG cannot explain how local regime changes propagate through the project. JuGeo should state this division of labor computationally, not only rhetorically.

58.4 Implementation consequences

Chapter 59

Implementation path for a mathematical discovery engine

If the preceding chapters are to guide implementation, they must culminate in a concrete subsystem description. The objective of this chapter is to specify the persistent objects, event flow, and validation discipline of a real mathematical-discovery engine. The engine should not be imagined as an inspirational appendix to the verifier. It should be specified as another routed subsystem of the same semantic center, consuming obstruction archives, theorem ecologies, treaty histories, and benchmark traces, and producing typed proposals about regimes, kinds, type constructors, bridges, and lemma portfolios.

A discovery record should have explicit form, for example

$$\mathfrak{d} = (\mathcal{P}, \Theta, \hat{\Delta}, \rho_{\text{evidence}}, \sigma, \mathcal{T}),$$

where $\mathcal{P}$ is the purpose object, Θ the proposed regime or asset, $\hat{\Delta}$ the predicted semantic delta, ρ_{evidence} the planned evidence route, σ the current status in the discovery pipeline, and $\mathcal{T}$ the provenance trace. Such records make it possible to ask serious questions later: which proposals were repeatedly useful, which failed because of transport issues, which failed because they were redundant, and which were attractive but overexpensive.

The theorem burden for the engine is modest but important. It should satisfy no-false-promotion for discovery proposals, provenance faithfulness for accepted regimes, and reuse honesty when later discovery work cites earlier archives. The evaluation burden is equally explicit: realized leverage, downstream reuse, bridge completion rate, proof-cost calibration, and failure analysis by rejection reason. The implementation path should therefore include proposal stores, archive traces, held-out replay tasks, and a discovery ledger that survives across projects.

59.1 A real mathematical-discovery subsystem: inputs, proposal records, and archive traces

A real discovery subsystem should ingest obstruction archives, bridge inventories, treaty histories, failed transports, and held-out pressure tasks. Its core durable objects should be a `DiscoveryProposal`, a `TheoryTicket`, a `TransportProbe`, and a `DiscoveryOutcome`. Each proposal should carry purpose, target obstruction family, candidate regime schema, expected leverage, proof route, and invalidation dependencies.

The event flow should also be explicit: mine pressure clusters, retrieve donor regimes, construct proposal records, run admissibility and transport probes, schedule validated theory tickets, and

archive realized outcomes against predictions. That is the minimal subsystem architecture required if discovery is to influence implementation rather than merely inspire it.

59.2 Evaluation and calibration: realized leverage, reuse, and failure analysis

Evaluation should compare predicted leverage to realized leverage over time. The system should measure reuse breadth of accepted proposals, fraction of held-out pressure points relieved, bridge completion rate, false-frontier rate, and proof-cost calibration. Every metric should be stratified by obstruction family and by candidate regime class so that JuGeo can learn which kinds of mathematical proposals are reliably worthwhile.

Calibration matters because a discovery engine that repeatedly overstates its future payback is worse than a cautious one. The archive therefore needs per-class calibration curves, not just aggregate success counts.

59.3 Theorem and falsification burden for mathematical discovery

The theorem burden for mathematical discovery should include no-false-promotion, provenance faithfulness, and bridge honesty. No-false-promotion says a proposal may change search priority but may not increase trust before downstream validation. Provenance faithfulness says later summaries preserve whether the proposal was conjectural, probed, accepted, or rejected. Bridge honesty says a regime may not be described as reusable across packs or purposes until the requisite transports are actually validated.

The falsification burden is equally sharp. The discovery story is weakened if proposed regimes rarely beat archive retrieval, if claimed leverage correlates weakly with realized leverage, or if accepted proposals destabilize existing certificates more often than their payback justifies. JuGeo should say this directly.

Chapter 60

Domain formation, new type constructors, and mathematical regime bootstrapping

This chapter should say plainly that the right discovery is often a new semantic region rather than a new theorem within the old region. The objective is to explain how obstruction pressure can justify domain formation, how dependent type theory enters as a search over new type constructors, and how a proposed regime should be bootstrapped under explicit caution before it is allowed to influence the rest of the system.

60.1 Domain formation: when the right discovery is a new semantic region rather than a new theorem

A new semantic region should be formed when recurring obstruction clusters cannot be explained by adding one more theorem to the existing basis because the existing carriers and judgment forms themselves are inadequate. The decision should depend on repeated deficiency descriptions, failed bridge attempts, and the extent to which a proposed region would create reusable structure across many obligations rather than solving only one. In implementation terms, domain formation should require a `ProvisionalDomain` record with explicit carriers, laws, witnesses, and affected support regions.

This makes region formation a controlled computational act. JuGeo should not create new domains merely because a topic sounds deep; it should do so when the obstruction archive repeatedly says the current language cannot speak the needed distinctions.

60.2 New type constructors: evidence of domain inadequacy and design pressure

Constructor search should begin from inadequacy reports, not from aesthetic preference. A candidate constructor must answer three questions: what recurring residual cannot be expressed cleanly now, what new witness form would the constructor enable, and what bridge obligations would it induce for existing packs? A practical search loop should generate a small family of candidate constructors, type-check them against representative goals, and reject any whose bridge burden dominates their forecast benefit.

This is the DTT face of mathematical discovery. New type constructors are justified when they change inhabitability and proof structure in a measurable way, not when they merely offer a nicer notation.

60.3 Regime bootstrapping: provisional carriers, bridge stubs, and experimental laws

Bootstrapping should proceed through provisional authority, not immediate foundation status. A new regime should begin with provisional carriers, bridge stubs, and experimental laws restricted to a held-out sandbox of obligations. Only after replay on held-out tasks, transport probes, and proof-cost measurement should the regime receive a stable seal and broader authority scope.

This staged process is what protects JuGeo from importing speculative mathematics directly into its trusted core. A provisional regime is allowed to guide search, but not to justify ordinary certificate promotion until it passes the same replay and bridge discipline as existing packs.

60.4 Implementation consequences

The system implied by this chapter would need explicit support for provisional domains, candidate type constructors, bridge stubs, and experimental law harnesses. That points toward future modules such as `provisional_domains.py`, `type_constructor_search.py`, `bridge_stubs.py`, and `experimental_laws.py`. The important point is not the filenames themselves. It is that the code should have a home for mathematical regime bootstrapping rather than forcing every new idea to masquerade as either a theorem or a comment.

Chapter 61

Theorem ecologies, lemma portfolios, and semantic compounding

The value of a theorem should not be measured only by the obligation it closes immediately. The objective of this chapter is to treat mathematical assets ecologically. Some results are local consumables. Others change the environment of future reasoning by altering what bridge routes become available, what normal forms become standard, what kinds of explanations compress well, and what obstruction families become rare. A mature theory of JuGeo therefore needs theorem ecologies, not only theorem counts.

61.1 Theorem ecologies: from local closure to changes in the reasoning environment

A theorem ecology should be modeled as a graph from theorem assets to the obstruction families, bridge routes, and normal forms they affect. Some theorems are local consumables that close one obligation. Others change the environment by shrinking many future proof obligations or by making new transports admissible. JuGeo should measure both kinds of effect.

This ecological view is one of the clearest improvements over traditional verification accounting, which usually counts only proved statements. JuGeo should count changed reasoning environments: new bridge routes, lower future obstruction rank, lower replay drag, and new kinds of admissible local sections.

61.2 Lemma portfolios: coordinated families of results with shared purpose

A lemma portfolio should be a purpose-indexed family of theorems, bridges, and normal-form results chosen because they work together on one recurring obstruction class. A portfolio record should list the obligations it covers, the bridges it unlocks, the packs it touches, and the replay conditions under which it remains valid. This is more useful than isolated theorem accumulation because engineering work usually needs clusters of compatible results rather than one elegant statement.

Portfolio selection should therefore optimize combined coverage and reuse, not individual theorem beauty. A small family of mutually supportive lemmas often buys more closure than one globally ambitious result.

61.3 Ecological metrics: reuse breadth, compounding, decay, and downstream compression

Ecological metrics should make theorem payback explicit. Reuse breadth counts how many distinct obstruction families or packs benefit. Compounding measures whether one accepted theorem makes later theorems cheaper to prove or easier to transport. Decay measures how often a theorem's useful scope shrinks under changing foundations or bridge seals. Downstream compression measures how many repeated failures, proof steps, or explanation fragments collapse into a smaller reusable law package.

These metrics are the right computational summary of semantic compounding. If JuGeo cannot measure them, then "theorem ecology" remains a metaphor rather than an implementation guide.

61.4 Implementation consequences

Chapter 62

Analogy, transfer, and purpose-preserving transport across mathematical areas

The search for new mathematics should often proceed by transport rather than by isolated invention. The objective of this chapter is to show how JuGeo should move from one mathematical area to another under explicit purpose-preservation constraints, and how it should tell the difference between a productive analogy, a nontransportable metaphor, and a case in which the failed transport itself is evidence that an intermediate regime is missing. This is the chapter in which "finding the right new area of mathematics for a purpose" becomes a controlled problem of cross-regime descent.

A useful state object is

$$\mathcal{T}_{\mathrm{ana}} = (\mathfrak{U}_{1,\bullet}, \mathfrak{U}_{2,\bullet}, \mathcal{O}_1, \mathcal{O}_2, \mathcal{P}, \mathcal{A}),$$

where $\mathfrak{U}_{1,\bullet}$ and $\mathfrak{U}_{2,\bullet}$ are covers or hypercovers of the source and target theory regions, $\mathcal{O}_1$ and $\mathcal{O}_2$ are the corresponding obstruction fields, $\mathcal{P}$ is the declared purpose, and $\mathcal{A}$ is the archive of earlier transfer attempts. A proposed analogy should be treated as a family of local transport maps over matched patches of these covers. It becomes mathematically serious only when the local transports satisfy Čech-style compatibility on overlaps and preserve the structures that matter for the purpose object. If the local maps solve pressure locally but fail to glue globally, the resulting cocycle should be archived as a signal either that the analogy is false or that a missing intermediate mathematics has been exposed.

A useful transfer score is

$$\mathcal{J}_{\mathrm{tr}}(\tau \mid \mathcal{P}, \mathcal{S}) = \alpha\,\mathrm{PurposePreservation}(\tau) + \beta\,\mathrm{ObstructionTransferGain}(\tau) + \gamma\,\mathrm{NoveltyRegion}(\tau, \mathcal{A}) + \eta\,\mathrm{DescentComp}$$

where PurposePreservation measures whether the transferred structure still serves the declared downstream goal, ObstructionTransferGain measures whether source-side mathematics attacks target-side obstruction mass, NoveltyRegion measures entry into underexplored but relevant semantic territory, DescentCompatibility measures overlap coherence, and the penalties track proof burden, coefficient mismatch, and distortion introduced by an overaggressive analogy. This makes analogy a scored search over candidate mathematical futures rather than a rhetorical flourish.

The AG role is therefore primary. Mathematical areas should be treated as regions in theory space with their own covers, candidate sheaves, and compatibility conditions. Transport may require a change of cover, a change of coefficient theory, or a provisional intermediate regime before descent succeeds. The DTT role is to make preservation claims typed and checkable: which dependent goals

are meant to survive, which invariants are preserved, which constructors are transported, and what new proof obligations arise. The AI role is controlled suggestion of donor areas, local transport maps, and intermediate regimes. AI is especially valuable here because useful donor regions are often not syntactically adjacent to the current code or theorem pack, but its proposals become authoritative only after purpose-linked preservation checks.

The implementation consequences are concrete. JuGeo should maintain an analogical donor index, local transport proposals with provenance, compatibility checks on overlaps and higher overlaps, failure archives for non-gluing transports, and replay tasks that test whether a claimed analogy actually improves closure on held-out target obligations. The theorem burden should include preservation honesty, purpose-linked scope honesty, and transport-faithfulness under recorded changes of cover or coefficients. That is how analogy becomes a disciplined route for discovering new mathematical areas rather than an invitation to confuse verbal resemblance with useful mathematics.

Chapter 63

From mathematical discovery to pack federation and implementation authority

A discovery process that never decides authority remains incomplete. The objective of this chapter is to define how accepted mathematics enters implementation governance. A new regime may be rejected, archived provisionally, federated as a specialized pack, or elevated into the common foundation. That choice should not be aesthetic. It should depend on reuse breadth, bridge burden, maintenance cost, and the extent to which the regime changes the ordinary language of judgment used elsewhere in the system.

A useful decision object is

$$\mathcal{R} = (\Theta, s_{\text{reuse}}, c_{\text{bridge}}, c_{\text{maint}}, a_{\text{scope}}, \kappa_{\text{evidence}}),$$

where Θ is the candidate regime, s_{reuse} its observed scope of reuse, c_{bridge} its bridge burden, c_{maint} its expected maintenance cost, a_{scope} the authority scope under consideration, and κ_{evidence} the evidence bundle supporting elevation. This makes regime elevation a typed judgment rather than an ad hoc architectural rewrite.

The AG contribution is visible in support breadth and overlap consequences. The DTT contribution is visible in the new judgment forms and constructor burden imposed by elevation. The AI contribution is limited to proposal and synthesis; it may suggest federation or foundation, but it does not decide authority. The implementation path should include elevation ledgers, federation compatibility checks, substrate-change audits, and explicit rejection reasons. That is how mathematical discovery acquires a disciplined route into implementation authority.

63.1 Authority choice: rejection, provisional archive, federation, or foundation

63.2 Federation versus foundation: scope, bridge burden, and semantic economy

63.3 Implementation consequences

Part IX

Methodology, Evaluation, and Limits

Chapter 64

Methodology

The dissertation needs a method strong enough to decide whether AG+DTT+AI judgment geometry is true, useful, or merely ornate. The right method is therefore not "build a system and report that it feels principled." It is to define a family of semantic state objects, compile every major theoretical claim into authority-bearing runtime objects, measure local-to-global behavior under controlled task families, and state in advance which observations would count against the thesis.

A useful methodological state object is

$$\mathcal{M} = (\mathcal{S}, \mathfrak{U}, \mathcal{H}_\bullet, \mathcal{J}, \mathcal{O}, \mathcal{E}, \mathcal{A}, \Pi, \Xi),$$

where $\mathcal{S}$ is the coordinate site, $\mathfrak{U}$ a chosen cover, $\mathcal{H}_\bullet$ an optional hypercover refinement, $\mathcal{J}$ the current judgment family, $\mathcal{O}$ the current obstruction field, $\mathcal{E}$ the routed evidence family, $\mathcal{A}$ the action log of proposals and repairs, Π the theorem burden attached to the subsystem under study, and Ξ the evaluation record. A methodology worthy of the dissertation is one that can replay transitions of $\mathcal{M}$, compare them across problem classes, and explain why a success or failure occurred in explicitly geometric terms.

64.1 A thesis needs a method, not only a design

The central methodological claim is that the theory should be evaluated as a theory of local-to-global semantic control. That means the unit of observation is not only a file, a prompt, a theorem, or a benchmark instance. The unit of observation is a judgment episode over a cover: a cover is chosen, local sections are elaborated, overlap laws are tested, obstructions are recorded, gluing is attempted, replay is performed after change, and the whole episode is archived with its support and provenance.

Definition 64.1 (Research episode)**.** *A research episode for JuGeo is a tuple*

$$\mathfrak{R} = (P, \mathfrak{U}, \mathcal{H}_\bullet, \mathcal{G}_0, \mathcal{G}_1, \Sigma, W, F),$$

where P is a task instance, $\mathfrak{U}$ and $\mathcal{H}_\bullet$ are the chosen cover and hypercover, $\mathcal{G}_0$ and $\mathcal{G}_1$ are the initial and terminal judgment states, Σ is the sequence of typed actions taken, W is the achieved witness material if any, and F is the failure signature if descent or verification does not close.

This definition is practical, not ceremonial. It says what a benchmark run must store if the dissertation is to support claims about verification, equivalence, bugfinding, generation, orchestration, and ideation in one language. Every one of those problem classes should be represented as a special case of an episode over local sections, overlaps, and obstructions.

The method should also insist that AG quantities be measured rather than merely invoked. At minimum, every episode should log:

- *cover quality*, meaning how well the chosen cover localizes obligations while keeping overlap burden interpretable;
- *overlap entropy*, meaning how diffusely obligations and exported laws are distributed across overlaps;
- *obstruction visibility*, meaning how early true global failures become visible as local obstruction signatures;
- *descent success rate*, meaning how often apparently compatible local families actually yield global sections;
- *support-sensitive reopening cost*, meaning how much of the judgment state reopens after a local change;
- *hypercover refinement gain*, meaning whether added higher-order structure improved closure enough to justify its cost.

The thesis becomes scientifically interesting only if these quantities move in predicted directions and only if their movement explains task outcomes better than flatter baselines such as filewise checking, prompt-only generation, or undifferentiated agent loops.

64.2 Formalization loop

The formalization loop should begin from a harsh rule: no major theoretical object may remain only a chapter noun. If the dissertation appeals to coordinates, covers, overlaps, treaties, obstructions, bridge theorems, or semantic futures, each such object must compile into a canonical record, admissible transitions on that record, and observable invariants.

Proposition 64.2 (Compilation burden for theoretical claims)**.** *For every major theoretical object X introduced in the dissertation, the implementation must provide:*

(i) a normalized internal representation of X;

(ii) at least one lawful construction route for X from lower-level artifacts;

(iii) at least one observable invariant or theorem schema attached to X;

(iv) at least one failure signature showing how X records nonclosure or residual uncertainty;

(v) at least one evaluation hook by which the practical value of X can be measured.

A theoretical object lacking this compilation route does not yet belong to the implementation-complete doctrine.

The formalization loop should therefore proceed in five passes. First, state the object in semantic terms. Second, define its authority center in code. Third, specify theorem obligations and non-theorems. Fourth, define the metrics by which its use can be evaluated. Fifth, archive positive and negative episodes involving the object. This is the discipline that prevents the dissertation from using algebraic geometry as atmosphere. AG enters exactly where covers, overlaps, hypercovers, obstruction classes, and descent witnesses become executable and measurable.

Some theorem burdens should be stated here. One wants normalization faithfulness for coordinates, restriction functoriality for transport, gluing soundness for descent routines, provenance

faithfulness for evidence routing, and obstruction traceability for failure reports. One also wants explicit non-theorems. In particular, the implementation should *not* claim that every recorded obstruction is a canonical cohomology class independent of modeling choice. Many recorded failures will be pragmatic Čech-style signatures relative to a chosen site and cover. That is still mathematically meaningful, but it is weaker than a claim of cover-independent invariance.

64.3 Implementation loop

The implementation loop is the place where the theory must become guidance for building JuGeo rather than a separate commentary on it. The loop should be the same across problem classes even when the local solvers differ:

1. Construct or refine a coordinate system for the task.
2. Synthesize a cover or hypercover that localizes obligations.
3. Elaborate local sections under typed evidence policies.
4. Materialize overlap objects and treaty candidates.
5. Attempt descent or record obstruction classes.
6. Reopen support-locally after changes or new evidence.
7. Archive the episode with enough detail for replay and ablation.

A good implementation loop should force each JuGeo capability into this discipline:

- *Verification*: elaborate unary judgments, discharge structural clauses, route semantic clauses, and determine whether the resulting section attains the target trust policy.
- *Equivalence*: build paired coordinates, elaborate relational sections, and test whether relation laws descend over overlaps of paired supports.
- *Bugfinding*: search for counterexamples, mismatch witnesses, or failed gluing data, then classify the result by obstruction signature rather than by error string alone.
- *Generation*: construct local inhabitants over a cover induced by intent, interfaces, or donor fragments, then attempt global assembly under replay.
- *Orchestration*: choose actions that maximize expected reduction in obstruction mass, overlap entropy, or support-sensitive future cost.
- *Ideation*: propose new covers, bridge theorems, treaty forms, or even new domain packs when recurring obstruction families suggest representational insufficiency.

The implementation loop should score candidate covers by an explicitly geometric objective, for example

$$Q_{\mathrm{cov}}(\mathfrak{U}) = \alpha L_{\mathrm{loc}} - \beta H_{\mathrm{ov}} - \gamma C_{\mathrm{supp}} - \delta R_{\mathrm{frag}},$$

where L_{loc} measures obligation localization quality, H_{ov} overlap entropy, C_{supp} predicted support-sensitive replay cost, and R_{frag} predicted fragility under revision. The point is not that this exact formula is final. The point is that cover choice must be auditable and optimizable rather than intuitive and forgotten.

A useful overlap-entropy statistic is

$$H_{\mathrm{ov}}(\mathfrak{U}) = -\sum_{\omega \in \Omega(\mathfrak{U})} p_\omega \log p_\omega,$$

where $\Omega(\mathfrak{U})$ is the family of overlap law-bundles induced by the cover and p_ω is the normalized obligation mass carried by overlap bundle ω. High overlap entropy means the system has scattered semantic responsibility too diffusely across intersections. In practice that should predict integration pain, treaty churn, and poor replay locality.

64.4 Evaluation loop

The evaluation loop should be episode-based, class-balanced, and replay-aware. The dissertation should not be satisfied by single-pass benchmark success, because the theory is explicitly about persistent semantic state under change. The same task families should be tested under initial construction, repair after localized perturbation, and refinement after hypercover change.

The evaluation corpus should be partitioned by problem class and by geometric difficulty. Difficulty should include not only size in lines of code or theorem statements but also cover depth, maximum overlap degree, entropy of exported laws, density of cross-region obligations, and frequency of previously observed obstruction motifs. This is where the AG account becomes explanatory rather than decorative: two tasks of similar textual size may differ radically in geometric complexity.

A minimal evaluation suite should include:

- held-out verification tasks with known clause truth values and realistic mixed evidence routes;
- held-out equivalence tasks with subtle relational mismatches;
- bugfinding tasks containing both seeded and organic integration failures;
- generation tasks where interfaces are known only partially at the start and must be stabilized through overlap work;
- orchestration tasks requiring action selection over multiple plausible repair or construction routes;
- ideation tasks in which repeated failures create pressure for new treaties, bridges, or mathematical regimes.

The evaluation loop should produce both clausewise and project-scale traces. Clausewise traces answer whether local laws were satisfied, challenged, or left residual. Project-scale traces answer whether a coherent global section was attained, how often it survived replay, and what geometric features predicted success or failure.

Hypercover refinement should itself be evaluated as an intervention. If $\mathcal{H}_\bullet \rightsquigarrow \mathcal{H}'_\bullet$ is a refinement, one should compute

$$\Delta_{\mathrm{refine}} = \big(D(\mathcal{H}'_\bullet) - D(\mathcal{H}_\bullet)\big) - \lambda\big(C(\mathcal{H}'_\bullet) - C(\mathcal{H}_\bullet)\big),$$

where D is descent success rate and C is total support-sensitive cost. A refinement is justified only when it improves closure enough to outweigh the extra bookkeeping and checking burden.

64.5 Falsification loop

The falsification loop must be stated as plainly as the thesis claims. The dissertation is weakened if any of the following patterns persist under serious implementation:

- cover-aware decomposition does not outperform simple file or module partitioning on descent success, replay locality, or human repair speed;
- overlap entropy fails to predict integration difficulty better than naive dependency counts;
- obstruction visibility remains low, so that true global failures are usually discovered only at the final integration stage;
- support-sensitive reopening is not materially smaller than coarse global invalidation;
- hypercover refinement rarely improves closure or explainability enough to justify its cost;
- cohomological failure signatures are unstable under benign refactoring, making the obstruction archive useless for replay or ideation;
- orchestration based on obstruction geometry fails to outperform flatter queueing or prompt-and-tool baselines;
- ideation guided by obstruction fields does not produce better future leverage than generic novelty search or unrestricted LLM brainstorming.

Remark 64.3 (Methodological non-theorems)**.** *The methodology should not promise a unique correct cover, a unique obstruction basis, or a solver-independent notion of semantic truth. It should also not promise that every persistent failure will be illuminated by refinement of the current hypercover. Some failures reflect genuinely missing mathematics, genuinely nonlocal semantics, or irreducible oracle dependence. Those are not embarrassments to be hidden. They are part of the point of the framework.*

The practical consequence is decisive: JuGeo should be built so that these falsification routes are cheap to run. A theory that can only survive when its critical metrics are absent is not a theory but a branding layer.

Chapter 65

Evaluation design

The evaluation design should treat JuGeo as a semantic machine whose claims come in different granularities. Some claims are local and clausewise. Others are overlap-level. Others are global descent claims. Still others are forecasts about future usefulness, as in orchestration and ideation. The evaluation architecture must therefore record outcomes at every one of these granularities and preserve the geometric relations among them.

65.1 Clausewise evaluation

Clausewise evaluation is the irreducible unit because every larger success claim factors through local judgments and overlap laws. A clause should not be scored only as pass or fail. It should be classified at least into the following states:

- *descended*: locally supported and globally integrated;
- *local-only*: supported in its local coordinate but not yet globally descended;
- *obstructed*: contradicted on overlap or by higher compatibility conditions;
- *residual*: not contradicted, but left with explicit unmet obligations;
- *misrouted*: evaluated under an inadmissible evidence route or wrong support.

For each clause family one should report ordinary correctness metrics, but also AG-aware ones. A basic descent success rate is

$$D_0 = \frac{\#\{\text{clause families yielding accepted global sections}\}}{\#\{\text{attempted clause families}\}}.$$

A basic obstruction visibility rate is

$$V_{\mathrm{obs}} = \frac{\#\{\text{true global failures signaled before final integration}\}}{\#\{\text{true global failures}\}}.$$

A basic obstruction localization score should compare predicted support of failure with the support ultimately implicated by replay or human confirmation. The right question is not merely whether the system found a bug, but whether it found the right region and the right overlap law family.

The evaluation should also log *cohomological failure signatures*. These need not be canonical cohomology classes in the strongest mathematical sense, but they should at least package degree, support shape, failed treaty family, witness pattern, and repair history:

$$\sigma_{\mathrm{fail}} = (k, \mathrm{supp}, \tau_{\mathrm{fail}}, w_{\mathrm{pat}}, h_{\mathrm{repair}}).$$

A mature evaluation design asks whether similar real failures recur with similar signatures and whether those signatures improve repair policy, hypercover refinement, or domain-pack growth.

65.2 Ablation philosophy

The ablation design should be faithful to the thesis structure. It is not enough to remove one model or one solver and call that an ablation of the theory. The right ablations remove semantic powers one at a time:

- remove cover synthesis and replace it with fixed filewise partitioning;
- remove hypercover refinement and force all integration through a flat cover;
- remove overlap objects and treaty memory;
- remove support-sensitive reopening and revert to coarse invalidation;
- remove obstruction archives and keep only error strings or failing traces;
- remove purpose-conditioned ideation and use generic proposal generation;
- remove mixed evidence routing and collapse support into a single untyped confidence score.

These ablations test which parts of the AG+DTT+AI story are actually doing work. The prediction is not that every AG-aware component helps equally on every task. The prediction is that the geometric components matter most on high-overlap, high-replay, long-horizon tasks, exactly where flatter systems lose coherence.

A particularly important ablation compares plain covers with hypercovers. If the theory is right, hypercovers should help most when the decomposition itself has internal depth and when significant failure mass lives on higher overlaps rather than only pairwise ones. If they do not, then either the tasks do not require that structure or the implementation is failing to expose it.

65.3 Calibration metrics

Calibration is essential because JuGeo mixes proof, solver evidence, runtime witness, and controlled semantic judgment. The evaluation should therefore ask not only whether claims are true, but whether the reported support profile is honest.

One wants at least four calibration families:

- *trust calibration*: predicted trust versus realized survival under replay, challenge, or stronger evidence;
- *support-scope calibration*: predicted support region versus actual region affected by confirmation or failure;
- *overlap-risk calibration*: predicted instability of overlaps versus later treaty churn or integration failure;
- *future-leverage calibration*: predicted semantic gain of an orchestration or ideation move versus realized reduction in obstruction mass or replay cost.

A useful support-scope calibration error is

$$\mathrm{Err}_{\mathrm{supp}} = \mathbb{E}\left[\frac{|\widehat{\mathrm{supp}}(x) \triangle \mathrm{supp}_{\mathrm{real}}(x)|}{\max(1, |\mathrm{supp}_{\mathrm{real}}(x)|)}\right],$$

where $\widehat{\mathrm{supp}}(x)$ is the predicted support of claim or failure x and $\triangle$ denotes symmetric difference. If this quantity stays high, then support-aware reasoning is not yet trustworthy enough to justify its architectural complexity.

For ideation and orchestration, calibration should be measured against realized semantic delta rather than only acceptance rate. A proposal that looks elegant but reduces no obstruction mass should count as miscalibrated even if a human reviewer liked it.

65.4 Project-scale metrics

Project-scale evaluation should ask whether the system can repeatedly attain and maintain global sections under realistic change. Relevant metrics include:

- time-to-first-global-section;
- survival of the global section under localized replay;
- reopened support volume after local change;
- average overlap entropy at stabilization;
- bridge reuse breadth and treaty reuse breadth;
- frequency of recurring obstruction signatures;
- fraction of closures requiring hypercover refinement;
- cumulative semantic work per unit of retained closure.

The geometric cost of change should be reported explicitly. If δ is a localized edit, one wants to measure

$$R_{\mathrm{supp}}(\delta) = \frac{\left|\mathrm{supp}(\mathrm{Reopen}(\delta))\right|}{\left|\mathrm{cl}(\mathrm{Star}_{\mathcal{H}}(\mathrm{supp}(\delta)))\right|},$$

up to recorded nonlocal dependencies. This ratio measures whether the implementation is actually exploiting semantic locality or merely naming it.

Project-scale metrics should also separate *local correctness without descent* from *global closure*. That distinction is one of the major contributions of the AG formulation. A system that produces many locally persuasive artifacts but poor global descent is not nearly as mature as one with slightly weaker local scores but strong gluing success and replay stability.

65.5 Human-facing evaluation

Human-facing evaluation should test whether the geometry makes the system more intelligible, not only more accurate. The user should be able to ask: where did the claim live, what overlap failed, what evidence routes were used, why did this reopening propagate that far, and what refinement would most likely help? If the system cannot answer these questions better than a conventional checker or agent log, then the thesis has not yet paid off at the interface level.

The relevant human-facing metrics are not generic satisfaction scores alone. One wants:

- time-to-locate the decisive obstruction;
- time-to-select the next intervention with highest realized leverage;
- error in user understanding of support scope;

- rate of false global confidence induced by summaries;
- user success in predicting which hypercover refinement or theorem-growth move will help, given the system's reports.

Proposition 65.1 (Evaluation burden for public honesty)**.** *A public-facing JuGeo report should never compress local support, overlap fragility, or residual obligations into a stronger global claim than the underlying semantic state warrants. Evaluation should therefore include adversarial summary tests in which readers inspect system reports and are measured for induced overconfidence, underconfidence, or support-scope misunderstanding.*

This is where the mixed-evidence story meets the AG story. A user can tolerate partial support. What they cannot tolerate is a system that hides whether a claim is a robust global section, a local section awaiting descent, or a fragile artifact resting on unstable overlaps.

Chapter 66

Complexity, scaling laws, and limits

Scaling needs its own theory because JuGeo is not trying merely to run larger proofs or longer generations. It is trying to control local-to-global semantic state over changing covers. The dominant costs therefore depend not only on artifact size, but on support geometry, overlap density, hypercover depth, evidence routing burden, and obstruction recurrence.

66.1 Why scaling needs its own theory

A naive cost model in terms of lines of code or raw clause count misses the main difficulty. Two projects of similar textual size can have radically different semantic difficulty because one admits a sparse cover with low overlap entropy while the other has dense nonlocal interactions and unstable treaties. The correct complexity variables are geometric.

A useful project complexity profile is

$$\mathfrak{C}(P) = \big(|\mathfrak{U}|, \deg_{\max}(\mathfrak{U}), H_{\mathrm{ov}}(\mathfrak{U}), d(\mathcal{H}_\bullet), \rho_{\mathrm{obs}}, \kappa_{\mathrm{bridge}}, \mu_{\mathrm{replay}}\big),$$

where $|\mathfrak{U}|$ is cover size, $\deg_{\max}$ maximum overlap degree, H_{ov} overlap entropy, $d(\mathcal{H}_\bullet)$ hypercover depth, ρ_{obs} obstruction density, κ_{bridge} bridge scarcity, and μ_{replay} average replay spread under local change. Scaling laws should be stated in terms of this profile rather than text size alone.

The theory predicts that JuGeo gains advantage when semantic locality is real, overlaps are explicit enough to be modeled, and recurring obstruction motifs can be clustered and reused. It predicts pain when semantics are effectively global, when support inference is unstable, or when overlap obligations are so dense that every local move becomes everyone else's boundary condition.

66.2 Support-sensitive cost model

A support-sensitive cost model should count work where semantic propagation actually occurs. If a change δ has support S, then the effective cost of responding to it should be modeled by the star of S in the active hypercover rather than by the whole project:

$$C_{\mathrm{supp}}(\delta) \approx \sum_{u \in \mathrm{Star}_{\mathcal{H}}(S)} c_{\mathrm{loc}}(u) + \sum_{\omega \in \partial\,\mathrm{Star}_{\mathcal{H}}(S)} c_{\mathrm{ov}}(\omega) + \sum_{k \geq 2} c_k N_k(S),$$

where c_{loc} is local elaboration cost, c_{ov} overlap reconciliation cost, and $N_k(S)$ counts higher-order compatibility checks induced by the change. The third term matters precisely because JuGeo is

not limited to pairwise integration; higher overlaps are where some of the most persistent and least visible failures live.

This model yields implementation-guiding predictions. If c_{ov} dominates, cover design is bad or overlap laws are too implicit. If higher-order terms dominate, the project may need hypercover refinement, treaty reorganization, or new bridge theorems. If local cost dominates but overlap cost stays low, the system is in the regime where JuGeo should scale well.

Remark 66.1 (Support-sensitive non-theorem)**.** *The support-sensitive model does not imply that every semantic effect is near its syntactic origin. Imports, reflective mutation, dynamic loading, ambient registries, and hidden resource protocols can create genuine long-range dependencies. The theory should therefore aim at explicit nonlocality accounting, not magical locality.*

66.3 Phase changes

The scaling picture should include phase changes rather than smooth extrapolation. There are thresholds beyond which the character of the system changes:

- a *cover phase change*, when adding one more region lowers local complexity more than it raises overlap burden;
- an *overlap phase change*, when overlap density crosses a threshold and local elaboration ceases to be the dominant cost;
- a *replay phase change*, when support-local changes begin reopening a macroscopic fraction of the project;
- a *theory-growth phase change*, when repeated obstructions make new mathematics cheaper than further local repair;
- an *fleet phase change*, when multiple concurrent proposals create more treaty churn than semantic progress.

A useful schematic law is that if overlap entropy and replay spread remain subcritical while bridge reuse grows, then effective cost can be near-linear in active support mass. If overlap entropy, replay spread, and obstruction density grow together, cost becomes superlinear and may become effectively global. The point of the theory is not to deny these bad regimes. It is to detect them early and respond by changing covers, treaties, or mathematical basis rather than merely adding more prompts or compute.

66.4 Scaling pathologies

Several pathologies should be stated plainly.

First, *cover thrashing*: the system repeatedly changes decomposition because no chosen cover yields stable overlaps. Second, *obstruction opacity*: failures exist but are visible only as late-stage global collapse rather than as interpretable local signatures. Third, *treaty churn*: interfaces stabilize too slowly because neighboring regions keep exporting incompatible summaries. Fourth, *hypercover explosion*: refinement reveals so many higher-order obligations that bookkeeping dominates semantic gain. Fifth, *archive drift*: recurring failures are not recognized as the same family because normalization is too weak. Sixth, *ideational overproduction*: the system proposes more theoretical moves than can be evaluated, creating semantic debt rather than leverage.

These are not mere implementation bugs. They are the concrete ways in which the AG picture can fail to help. They should therefore be benchmarked and reported, not hidden.

66.5 Scaling success

Scaling success should be defined more sharply than "it worked on a bigger repository." The mature success regime is one in which:

- support-local changes usually reopen only a bounded semantic neighborhood;
- overlap entropy is controlled by explicit interface laws rather than by hidden coupling;
- obstruction signatures recur in stable enough form to guide repair and theory growth;
- hypercover refinement is used selectively and pays for itself;
- theorem growth reduces future semantic work rather than accumulating ornamental mathematics;
- orchestration decisions can be justified by predicted reduction in obstruction mass or replay spread;
- human readers can understand why the system is stuck, not only that it is stuck.

One can then state a limited but meaningful scaling aspiration: JuGeo should exhibit *semantic compounding*. That means new treaties, bridge theorems, and archived obstruction families reduce future work on related tasks rather than only solving the present one. If that effect is absent, the system may still be a useful verifier, but it is not yet the judgment geometry claimed by the dissertation.

66.6 Threats to validity, non-theorems, and falsifiability conditions

Several non-theorems should be made explicit.

The dissertation should not claim:

- a canonical Grothendieck topology for all software work;
- cover-independent cohomology classes for every practical failure signal;
- polynomial-time global gluing in the general mixed-evidence setting;
- reliable semantic oracle judgments at arbitrary frontier depth;
- universal superiority over simpler toolchains on low-overlap or short-horizon tasks;
- a principled guarantee that hypercover refinement always improves matters;
- a guarantee that theorem growth is easier to validate than local repair.

Threats to validity follow directly. Benchmarks may overrepresent tasks with strong locality. Seeded bugs may be too geometrically clean. Human studies may conflate improved reporting with improved reasoning. Replay studies may underestimate ambient nonlocal effects. Ideation evaluations may reward plausible novelty instead of realized leverage. Each threat should be countered not by rhetoric but by held-out tasks, negative results, and archived failure analyses.

Proposition 66.2 (Falsifiability conditions for the scaling thesis)**.** *The scaling thesis is materially weakened if, across realistic held-out tasks,*

(i) support-sensitive reopening does not materially improve over coarse invalidation,

(ii) overlap entropy and obstruction visibility fail to predict integration outcomes,

(iii) hypercover refinement yields negligible leverage or intolerable cost,

(iv) recurring obstruction archives do not reduce future repair or theorem-growth effort,

(v) project-scale orchestration guided by semantic geometry fails to outperform flatter baselines.

These are strong conditions, and that is appropriate. A theory of scale that cannot articulate how it might fail is not ready for architectural authority.

Part X

Conclusion

Chapter 67

The mature picture

The mature picture should show JuGeo not as a pipeline that starts with intent and ends with a certificate, but as a cyclic semantic machine whose state evolves under repeated local-to-global pressure. In the mature system, generation, verification, repair, equivalence checking, orchestration, and ideation are not separate products. They are different modes of moving through one geometric judgment state.

67.1 The system should be cyclic, not pipeline-linear

A mature JuGeo run should be understood as repeated motion through the cycle

cover choice $\longrightarrow$ local elaboration $\longrightarrow$ overlap negotiation $\longrightarrow$ descent or obstruction $\longrightarrow$ repair or theory growth

The important point is that failure is not an exit condition. Failure is semantic information. A persistent obstruction may demand a new treaty, a different cover, a stronger bridge theorem, a domain-pack extension, or a change in what the system treats as the relevant support. That is exactly why algebraic geometry is core rather than optional. Without the AG layer, these transitions collapse into ad hoc retries.

In the mature picture, global artifacts are always provisional H^0-like achievements relative to a modeled site, and bugs, mismatches, regressions, or donor-integration failures are recorded as failures of descent rather than as miscellaneous bad events. Some of those failures are repaired locally. Others reveal that the current coordinate system is too coarse or the current law family too weak. Still others indicate genuinely missing mathematics. The system should know the difference well enough to change behavior.

67.2 From ideation to orchestration to proof and back

The mature interaction among subsystems should be explicit.

Ideation reads the obstruction field and proposes semantic futures: a better cover, a bridge theorem, a new treaty family, a new relational invariant, a new domain pack, or a different decomposition of authority. Orchestration then decides which of those moves is worth trying under budget, support scope, and predicted leverage. Proof, solving, runtime witness, and human review determine what evidence can actually validate the move. The result changes the judgment state, which in turn changes the obstruction field, which then feeds ideation again.

This cycle is where AG, DTT, and AI stop being separate chapter families. AG supplies the local-to-global geometry, the overlap graph, the obstruction field, and the refinement options. DTT

supplies contexts, obligation forms, dependent targets, and admissibility discipline. AI supplies controlled proposal, analogical transfer, and frontier search over futures not syntactically adjacent to the present state. Remove any one of the three and the cycle degrades: without AG the system loses locality and obstruction structure, without DTT it loses typed discipline, and without AI it loses sufficient search power at project scale.

A mature implementation should therefore expose explicit authority centers for coordinates, covers, hypercovers, overlap objects, treaties, obstruction archives, replay traces, proposal ledgers, bridge stores, and theorem-growth queues. If those objects do not exist, then the dissertation's mature picture has not yet reached code.

67.3 Why this could matter beyond JuGeo itself

What matters beyond JuGeo is not a claim that all software engineering is secretly algebraic geometry. The stronger and more defensible claim is narrower: when work is governed by locality, overlap, integration, replay, and persistent mismatch under change, local-to-global mathematics gives a better discipline than undifferentiated workflow state. That discipline may matter in verified programming, agent orchestration, scientific software, proof assistants, model-based systems, and perhaps in some forms of mathematical research assistance.

The external relevance should therefore be stated through transportable ideas rather than through grand analogy. Support-sensitive cost is transportable. Overlap entropy is transportable. Obstruction visibility is transportable. Hypercover refinement as a controlled response to layered integration failure is transportable. So is the rule that every semantic subsystem should state its theorem burden, evaluation burden, and non-theorems.

The dissertation should also preserve modesty here. The mature picture is not that JuGeo has found the one true formalism for all intelligent engineering. The mature picture is that it has identified a disciplined semantic architecture for domains where local work, overlap law, and failure of descent are structurally central.

67.4 The final practical consequence

The final practical consequence is severe and simple: JuGeo should be built as a judgment-geometry runtime, not as a verifier with a few extra agent features and not as a code generator with a few extra checks. Its runtime state should make covers, overlaps, supports, obstructions, hypercover refinements, and semantic futures first-class. Its public reports should distinguish local sections from global sections, residuals from contradictions, and stable closure from fragile treaty-dependent assembly. Its research program should measure whether those distinctions buy real leverage.

If the theory succeeds, the reward is large. Verification becomes more local without becoming narrow. Bugfinding becomes more explanatory. Equivalence becomes a relational gluing discipline rather than a bag of heuristics. Generation becomes section construction over a hypercover rather than long autocomplete. Orchestration becomes semantic control rather than workflow decoration. Ideation becomes forecast over future judgment states rather than elegant speculation.

If the theory fails, it should fail visibly: poor cover quality, high overlap entropy, low obstruction visibility, bad replay locality, unhelpful hypercover refinement, or ideation with no realized leverage. That explicit possibility of failure is part of the mature picture too.

The closing claim of the monograph should therefore be this: JuGeo is worth building only if algebraic geometry remains operationally central. The system should reason in terms of covers because projects are locally constructed; in terms of overlaps because interfaces are where most

real trouble lives; in terms of descent because local success is not global closure; and in terms of obstruction because the most important failures are the ones that persist after local cleverness. Everything else in the dissertation is downstream of that decision.